

KVERIF

新手上手与二次开发手册

先用三条命令跑通，再按任务查表

面向芯片验证工程师、回归平台开发者与 AI 工具集成人员。
按任务分类检索命令；每个命令提供参数、绝对路径示例和输出解释。

106 项 KDebug 操作	22 条 KCov 命令	11 类对外工具
--------------------	-----------------	-------------

版本 1.1
发布日期 2026-08-01
兼容基线 Verdi O-2018.09-SP2 / 普通用户 host

文档控制

项目	内容
文档用途	说明各工具能做什么、命令怎么写、参数怎么填、输出怎么看。
推荐调用方式	二次开发脚本只运行 tools 目录中的可执行文件，然后检查退出码并读取 JSON 输出。
不包含	内部 Tcl/NPI 函数调用、私有 SDK、真实凭据和许可证服务器信息。
内容来源	当前命令行实现、JSON 字段定义、仓库示例、README 和 VM 测试结果。
验证基线	普通用户 host, Verdi O-2018.09-SP2, VCS O-2018.09-1。

阅读方式

PDF 目录、书签和操作索引都可以点击。参数表中的“必填”来自程序实际使用的 JSON 字段定义。标有“VM 实测”的输出来自测试机；标有“标准输出示例”的内容只用于说明字段，实际数值取决于你的设计、FSDB 或 VDB。

阅读前先认识这些词

手册中的写法	通俗解释
action	操作名。它告诉 KDebug 要做什么，例如 module.inspect 表示查看模块。
target	要读取的设计、波形或已打开会话，例如 daidir、FSDB 或 session_id。
args	这项操作要用的具体参数，例如模块名、信号名和时间。
summary	简要结果。脚本通常先读这里，快速判断数量、状态或是否找到对象。
data	详细结果。需要逐个对象、逐笔事务或完整路径时读取这里。
truncated	结果是否因为数量上限被截短。需要完整清单时必须确认它是 false。
session	已经加载好设计或波形的后台进程。重复查询时不用每次重新打开大文件。

目录

文档控制	2
阅读前先认识这些词	2
第一次使用：10 分钟拿到真实结果	6
先运行这三条命令	6
每次任务会留下什么	6
第一次分析模块	6
第一次分析波形	7
从可运行脚本开始二次开发	7
按任务选择工具	7
快速决策	8
安装、环境与首次验证	8
普通用户 host 的推荐环境	8
复制到另一台机器前要知道什么	9
怎样调用工具、怎样读取结果	9
命令行和 JSON 两种写法	9
JSON 请求里各字段是什么意思	9
JSON 返回结果里各字段是什么意思	10
KDebug 命令和通用参数	10
常用快捷命令	10
快捷命令通用参数	11
功能参数汇总	11
输入、数量限制和输出设置	14
操作快速索引	15
本手册覆盖哪些常用 NPI 任务	17
演示：查找模块实例和实际参数值	18
演示：模块端口、方向、位宽与上下层连线	22
演示：列出普通实例、generate 实例和 primitive	23
演示：模块内部对象、过程与源码位置	23
演示：对照 RTL 设计对象和展开后的网表对象	25
演示：读取源码并在副本中修改设计	26
演示：NPI 能力、VCS 与 CRDB 跨层检查	27
演示：创建 Transaction/FSDB 并检查 Power 许可证	28
KDebug 操作参考	29
目录与内建	29
会话生命周期	32
设计与波形联合分析	36

静态设计与 Tcl NPI 模型	38
波形、协议与 FSDB 写出	67
KCov: 查询和推进覆盖率	111
公共查询参数	112
全部命令	113
KBit: 位宽和位运算	125
全部参数	125
KEntry: 拼接并拆解多拍数据	134
全部参数	135
KLoc: 从日志编号找到源码	137
全部参数	137
KSVA: 读懂和检查断言	139
全部参数	139
KBerif: 保存和查询验证环境知识	142
公共与命令参数	142
KEDA Runner: 按批准清单运行 EDA 命令	147
全部参数	147
复用长会话: Loop、MCP 与 LSF	149
启动 Loop 服务	149
Loop 客户端参数	150
给 AI 客户端使用的 MCP 入口	154
LSF 自检	156
脚本怎样调用 KVerif	156
Bash: 检查波形是否正常	156
csh: 判断是否还有未覆盖项	157
Perl: 多时刻批量采样	157
Python: 模块连接完整性	157
几个常用的组合用法	158
错误处理、安全与验证	158
脚本怎样判断成功还是失败	158
凭据与日志	158
Verdi 2018 VM 验证结果	158
附录	159
环境变量速查	159
常见状态和术语	160
KDebug 操作字母索引	161

本手册包含哪些内容 163

第一次使用：10 分钟拿到真实结果

先跑通，再查参数

第一次使用时，不需要先理解 action、target、args 或 NPI 对象。先运行环境检查和随库教程，看到 PASS 并找到 result.json、tool-response.json、replay.sh 后，再查询自己的项目。

先运行这三条命令

普通用户 host

```
su - host
export KVERIF_HOME=/home/host/kverif
export PATH="$KVERIF_HOME/tools:$PATH"

/home/host/kverif/tools/kverif doctor
/home/host/kverif/tools/kverif tutorial waveform
/home/host/kverif/tools/kverif tasks
```

命令	它检查什么	成功时看到什么
kverif doctor	用户、Python、KDebug、Verdi、Tcl NPI 和随库文件。	最后是 PASS，required failures 为 0。
kverif tutorial waveform	真实 Verdi 2018 FSDB 能否被读取和校验。	变化 5 次、unknown 0、Tutorial checks: PASS。
kverif tasks	当前提供哪些任务式入口。	列出模块检查、波形检查、教程和脚手架。

每次任务会留下什么

文件	给谁看	内容
result.json	验证人员和上层脚本	整理后的结论和常用字段。
tool-response.json	需要追查细节的人	KDebug 返回的原始 JSON，保留底层事实。
replay.sh	复现问题的人	本次实际使用的底层命令，可直接重放。
error.json	失败排查	失败码、提示、相关诊断和产物路径。只在失败时生成。

第一次分析模块

模块查询需要 VCS 生成的 simv.daidir。它适合回答模块实例在哪里、parameter 展开后是多少、端口方向和连接是什么。FSDB 只有波形值，不能替代 simv.daidir 做这类设计查询。

模块检查

```
# 先用随库的参数化模块完成教程
/home/host/kverif/tools/kverif doctor --require-vcs
/home/host/kverif/tools/kverif tutorial module-inspect --out /home/host/kverif_tutorial

# 再查询自己的设计
/home/host/kverif/tools/kverif inspect-module --input /data/project/build/simv.daidir --module tb_top.dut.u_core.u_alu --out
/data/project/reports/alu-module
```

教程期望输出

```
PASS module inspection completed

Module path: npi_fixture_top.u_alu
Definition: npi_fixture_alu
Parameters: 3
- WIDTH = 12
- BIAS = 1
- RESULT_WIDTH = 12 (localparam)
Ports: 3
- input lhs [12 bits]
- input rhs [12 bits]
- output result [12 bits]
Tutorial checks: PASS
```

module 要填完整实例路径

例如 tb_top.dut.u_core.u_alu，而不是 RTL 中的 ALU 定义名。只想检查生成的命令时，增加 --dry-run --show-command，不会启动 Verdi。

第一次分析波形

波形检查

```
/home/host/kverif/tools/kverif trace-signal --input /data/project/run/waves.fsdb --signal tb_top.dut.req_valid --begin 0ns --end 2us --format hex --max-rows 500 --out /data/project/reports/req-valid
```

输出示例

```
PASS waveform inspection completed

Signal: tb_top.dut.req_valid
Window: 0ns .. 2us
Changes: 14
Unknown values: 0
Truncated: no
```

终端只显示最常用的结论。脚本需要详细跳变时读取 tool-response.json；不要用 grep 从终端文字猜 JSON 字段。

从可运行脚本开始二次开发

生成并运行

```
/home/host/kverif/tools/kverif new signal-check --lang perl --out /data/project/tools/check_req_valid

cd /data/project/tools/check_req_valid
KVERIF_HOME=/home/host/kverif bash ./example.sh
```

--lang	生成入口	怎样处理结果
sh	sh/signal_health.sh	调用 KDebug，再用独立 JSON helper 生成结论。
csh	csh/signal_health.csh	适合传统 EDA 环境，仍只调用可执行文件。
perl	perl/signal_health.pl	只使用 Perl 核心模块处理工具输出。
python	py/signal_health.py	只使用 subprocess 和 json 标准库，不导入 KVerif。

二次开发边界没有改变

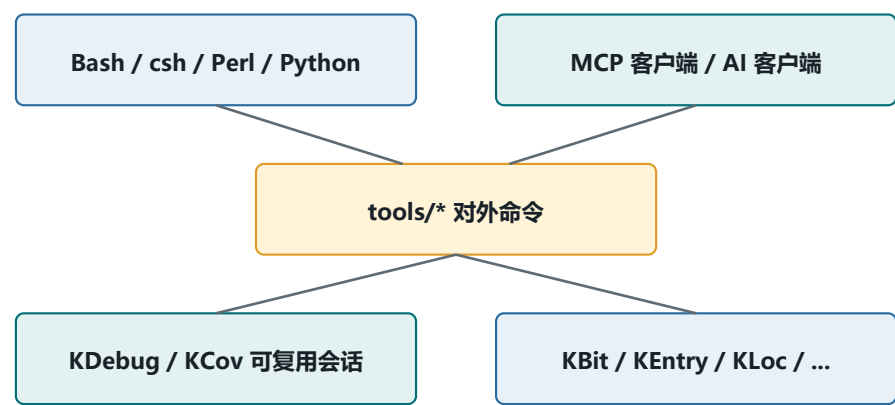
生成目录不包含 NPI Tcl、NPI C/C++ 头文件，也不导入 KVerif 内部模块。业务脚本只运行 tools 目录中的命令。

按任务选择工具

选工具时先看手里有什么文件，再看要解决什么问题。查设计或波形用 KDebug；查覆盖率用 KCov；位运算、日志定位和断言解释则分别使用后面的专用工具。

工具	任务域	主要输入	典型输出
KVerif Task CLI	新手自检与常见任务	FSDB、simv.daidir 或随库示例	直接结论、原始 JSON 和可重放命令
KDebug	查设计和波形	simv.daidir、FSDB、源码、CRDB	模块、端口、参数、驱动关系、波形值和协议事务
KCov	查覆盖率	VCS/Verdi VDB	覆盖率汇总、未覆盖项、设计层次和源码位置
KBit	做位运算	SV 数值或表达式	进制转换、切片、拼接、扩展和条件计算
KEntry	拼接并拆解多拍数据	YAML 字段配置和 JSONL 采样数据	完整数据、字段原始值和每个字段来自哪一拍
KLoc	从日志跳到源码	日志和 JSONL 位置表	源码文件、行号、上下文和高频报错位置
KSVA	理解 SVA 断言	SVA 源码	语法检查、通俗解释和结构化结果
KBerif	查询验证环境知识	项目文件和 .kberif 数据	知识条目、详细说明和按场景整理的上下文

工具	任务域	主要输入	典型输出
KEDA Runner	按批准清单运行 EDA 命令	.keda-runner.yaml	环境信息、实际命令、进程号和退出码
Loop Wrapper	让脚本复用长会话	Unix socket 和 JSONL	打开、查询和关闭 KDebug/KCov 会话
KVerif MCP	供 AI 客户端调用工具	MCP 标准输入/输出	把 KVerif 命令注册为 MCP 工具，可直接运行或提交到 LSF



KDebug/KCov 内部调用 Tcl NPI；二次开发脚本只运行命令并读取 JSON。

快速决策

问题	首选命令	下一步
某个时刻信号是什么值	kdebug value-at	如果还要查为什么是这个值，再用 trace.active_driver。
谁在驱动这个信号	kdebug trace-driver	如果要结合波形判断当时哪条赋值真正生效，用 active-driver-chain。
模块在哪里例化，参数是多少	kdebug action module.inspect	需要分类列出端口或实例时，用 module.objects。
哪些覆盖项还没命中	kcov cov-holes	再用 source-map 找到对应源码，并比较回归前后的变化。
计算位宽、符号或切片	kbit eval	交给 KBit 计算，避免脚本自己处理符号扩展。
日志中的 loc_id 对应哪行源码	kloc resolve/context	先找高频位置时使用 stats。
一条断言到底检查什么	ksva explain	脚本需要结构化结果时使用 parse。

安装、环境与首次验证

普通用户 host 的推荐环境

Bash / Zsh

```
ssh host@192.168.31.116
export KVERIF_HOME=/home/host/kverif
export PATH="$KVERIF_HOME/tools:$PATH"
export PYTHON=/usr/local/bin/python3.8
export VERDI_HOME=/home/synopsys/verdi/Verdi_0-2018.09-SP2
export VCS_HOME=/home/synopsys/vcs/0-2018.09-SP2

/home/host/kverif/tools/kverif doctor
/home/host/kverif/tools/kverif tutorial waveform
/home/host/kverif/tools/kverif tasks
```


csch / tcsh

```
setenv KVERIF_HOME /home/host/kverif
setenv PATH "$KVERIF_HOME/tools:$PATH"
setenv VERDI_HOME /home/synopsys/verdi/Verdi_0-2018.09-SP2
setenv PYTHON /usr/local/bin/python3.8
setenv VCS_HOME /home/synopsys/vcs/0-2018.09-SP2

/home/host/kverif/tools/kverif doctor
/home/host/kverif/tools/kverif tutorial waveform
```

复制 to 另一台机器前要知道什么

不能只复制一个 kdebug 文件
tools/kdebug 只是命令入口，运行时还需要同一版本的 KVerif 程序、Verdi/VCS、Tcl NPI 库和相应许可证。纯 Python 工具也需要兼容的 Python。迁移到新机器时应复制完整 KVerif 目录；用户脚本仍只调用 tools 目录中的命令。

检查	命令	期望
新手环境检查	/home/host/kverif/tools/kverif doctor	required failures 为 0。
真实 FSDB 教程	... kverif tutorial waveform	Tutorial checks: PASS。
KDebug 操作列表	/home/host/kverif/tools/kdebug actions --json	ok=true，并列出当前支持的操作。
KDebug NPI	... kdebug --json action npi.capabilities	列出当前 Verdi 能找到哪些 NPI 命令。
KCov 假数据	... kcov cov-holes --vdb fake --fake --json	无需 EDA 许可证即可检查命令行是否可用。
KBit	... kbit conv 8'hff --json	width=8, unsigned=255。
LSF/MCP	... kverif-lsf-doctor --fake	process_started/actions_ok/quit_ok。

怎样调用工具、怎样读取结果

命令行和 JSON 两种写法

人在终端里通常直接写命令行参数；脚本需要保存或批量生成请求时，也可以提交完整 JSON。--json 只表示“把结果输出为 JSON”，不是让你提供一个名为 json 的文件。

对照示例

```
# 同一查询的参数式写法
/home/host/kverif/tools/kdebug --json value-at \
--fsdb /proj/out/waves.fsdb --signal top.clk --time 10ns

# 完整 JSON 从 stdin 输入；最后一个 - 表示 stdin
printf '%s'
'{"api_version":"kdebug.v1","action":"value.at","target":{"fsdb":"/proj/out/waves.fsdb"},"args":{"signal":"top.clk","time":"10ns"}}' \
| /home/host/kverif/tools/kdebug --json -
```

JSON 请求里各字段是什么意思

字段	要求	含义
api_version	必填	kdebug.v1
request_id	可选	由调用脚本填写，用来把请求和返回结果对应起来。
action	必填	要执行的操作名。
target	按操作填写	输入文件或会话，例如 daidir、fsdb、session_id。
args	按操作填写	模块名、信号名、时间等功能参数。
limits	可选	最多返回多少项、最多查多深等数量限制。
output	可选	输出格式、详细程度等设置。

JSON 返回结果里各字段是什么意思

字段	怎样使用
ok	第一步先看它。true 表示成功；false 表示失败。
action / request_id	确认这份结果对应哪项操作、哪次请求。
summary	简要结果，例如找到多少项、返回多少项、是否截断。
data	详细结果，例如对象列表、依赖图或事务列表。
findings / warnings	补充说明。即使 ok=true，也要留意这里是否提示结果不完整。
error.code / message	失败原因。脚本应优先按 error.code 分类，不要只匹配文字。
meta	运行后端、耗时和截断状态等附加信息。

KDebug 命令和通用参数

常用快捷命令

命令	用途	参数
actions	列出当前支持的操作	[--json]
schema	查看某项操作可输入和返回哪些字段	--action NAME [--kind request response] [--json]
session-open	打开复用会话	--name ID [--daidir PATH] [--fsdb PATH] [--transport uds tcp file]
session-list	列出会话	[--json]
session-close	关闭会话	--session ID
session-doctor	诊断会话	[--session ID]
session-kill	强制终止会话	--session ID
session-gc	清理失效会话	[--arg key=value]
scope-list	列出 FSDB 中的层次和信号	--fsdb FILE --session ID [--path SCOPE] [--max-rows N]
value-at	读取单信号值	--fsdb FILE --session ID --signal NAME --time TIME [--format hex]
value-batch	同一时刻读取多信号	--fsdb FILE --session ID --signal NAME... --time TIME
trace-driver	查询静态驱动	--daidir DIR --session ID --signal NAME [--include-source]
trace-graph	构建依赖图	--daidir DIR --session ID --signal NAME [--max-depth N]
source-context	读取源码上下文	--arg file=PATH --arg line=N
active-driver	定位指定时刻生效驱动	--session ID --daidir DIR --fsdb FILE --signal NAME --time TIME
active-driver-chain	追踪生效驱动链	--session ID --daidir DIR --fsdb FILE --signal NAME --time TIME
action	按操作名调用任意 KDebug 功能	ACTION [--session ID] [--daidir DIR] [--fsdb FILE] [--arg key=value]

快捷命令通用参数

参数	填写内容	写入的 JSON 字段
--json	无	输出原始 JSON；适合脚本、CI 和字段完整访问。
--text / --kout	无	强制输出 KOUT 结构化文本。
--session / --session-id	ID	同时写入 target.session_id 和 args.session_id。
--name	字符串	写入 args.name。
--daidir	路径	写入 target.daidir。
--fsdb	路径	写入 target.fsdb。
--signal	层次名	写入 args.signal；value-batch 中可重复形成 signals 数组。
--signals	逗号列表	写入 args.signals 数组。
--time / --at	时间	写入 args.time；active-driver 系列写入 requested_time。
--requested-time	时间	显式写入 args.requested_time。
--format / --radix	格式	同时写入 args.format 和 args.radix。
--path	路径	写入 args.path。
--scope	层次	写入 args.scope；scope-list 同时映射为 args.path。
--kind	字符串	写入 args.kind。
--action	action	写入 args.action，常用于 schema。
--transport	uds tcp file	写入 args.transport。
--host	主机	TCP transport 的客户端可达地址。
--bind-host	主机	TCP daemon 监听地址。
--port	整数	TCP 端口；0 表示自动选择。
--include-source	无	写入 args.include_source=true。
--include-trace	无	写入 args.include_trace=true。
--include-control	无	写入 args.include_control=true。
--include-raw	无	写入 args.include_raw=true。
--verbosity	compact full debug	写入 output.verbosity。
--max-rows	整数	写入 limits.max_rows。
--max-results / --max-items	整数	写入 limits.max_results 和 limits.max_items。
--max-depth	整数	写入 limits.max_depth 和 args.max_depth。
--timeout-ms	整数	写入 limits.timeout_ms。
--arg	key=value	可重复；按点号写入嵌套 args，自动识别布尔、null、数字和 JSON。
--target	key=value	可重复；按点号写入 target。
--limit	key=value	可重复；按点号写入 limits。
--output	key=value	可重复；按点号写入 output。

功能参数汇总

下表汇总所有操作可能用到的功能参数。每项操作只需要其中一部分；具体要填哪些参数，请看对应操作页面。

字段	类型	要求	含义	示例
args.action	string	可选	要执行的操作名，例如 module.inspect。	signal.statistics
args.aggregate_only	boolean	可选	只返回聚合统计，不返回逐项明细。	True
args.begin	string	可选	查询窗口起点，例如 0ns。	0ns
args.begin_time	integer	可选	查询窗口起点的完整字段名。	0
args.clk_period	string	可选	显式指定时钟周期，用于采样或时间换算。	10ns
args.clock	string	至少一项必填	采样时钟的完整层次信号名。	top.clk
args.cnt	string	至少一项必填	要统计的计数器信号。若计数器分成高、低两段，也可用 hi/lo 指定两个信号。	{top.cnt_hi,top.cnt_lo}

字段	类型	要求	含义	示例
args.conditions	array	至少一项必填	待验证条件数组，每项描述信号、表达式和期望值。	[{"signal":"top.u.valid","op":"==","value":"h1"}]
args.config_path	string	至少一项必填	协议、事件、stream 或 position 映射配置文件路径。	wave_view.json
args.content	string	至少一项必填	写入或替换的文本内容。	assign ready = valid && enable;
args.crdب	string	至少一项必填	CRDB 数据库或映射文件路径。	/data/build/dut.crdب
args.definition	string	至少一项必填	模块定义名，用于反查全部实例。	npi_fixture_alu
args.dynamic	boolean	可选	是否读取随时间变化的关系或运行时数值。	True
args.end	string	可选	查询窗口终点，例如 1us。	100ns
args.end_time_delta	integer	可选	相对起点的结束时间偏移。	100
args.expr	string	至少一项必填	待解析、求值或查找事件的表达式。	valid && !ready
args.file	string	至少一项必填	源文件、配置文件或输入文件路径。	rtl/foo.sv
args.format	string	可选	值显示格式，例如 hex、bin、dec 或 sv。	tsv
args.from_signal	string	至少一项必填	路径搜索起点信号。	top.u.valid
args.include_accesses	boolean	可选	在输出中包含每一笔 APB/AXI 访问的明细。	False
args.include_preview	boolean	可选	在输出中包含写出文件或修改结果预览。	True
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False
args.instance	string	至少一项必填	设计实例的完整层次名。	top.u
args.interface	string	至少一项必填	SystemVerilog interface 实例名。	if0
args.kind	string	至少一项必填	要列出的对象类别，例如 ports、parameters 或 instances。	request
args.level	string	可选	层级、深度或输出详细级别。	RTL
args.limit	integer	可选	最多返回多少条结果。部分操作使用这个简写字段。	32
args.line	integer	至少一项必填	从 1 开始的源文件行号。	123
args.max_edges	integer	可选	依赖图最多返回的边数。	256
args.max_examples	integer	可选	摘要中最多保留的代表性示例数。	5
args.max_items	integer	可选	最多返回的对象、事务或结果项数。	20
args.max_samples	integer	可选	最多返回的波形采样点数。	1000
args.mode	string	可选	查询方式，例如只看开头、结尾、摘要或未覆盖项。具体可选值由操作决定。	replace
args.module	string	至少一项必填	通常填写模块实例的完整层次名；若某项操作需要模块定义名，本页会单独说明。	npi_fixture_top.u_alu
args.name	string	至少一项必填	会话、配置、游标、列表或对象的名称。	if0
args.net_type	string	可选	新增或筛选 net 时使用的类型。	npiDmNetWire
args.new_name	string	至少一项必填	克隆后模块或对象的新名称。	alu_debug
args.object_type	string	至少一项必填	NPI 模型中的对象类型。	npiParameter
args.op	string	至少一项必填	游标或配置操作类型。	begin
args.operations	array	可选	批量执行的操作数组。	[{"op":"scope","type":"npiFsdبScopeSvModule","name":"top"}, {"op":"scope","type":"npiFsdبScopeSvModule","name":"u_a"}, {"op":"up"}, {"op":"scope","type":"npiFsdبScopeSvModule","name":...}]
args.output	string	至少一项必填	操作生成的文件路径；某些操作也用它保存一组输出设置。	/data/out/top.patched.sv
args.output_dir	string	至少一项必填	生成或修改结果的输出目录。	/data/out/dm_add_net
args.output_file	string	可选	导出结果文件路径。	/tmp/req_stream.tsv
args.output_prefix	string	可选	批量导出文件名前缀。	/tmp/if0_axi
args.overwrite	boolean	可选	是否允许覆盖已有输出文件。	False
args.packed_left	integer	可选	新增 packed 对象的左边界。	7

字段	类型	要求	含义	示例
args.packed_right	integer	可选	新增 packed 对象的右边界。	0
args.path	string	可选	scope 路径、配置路径或目标对象路径。	
args.port	string	至少一项必填	待追踪的端口完整层次名。	ready
args.query	string	至少一项必填	数据流操作或通用操作使用的查询条件。	summary
args.rc_path	string	至少一项必填	生成的 Verdi RC 文件路径。	signal.rc
args.ready	string	至少一项必填	ready 握手信号完整层次名。	top.u.ready
args.relation_type	string	至少一项必填	要查询的对象关系，例如 npiParamConn 表示端口的上层连接。	npiParamConn
args.relations	array	可选	transaction writer 中的关系数组。	[{"relation": "npiParamConnRelParentChild", "master": 0, "slave": 1}]
args.requested_time	string	至少一项必填	联合分析要求的波形时间。	120ns
args.requests	array	至少一项必填	batch 操作中要依次执行的子请求列表。	[{"api_version": "kdebug.v1", "action": "schema"}]
args.sampling	string	可选	采样策略或采样配置对象。	posedge
args.scope	null	至少一项必填	FSDB 或 coverage 层次 scope。	按项目填写
args.sections	array	可选	module.inspect 要返回哪些部分，例如 ports、parameters 或 nets。	["instances", "parameters", "ports", "io", "nets", "variables"]
args.session_id	string	至少一项必填	已打开会话的 ID 或别名。	case_a
args.signal	string	至少一项必填	单个信号的完整层次名。	top.u.ready
args.signals	array	至少一项必填	多个信号完整层次名组成的数组。	["top.u.valid", "top.u.ready"]
args.start	string	可选	查询窗口起点，部分 action 使用 start 命名。	0ns
args.stream	string	至少一项必填	命名 stream 或 transaction stream 定义。	req_stream
args.streams	array	至少一项必填	要加载的 stream 配置数组。	[{"name": "req_stream", "clock": "top.clk", "clock_edge": "posedge", "reset": "!top.rst_n", "vld": "top.req_vld", "rdy": "top.req_rdy", "sop": "top.req_sop", "eop": "top.req_eop", "data_fields": {" ...
args.time	string	至少一项必填	波形时间，例如 100ns。	100ns
args.time_a	string	至少一项必填	比较的第一个时间点。	100ns
args.time_b	string	至少一项必填	比较的第二个时间点。	200ns
args.time_range	object	至少一项必填	包含 begin/end 的时间范围对象。	{"begin": "0ns", "end": "1us"}
args.to_signal	string	至少一项必填	路径搜索终点信号。	top.u.ready
args.transactions	array	至少一项必填	transaction writer 要写入的事务数组。	[{"start_delta": 10, "duration": 20, "type": "npiParamConnTransTransaction", "label": "req0", "tags": ["read"]}, {"start_delta": 5, "duration": 10, "type": "npiParamConnTransTransaction", "label": "rsp0"}]
args.unit	string	可选	无显式单位数值使用的默认时间单位。	1ns
args.valid	string	至少一项必填	valid 握手信号完整层次名。	top.u.valid
args.verbose	boolean	可选	是否返回更详细的诊断信息。	False
args.vld	object	至少一项必填	计数器统计使用的有效采样条件。	{"expr": "valid && ready", "signals": {"valid": "top.valid", "ready": "top.ready"}}

输入、数量限制和输出设置

target 字段	含义
target.daidir	VCS 编译展开后生成的设计数据库目录，通常为 simv.daidir。
target.defines	编译宏数组。
target.filelist	设计 filelist 路径，用于源码加载模式。
target.fsdb	真实 FSDB 波形文件路径。
target.mode	输入类型：只读设计、只读波形，或同时读取两者。
target.name	目标资源的可读名称。
target.session_id	已打开的 KDebug 会话 ID。
target.top	顶层模块名。
target.upf	UPF 文件路径。
target.upf_version	UPF 版本，例如 2.0。
target.workdir	工具启动和相对路径解析目录。

limits 字段	含义
limits.max_rows	最多返回的表格或明细行数。
limits.max_events	最多返回的事件数。
limits.max_samples	最多返回的波形采样点数。
limits.max_depth	trace、scope 或关系遍历的最大深度。
limits.max_paths	路径搜索最多返回的路径数。
limits.max_results	通用结果条数上限。
limits.max_items	通用对象或事务条数上限。
limits.timeout_ms	显式请求时限；仅在对应该后端支持时生效。

output 字段	含义
output.format	输出格式：json 适合脚本读取，kout 适合直接在终端查看。
output.verbosity	输出详细程度：compact、full 或 debug。
output.pretty	是否缩进 JSON。
output.include_suggestions	错误或空结果时是否附带下一步建议。

操作快速索引

操作名	分类	状态	需要的输入
actions	builtin	stable	none
apb.config.list	waveform	stable	waveform
apb.config.load	waveform	stable	waveform
apb.cursor	waveform	stable	waveform
apb.query	waveform	stable	waveform
apb.transfer_window	waveform	experimental	waveform
axi.analysis	waveform	stable	waveform
axi.export	waveform	stable	waveform
axi.channel_stall	waveform	experimental	waveform
axi.config.list	waveform	stable	waveform
axi.config.load	waveform	stable	waveform
axi.cursor	waveform	stable	waveform
axi.latency_outlier	waveform	experimental	waveform
axi.outstanding_timeline	waveform	experimental	waveform
axi.query	waveform	stable	waveform
axi.request_response_pair	waveform	experimental	waveform
batch	builtin	stable	none
control.explain	design	stable	design
counter.explain	design	stable	design
counter.statistics	waveform	stable	waveform
cursor.delete	waveform	stable	waveform
cursor.get	waveform	stable	waveform
cursor.list	waveform	stable	waveform
cursor.set	waveform	stable	waveform
cursor.use	waveform	stable	waveform
detect_anomaly	waveform	stable	waveform
event.config.list	waveform	stable	waveform
event.config.load	waveform	stable	waveform
event.export	waveform	stable	waveform
event.find	waveform	stable	waveform
expr.eval_at	waveform	stable	waveform
expr.normalize	design	stable	none
fsm.explain	design	stable	design
handshake.inspect	waveform	stable	waveform
inspect_signal	waveform	deprecated	waveform
instance.map	design	stable	design
interface.resolve	design	stable	design
list.add	waveform	stable	waveform
list.create	waveform	stable	waveform
list.delete	waveform	stable	waveform

操作名	分类	状态	需要的输入
list.diff	waveform	stable	waveform
list.show	waveform	stable	waveform
list.validate	waveform	stable	waveform
list.value_at	waveform	stable	waveform
port.trace	design	stable	design
procedural.assignment	design	stable	design
sampled_pulse.inspect	waveform	experimental	waveform
schema	builtin	stable	none
scope.list	waveform	stable	waveform
sequential.update	design	stable	design
session.close	session	stable	session
session.doctor	session	stable	session
session.gc	session	stable	none
session.kill	session	stable	session
session.list	session	stable	session
session.open	session	stable	any
signal.canonicalize	design	stable	design
signal.changes	waveform	stable	waveform
signal.resolve	design	stable	design
signal.stability	waveform	stable	waveform
signal.statistics	waveform	stable	waveform
signal.trend	waveform	stable	waveform
source.context	design	stable	none
trace.active_driver	combined	stable	any
trace.active_driver_chain	combined	stable	any
trace.driver	design	stable	design
trace.expand	design	stable	design
trace.explain	design	stable	design
trace.graph	design	stable	design
trace.load	design	stable	design
trace.path	design	stable	design
trace.query	design	stable	design
rc.generate	waveform	stable	waveform
value.at	waveform	stable	waveform
value.batch_at	waveform	stable	waveform
signal.scan	waveform	stable	waveform
verify.conditions	waveform	stable	waveform
window.verify	waveform	stable	waveform
stream.config.load	waveform	stable	waveform
stream.config.list	waveform	stable	waveform
stream.show	waveform	stable	waveform
stream.validate	waveform	stable	waveform

操作名	分类	状态	需要的输入
stream.query	waveform	stable	waveform
stream.export	waveform	stable	waveform
npi.capabilities	builtin	experimental	none
language.resolve	design	experimental	design
language.iterate	design	experimental	design
language.relate	design	experimental	design
language.value	design	experimental	design
module.objects	design	experimental	design
module.find_instances	design	experimental	design
module.inspect	design	experimental	design
netlist.resolve	design	experimental	design
netlist.iterate	design	experimental	design
text.line	design	experimental	design
text.words	design	experimental	design
text.replace_line	design	experimental	design
dm.add_net	design	experimental	design
dm.clone_module	design	experimental	design
vcs.summary	design	experimental	design
power.resolve	design	experimental	design
power.list	design	experimental	design
crdb.resolve	design	experimental	none
crdb.correlates	design	experimental	none
transaction.writer.create	waveform	experimental	none
fsdb.writer.create_scope	waveform	experimental	none

本版操作数量
builtin=4 / combined=2 / design=38 / session=6 / waveform=56

本手册覆盖哪些常用 NPI 任务

实际工作通常要连续执行几项操作。例如，要读取某个实例的实际参数值，需要先根据模块定义找到实例路径，再查询这个实例的 parameter。下表列出本手册已经完整演示的常用任务。

本版检查范围
手册覆盖 22 项 NPI 操作、15 类模块对象和 8 个常用任务。每个任务都有可直接执行的命令、关键输出、字段说明和 VM 测试结果。

用户任务	操作顺序	module.objects 类别	VM 测试情况
定义到实例与实际参数	module.find_instances -> module.inspect -> module.objects -> language.value	parameters	Verdi 2018 VM 10/10 PASS
端口、方向、位宽与上下层连线	module.inspect -> module.objects -> language.relate	ports、io	ports/io/relate 各 10/10 PASS
普通实例、generate 实例和 primitive 层次	module.objects -> module.find_instances	instances、generate_scopes、instances_in_generate、primitives	4 kind 各 10/10 PASS
内部对象、过程与源码位置	module.objects -> text.line	nets、variables、functions、tasks、continuous_assignments、always_processes、initial_processes、language_interfaces	8 kind 与 text.line 各 10/10 PASS

用户任务	操作顺序	module.objects 类别	VM 测试情况
对照 RTL 对象和展开后的网表对象	language.resolve -> language.iterate -> language.relate -> netlist.resolve -> netlist.iterate	不适用	5 项操作各 10/10 PASS
读取源码并在副本中修改设计	text.line -> text.words -> text.replace_line -> dm.add_net -> dm.clone_module	不适用	5 项操作各 10/10 PASS
运行环境与跨层数据库	npi.capabilities -> vcs.summary -> crdb.resolve -> crdb.correlates	不适用	4 项操作各 10/10 PASS
创建 FSDB 并检查 Power 许可证	transaction.writer.create -> fsdb.writer.create_scope -> power.resolve -> power.list	不适用	两项 FSDB 创建操作共 20/20 PASS ; Power 共 20 次许可阻塞

下面先讲这些常用 NPI 任务。波形、协议、覆盖率和其他工具的用法，请查看后面的命令说明和“脚本怎样调用 KVerif”章节。

演示：查找模块实例和实际参数值

本节演示一个常见问题：已知模块定义名，怎样找到它在设计中的所有实例，并读取每个实例真正采用的参数值。整个过程分为四步。先找实例路径，再查看实例内容，然后列出全部 parameter/localparam，最后读取某一个参数。示例来自 2026-07-29 普通用户 host 的 Verdi O-2018.09-SP2 VM 实测结果。

先区分三个名字
模块定义名是 `npi_fixture_alu`；实例层次全路径是 `npi_fixture_top.u_alu`；实例中 `WIDTH` 参数的完整路径是 `npi_fixture_top.u_alu.WIDTH`。这三个名称用途不同，后面的命令不能混着填。

阶段	输入	操作	关键输出
1. 定义 -> 实例	definition=npi_fixture_alu	module.find_instances	npi_fixture_top.u_alu
2. 查看实例	module=npi_fixture_top.u_alu	module.inspect	参数、端口、连线等内容
3. 列出全部参数	kind=parameters	module.objects	WIDTH、BIAS、RESULT_WIDTH 的实际值
4. 读取单值	name=...u_alu.WIDTH	language.value	value=12

先看示例 RTL

压测所用 RTL 摘录

```
module npi_fixture_alu #(
    parameter int WIDTH = 8,
    parameter logic [WIDTH-1:0] BIAS = '0
) (...);
    localparam int RESULT_WIDTH = WIDTH;
endmodule

module npi_fixture_top;
    localparam int WIDTH = 12;
    npi_fixture_alu #(
        .WIDTH(WIDTH),
        .BIAS(12'h001)
    ) u_alu (...);
endmodule
```

默认值和实例实际值不同
模块定义中的默认值是 `WIDTH=8`、`BIAS='0`；实例覆盖参数后，VM 实测值变为 `WIDTH=12`、`BIAS=1`，并推导出 `RESULT_WIDTH=12`。分析具体实例时，应使用工具返回的实例实际值，而不能只看模块定义中的默认值。

第 1 步：根据定义查全部例化路径

命令

```
KDEBUG=/home/host/kverif/tools/kdebug
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir

$KDEBUG --json action module.find_instances \
  --daidir "$DAIDIR" \
  --arg definition=npi_fixture_alu \
  --limit max_rows=100 > /tmp/module.find_instances.json
```

VM 实测输出摘录

```
{
  "ok": true,
  "action": "module.find_instances",
  "summary": {
    "definition": "npi_fixture_alu",
    "count": 1,
    "returned_count": 1,
    "truncated": false
  },
  "data": {
    "instances": [
      {
        "object": {
          "name": "u_alu",
          "full_name": "npi_fixture_top.u_alu",
          "type": "npiModule",
          "def_name": "npi_fixture_alu",
          "line": 45,
          "def_line": 8
        }
      }
    ]
  }
}
```

读取 `data.instances[*].object.full_name`。本例返回一条路径 `npi_fixture_top.u_alu`。如果 `count` 大于 1，脚本必须逐实例分析，不能只取第一项。

第 2 步：一次检查实例、parameter 和端口

命令

```
$KDEBUG --json action module.inspect \
  --daidir "$DAIDIR" \
  --arg module=npi_fixture_top.u_alu \
  --arg 'sections=["parameters","ports","io","nets","variables"]' \
  --limit max_rows=100 > /tmp/module.inspect.json
```

VM 实测输出摘录

```
{
  "ok": true,
  "action": "module.inspect",
  "summary": {
    "module": "npi_fixture_top.u_alu",
    "section_count": 8,
    "truncated": false,
    "counts": {
      "parameters": 3,
      "ports": 3,
      "io": 3,
      "nets": 3,
      "variables": 1,
      "functions": 1,
      "continuous_assignments": 1,
      "always_processes": 1
    }
  },
  "data": {
    "module": "npi_fixture_top.u_alu",
    "parameter_values": {
      "WIDTH": "12",
      "BIAS": "1",
      "RESULT_WIDTH": "12"
    }
  }
}
```

本次实测返回 parameters=3、ports=3、io=3，且 truncated=false。如果想一次查看实例的参数和端口，优先使用这条命令。

第 3 步：列出全部 parameter 和 localparam

命令

```
$KDEBUG --json action module.objects \
--daidir "$DAIDIR" \
--arg module=npi_fixture_top.u_alu \
--arg kind=parameters \
--limit max_rows=100 > /tmp/module.parameters.json
```

名称	对象全路径	类别	宽度	dec	hex
WIDTH	npi_fixture_top.u_alu.WIDTH	parameter	32	12	0000000c
BIAS	npi_fixture_top.u_alu.BIAS	parameter	12	1	001
RESULT_WIDTH	npi_fixture_top.u_alu.RESULT_WIDTH	localparam	32	12	0000000c

表中数值直接来自 VM 测试结果文件 kdebug/tests/vm/npi_actions/evidence/responses/module.objects.parameters.json。其中 local_param=1 表示 RESULT_WIDTH 是实例内部定义的 localparam。

第 4 步：精确读取一个 parameter 值

命令

```
$KDEBUG --json action language.value \
--daidir "$DAIDIR" \
--arg name=npi_fixture_top.u_alu.WIDTH \
--arg format=npiDecStrVal > /tmp/u_alu.WIDTH.json
```

VM 实测输出摘录

```
{
  "ok": true,
  "action": "language.value",
  "data": {
    "name": "npi_fixture_top.u_alu.WIDTH",
    "type": "npiParameter",
    "format": "npiDecStrVal",
    "value": "12",
    "size": 32,
    "signed": 1
  }
}
```

自动化：遍历所有实例并生成结论

Python 二次开发示例

```
#!/usr/bin/env python3
import json, subprocess

KDEBUG = "/home/host/kverif/tools/kdebug"
DAIDIR = "/proj/out/simv.daidir"

def query(action, *args):
    cmd = [KDEBUG, "--json", "action", action, "--daidir", DAIDIR]
    for item in args:
        cmd += ["--arg", item]
    rsp = json.loads(subprocess.run(cmd, check=True, text=True,
                                   capture_output=True).stdout)
    if not rsp.get("ok") or rsp.get("summary", {}).get("truncated"):
        raise RuntimeError(rsp.get("error") or "incomplete KDebug result")
    return rsp

found = query("module.find_instances", "definition=npi_fixture_alu")
report = []
for row in found["data"]["instances"]:
    path = row["object"]["full_name"]
    params = query("module.objects", "module=" + path, "kind=parameters")
    report.append({
        "instance": path,
        "definition": row["object"]["def_name"],
        "parameters": {item["object"]["name"]: item["values"]["dec"]
                       for item in params["data"]["items"]},
    })
print(json.dumps({"instance_count": len(report), "instances": report},
                 indent=2, sort_keys=True))
```

脚本输出

```
{
  "instance_count": 1,
  "instances": [{
    "definition": "npi_fixture_alu",
    "instance": "npi_fixture_top.u_alu",
    "parameters": {"BIAS": "1", "RESULT_WIDTH": "12", "WIDTH": "12"}
  }]
}
```

迁移到自己的设计

需要替换	如何获得	注意事项
DAIDIR	VCS 编译生成的 simv.daidir 绝对路径	必须对应当前要分析的、已经编译展开的设计。
definition	RTL module 定义名	它不是层次路径。
module	module.find_instances	使用返回的 full_name，多个实例逐项处理。
kind=parameters	固定枚举值	读取 parameter/localparam；其他对象使用对应 kind。
max_rows	按设计规模设置	始终检查 truncated=false。

VM 实测结果

module.find_instances 10/10 PASS; module.inspect 10/10 PASS; module.objects(kind=parameters) 10/10 PASS; language.value 10/10 PASS。实测断言明确校验 WIDTH=12、BIAS=1、RESULT_WIDTH=12。

各返回字段的完整说明见 module.find_instances、module.inspect、module.objects 和 language.value。

演示：模块端口、方向、位宽与上下层连线

检查模块端口时，只有端口名还不够。至少还要知道方向、编译展开后的实际位宽、端口序号、上层连接字段 npiParamConn 和实例内部连接字段 npiParamConn。本例继续使用实例 npiParam_top.u_alu。

命令

```
KDEBUG=/home/host/kverif/tools/kdebug
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir

$KDEBUG --json action module.objects \
  --daidir "$DAIDIR" \
  --arg module=npiParam_top.u_alu \
  --arg kind=ports \
  --limit max_rows=100 > /tmp/u_alu.ports.json
```

port	direction	width	high connection	low connection
lhs	npiInput	12	npi_fixture_top.lhs	npi_fixture_top.u_alu.lhs
rhs	npiInput	12	npi_fixture_top.rhs	npi_fixture_top.u_alu.rhs
result	npiOutput	12	npi_fixture_top.result	npi_fixture_top.u_alu.result

什么时候用 ports，什么时候用 io

kind=ports 返回端口对象、端口序号、位宽以及上下层连接，适合检查模块连线；kind=io 返回 RTL 中的输入输出声明和方向。

VM 实测两者均返回 3 项：lhs/rhs=npiParamInput, result=npiParamOutput。

精确读取一个端口关系**命令**

```
$KDEBUG --json action language.relate \
  --daidir "$DAIDIR" \
  --arg name=npiParam_top.u_alu.result \
  --arg relation_type=npiParamConn > /tmp/u_alu.result.high.json
```

VM 实测输出摘录

```
{
  "ok": true,
  "source": "npiParam_top.u_alu.result",
  "relation_type": "npiParamConn",
  "related": {
    "full_name": "npiParam_top.result",
    "type": "npiParam",
    "size": 12,
    "line": 38
  }
}
```

脚本检查结果

```
{
  "module": "npiParam_top.u_alu",
  "port_count": 3,
  "io_declaration_count": 3,
  "unconnected_ports": [],
  "width_mismatches": [],
  "result_high_connection": "npiParam_top.result",
  "status": "PASS"
}
```

在自己的设计中，不要因为 high/low 连接字段非空就认定连线正确。脚本还应检查方向规则、位宽是否一致、允许悬空的白名单以及 interface/modport 语义。详细字段见 module.objects、module.inspect 和 language.relate。

VM 实测结果

module.objects(kind=ports) 10/10 PASS; module.objects(kind=io) 10/10 PASS; language.relate 10/10 PASS。

演示：列出普通实例、generate 实例和 primitive

只执行 kind=instances 会漏掉 generate scope 内的实例和 Verilog primitive。要完整列出层次，至少应查询下面四种 kind，并保留每项的模块定义名、源码行和完整层次名。

完整查询命令

```
KDEBUG=/home/host/kverif/tools/kdebug
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir

for KIND in instances generate_scopes instances_in_generate primitives; do
  $KDEBUG --json action module.objects \
    --daidir "$DAIDIR" \
    --arg module=npi_fixture_top \
    --arg kind="$KIND" \
    --limit max_rows=100 > "/tmp/top.$KIND.json" || exit 2
done
```

kind	完整层次路径	object type	definition	line
instances	npi_fixture_top.u_alu	npiModule	npi_fixture_alu	45
generate_scopes	npi_fixture_top.g_wide	npiGenScope		52
instances_in_generate	npi_fixture_top.g_wide.u_leaf	npiModule	npi_fixture_leaf	53
primitives	npi_fixture_top.u_not	npiGate	not	60

合并后的层次结论

```
{
  "instances": [
    "npi_fixture_top.u_alu"
  ],
  "generate_scopes": [
    "npi_fixture_top.g_wide"
  ],
  "instances_in_generate": [
    "npi_fixture_top.g_wide.u_leaf"
  ],
  "primitives": [
    "npi_fixture_top.u_not"
  ],
  "instantiated_unit_count": 3
}
```

本例为什么必须查四次

普通实例是 npi_fixture_top.u_alu; generate 形成的层次是 npi_fixture_top.g_wide; 该层次里的实例是 npi_fixture_top.g_wide.u_leaf; primitive 是 npi_fixture_top.u_not。generate scope 只是一个层次容器，并不是模块实例。统计实例数量时不要把它重复计算。

如果要“根据模块定义名找所有实例”，使用 module.find_instances；如果要“列出某个父模块下的所有子对象”，则合并上述几种 module.objects 查询结果。

VM 实测结果

instances、generate_scopes、instances_in_generate、primitives 四个 kind 均为 10/10 PASS。

演示：模块内部对象、过程与源码位置

查看模块内部内容时，可以先分成两类：net、variable、function、task 是声明或可调用对象；连续赋值、always、initial 是描述行为的语句块。每个返回对象都带 file 和 line，随后可用 text.line 读取源码。

完整查询命令

```
KDEBUG=/home/host/kverif/tools/kdebug
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir

query_kind() {
  MODULE=$1; KIND=$2
  $KDEBUG --json action module.objects --daidir "$DAIDIR" \
    --arg "module=$MODULE" --arg "kind=$KIND" --limit max_rows=100 \
    > "/tmp/${KIND}.json" || return 2
}

query_kind npi_fixture_top nets
query_kind npi_fixture_top.u_alu variables
query_kind npi_fixture_top.u_alu functions
query_kind npi_fixture_top tasks
query_kind npi_fixture_top.u_alu continuous_assignments
query_kind npi_fixture_top.u_alu always_processes
query_kind npi_fixture_top initial_processes
query_kind npi_fixture_top language_interfaces
```

kind	module	count	VM 返回示例	line
nets	npi_fixture_top	3	npi_fixture_top.result	38
variables	npi_fixture_top.u_alu	1	npi_fixture_top.u_alu.sum	17
functions	npi_fixture_top.u_alu	1	npi_fixture_top.u_alu.add_with_bias	19
tasks	npi_fixture_top	1	npi_fixture_top.check_result	62
continuous_assignments	npi_fixture_top.u_alu	1	npiContAssign	30
always_processes	npi_fixture_top.u_alu	1	npiAlways	26
initial_processes	npi_fixture_top	1	npiInitial	66
language_interfaces	npi_fixture_top	0	(测试设计中没有这类对象)	

零结果也必须被正确解释

测试设计中没有 SystemVerilog interface，因此 language_interfaces 返回 count=0，truncated=false。这表示命令执行成功，只是没有找到这类对象。只有出现 ok=false、error，或结果被截短时，脚本才不能继续给出完整结论。

从行为对象跳到源码行

命令

```
SOURCE=$(python3 -c \
  'import json; r=json.load(open("/tmp/always_processes.json")); \
  print(r["data"]["items"][0]["object"]["file"])')

$KDEBUG --json action text.line --daidir "$DAIDIR" \
  --arg "file=$SOURCE" --arg line=27 > /tmp/source.line27.json
```

VM 实测输出摘录

```
{
  "ok": true,
  "line": 27,
  "content": " sum = add_with_bias(lhs, rhs);",
  "word_count": 14
}
```

脚本整理后的结果

```
{
  "module": "npi_fixture_top.u_alu",
  "variable": "sum@17",
  "function": "add_with_bias@19",
  "always_process": "npiAlways@26",
  "assignment_line": "sum = add_with_bias(lhs, rhs);",
  "continuous_assignment": "npiContAssign@30"
}
```


这组命令可以生成模块内容清单、定位组合逻辑入口、检查 task/function 是否存在，并从 NPI 查询结果跳到对应源码。详细字段见 module.objects 和 text.line。

VM 实测结果

上述 8 个 module.objects kind 与 text.line 均为 10/10 PASS；其中 language_interfaces 的正确结果为 0 项。

演示：对照 RTL 设计对象和展开后的网表对象

NPI Language Model 看到的是 RTL 里的模块、参数、端口、源码行和对象关系；NPI Netlist Model 看到的是编译展开后的 net、cell 和位切片。两者回答的问题不同：查层次、参数和源码时先用 language.*；查位切片、门级对象或网表名称时先用 netlist.*。

两类对象的查询命令

```
KDEBUG=/home/host/kverif/tools/kdebug
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir

$KDEBUG --json action language.resolve --daidir "$DAIDIR" \
--arg name=npi_fixture_top.u_alu > /tmp/language.instance.json

$KDEBUG --json action language.iterate --daidir "$DAIDIR" \
--arg name=npi_fixture_top.u_alu --arg object_type=npiParameter \
--limit max_rows=100 > /tmp/language.parameters.json

$KDEBUG --json action netlist.resolve --daidir "$DAIDIR" \
--arg name=npi_fixture_top.result --arg object_type=npiNlNet \
> /tmp/netlist.result.json

$KDEBUG --json action netlist.iterate --daidir "$DAIDIR" \
--arg name=npi_fixture_top --arg object_type=npiNlNet \
--limit max_rows=100 > /tmp/netlist.nets.json
```

查询	关键对象	type	定义/数值/宽度	line	适合回答
Language resolve	npi_fixture_top.u_alu	npiModule	npi_fixture_alu	45	实例、定义与源码
Language iterate	WIDTH / BIAS / RESULT_WIDTH	npiParameter	12 / 1 / 12	9/10/16	实际参数与 localparam
Language relate	npi_fixture_top.result	npiNet	12	38	RTL 端口上下层关系
Netlist resolve	npi_fixture_top.result[11:0]	npiNlDeclNet	12	不适用	编译展开后的总线或位切片
Netlist iterate	7 nets	npiNlNet	result[11:0]	不适用	网表对象全集

对照后的结果

```
{
  "language_instance": "npi_fixture_top.u_alu",
  "definition": "npi_fixture_alu",
  "language_parameters": [
    "WIDTH",
    "BIAS",
    "RESULT_WIDTH"
  ],
  "rtl_high_connection": "npi_fixture_top.result",
  "netlist_resolved_name": "npi_fixture_top.result[11:0]",
  "netlist_width": 12,
  "netlist_contains_result": true
}
```

不要自行改写工具返回的名称

RTL 连接名是 npi_fixture_top.result，Netlist Model 返回 npi_fixture_top.result[11:0]。脚本应同时保存对象来自哪种 NPI 模型，以及 type、size 和原始 full_name。不要先删除 [11:0]，再假定两个名称一定指向同一类对象。

详细字段见 language.resolve、language.iterate、language.relate、netlist.resolve 和 netlist.iterate。

VM 实测结果

language.resolve/iterate/relate 与 netlist.resolve/iterate 五项均为 10/10 PASS。

演示：读取源码并在副本中修改设计

text.line 和 text.words 用来读取编译数据库对应的源码行及其中的单词、符号；dm.add_net 和 dm.clone_module 会生成修改后的设计文件。所有修改都写到新路径，不会直接覆盖原 RTL。生成文件后，仍要执行 diff、重新编译和回归。

Text Model 命令

```
KDEBUG=/home/host/kverif/tools/kdebug
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir
OUT=/tmp/kverif-npi-demo
mkdir -p "$OUT"

$KDEBUG --json action module.objects --daidir "$DAIDIR" \
  --arg module=npi_fixture_top.u_alu --arg kind=always_processes \
  --limit max_rows=100 > "$OUT/always.json"
SOURCE=$(python3 -c \
  'import json,sys; print(json.load(open(sys.argv[1]))["data"]["items"][0]["object"]["file"])' \
  "$OUT/always.json")

$KDEBUG --json action text.line --daidir "$DAIDIR" \
  --arg "file=$SOURCE" --arg line=27 > "$OUT/line.json"

$KDEBUG --json action text.words --daidir "$DAIDIR" \
  --arg "file=$SOURCE" --arg line=27 --limit max_rows=100 \
  > "$OUT/words.json"

$KDEBUG --json action text.replace_line --daidir "$DAIDIR" \
  --arg "file=$SOURCE" --arg line=27 \
  --arg 'content= sum = lhs - rhs;' \
  --arg "output=$OUT/design.patched.sv" > "$OUT/replace.json"
```

text.line 的 VM 输出摘录

```
{
  "line": 27,
  "content": " sum = add_with_bias(lhs, rhs);",
  "word_count": 14
}
```

序号	单词或符号	NPI 返回的属性
2	sum	npiTextSignal
4	=	npiTextOperator
6	add_with_bias	npiTextInstance
8	lhs	npiTextInSignal
11	rhs	npiTextInSignal

text.replace_line 输出

```
{
  "status": "written",
  "line": 27,
  "original": " sum = add_with_bias(lhs, rhs);",
  "replacement": " sum = lhs - rhs;",
  "output": "/tmp/kverif-npi-demo/design.patched.sv"
}
```

源码路径应以编译数据库返回的结果为准

在自己的项目中，应从 module.objects 或 language.resolve 返回的 object.file 读取源码完整路径。如果 daidir 是在另一个工作区编译出来的，不要凭目录结构猜 SOURCE。应使用数据库返回的路径，或重新生成与当前源码目录一致的 daidir。

生成修改后的设计文本

生成设计副本的命令

```
$KDEBUG --json action dm.add_net --daidir "$DAIDIR" \  
--arg module=npi_fixture_top --arg name=debug_bus \  
--arg net_type=npiDmNetWire --arg packed_left=7 --arg packed_right=0 \  
--arg "output_dir=$OUT/dm-add-net" > "$OUT/dm-add-net.json"  
  
$KDEBUG --json action dm.clone_module --daidir "$DAIDIR" \  
--arg module=npi_fixture_alu --arg new_name=npi_fixture_alu_clone \  
--arg "output_dir=$OUT/dm-clone" > "$OUT/dm-clone.json"  
  
diff -u "$SOURCE" "$OUT/design.patched.sv" || test $? -eq 1
```

操作	状态	生成文件或对象	实际结果
text.replace_line	written	design.patched.sv	第 27 行替换；原文件不变
dm.add_net	written	debug_bus	npiDmNetWire [7:0]
dm.clone_module	written	npi_fixture_alu_clone	在独立目录中生成复制后的模块

接受修改前必须完成这些检查

先确认 `ok=true` 且输出文件非空；再确认输出路径不是原文件；随后执行 `diff`、重新编译和目标回归。KDebug 返回 `written` 只表示文件已经写出，不表示 RTL 功能正确。

详细字段见 `text.line`、`text.words`、`text.replace_line`、`dm.add_net` 和 `dm.clone_module`。

VM 实测结果

`text.line/words/replace_line` 与 `dm.add_net/clone_module` 五项均为 10/10 PASS，输出文件非空且内容断言通过。

演示：NPI 能力、VCS 与 CRDB 跨层检查

批量分析前先检查三件事：当前 Verdi 中是否有需要的 Tcl NPI 命令，VCS/KDB 数据是否能由这个版本打开，以及 CRDB 等专用数据库能否正常读取。能看到命令并不代表许可证一定可用，因此 Power 功能还要执行真实查询。

环境与数据库检查命令

```
KDEBUG=/home/host/kverif/tools/kdebug  
DAIDIR=/home/host/kverif_npi_action_stress_20260729/fixture/design/simv.daidir  
CRDB=/home/host/kverif_npi_action_stress_20260729/fixture/crdb/dut.crdb  
  
$KDEBUG --json action npi.capabilities > /tmp/npi.capabilities.json  
  
$KDEBUG --json action vcs.summary --daidir "$DAIDIR" \  
> /tmp/vcs.summary.json  
  
$KDEBUG --json action crdb.resolve \  
--arg "crdb=$CRDB" --arg name=npi_crdb_top.state --arg level=RTL \  
> /tmp/crdb.resolve.json  
  
$KDEBUG --json action crdb.correlates \  
--arg "crdb=$CRDB" --arg name=npi_crdb_top.state --arg level=RTL \  
--limit max_rows=100 > /tmp/crdb.correlates.json
```

操作	关键值	核对值	结果说明
npi.capabilities	domains=12	available=12	12 个 Tcl NPI 功能域的命令都存在
vcs.summary	0-2018.09-SP2	modules=3	errors=0 warnings=1
crdb.resolve	npi_crdb_top.state	level=1	type=npiCrdbSignal size=1
crdb.correlates	count=1	npi_crdb_top.state	target_level=2

不要把查询条件 `level` 和对象属性 `level` 当成同一个值

本例使用 `level=RTL` 作为查询条件；查到的对象自身 `level` 为 1，关联对象的 `level` 为 2。脚本应分别保存查询条件、对象 `level` 和 `full_name`，不能只比较名称。

详细字段见 `npi.capabilities`、`vcs.summary`、`crdb.resolve` 和 `crdb.correlates`。

VM 实测结果

npi.capabilities、vcs.summary、crdb.resolve、crdb.correlates 四项均为 10/10 PASS；VCS 版本为 O-2018.09-SP2。

演示：创建 Transaction/FSDB 并检查 Power 许可证

transaction.writer.create 和 fsdb.writer.create_scope 会根据输入数据创建新的 FSDB；power.resolve 和 power.list 会加载设计与 UPF，再查询电源对象。新 FSDB 写出后要重新打开确认；Power 查询失败时，则要先分清是输入有误还是许可证不可用。

创建事务 FSDB

绝对路径命令

```
/home/host/kverif/tools/kdebug --json action transaction.writer.create \
--arg 'output=/tmp/kverif-npi-demo/transactions.fsdb' \
--arg 'overwrite=false' \
--arg 'unit=1ns' \
--arg 'begin_time=0' \
--arg 'stream=bus.requests' \
--arg 'transactions=[{"start_delta":10,"duration":20,"type":"npiFsdbwTransTransaction","label":"req0","tags":["read"]}, {"start_delta":5,"duration":10,"type":"npiFsdbwTransTransaction","label":"rsp0"}]' \
--arg 'relations=[{"relation":"npiFsdbwRelParentChild","master":0,"slave":1}]'
```

VM 实测结果

```
{
  "ok": true,
  "output": "/tmp/kverif-npi-demo/transactions.fsdb",
  "stream": "bus.requests",
  "transaction_count": 2,
  "relation_count": 1,
  "end_time": 45
}
```

创建层次 FSDB，并重新打开检查

绝对路径命令

```
/home/host/kverif/tools/kdebug --json action fsdb.writer.create_scope \
--arg 'output=/tmp/kverif-npi-demo/hierarchy.fsdb' \
--arg 'overwrite=false' \
--arg 'unit=1ns' \
--arg 'begin_time=0' \
--arg 'end_time_delta=100' \
--arg 'operations=[{"op":"scope","type":"npiFsdbScopeSvModule","name":"top"}, {"op":"scope","type":"npiFsdbScopeSvModule","name":"u_a"}, {"op":"up"}, {"op":"scope","type":"npiFsdbScopeSvModule","name":"u_b"}]'
```

重新打开并列出层次

```
/home/host/kverif/tools/kdebug --json action scope.list \
--fsdb /tmp/kverif-npi-demo/hierarchy.fsdb \
--path top --limit max_rows=100 > /tmp/kverif-npi-demo/reopen.json
```

VM 实测结果

```
{
  "ok": true,
  "output": "/tmp/kverif-npi-demo/hierarchy.fsdb",
  "scope_count": 3,
  "up_count": 1,
  "reopen_scope_list": "PASS 10/10"
}
```

检查 Power 功能的许可证

命令

```
KDEBUG=/home/host/kverif/tools/kdebug
POWER=/home/host/kverif_npi_action_stress_20260729/fixture/power

$KDEBUG --json action power.resolve \
  --target "filelist=$POWER/run.f" \
  --target "upf=$POWER/demo.upf" \
  --target "workdir=$POWER" \
  --target 'defines=["NOVAS_UPF_PKG"]' \
  --arg name=system/PD_TOP --arg object_type=npiPwPowerDomain \
  > /tmp/power.resolve.json
```

VM 实测结果

```
{
  "ok": false,
  "action": "power.resolve",
  "error": {
    "code": "LICENSE_UNAVAILABLE",
    "message": "required Synopsys license feature is unavailable: PowerAwareAnalysis",
    "recoverable": true,
    "feature": "PowerAwareAnalysis"
  }
}
```

看到 available 仍不能断定许可证可用
npi.capabilities 只能证明 Power Tcl 命令存在；本 VM 的 power.resolve 和 power.list 各执行 10 次，均因缺少 PowerAwareAnalysis 返回 LICENSE_UNAVAILABLE，recoverable=true。脚本应把它记录为环境阻塞，不能写成 PASS，也不能归因于 RTL。

详细字段见 transaction.writer.create、fsdb.writer.create_scope、power.resolve 和 power.list。

VM 实测结果
transaction writer 10/10 PASS；hierarchy FSDB writer 10/10 PASS 且重开 10/10 PASS；Power 两项共 20 次许可阻塞。

KDebug 操作参考

目录与内建

本节有 4 项操作。先通过下表找到需要的操作，再点击名称查看参数和命令。每项操作都包含：用途、需要的输入、命令示例、输出示例和读取方法。

操作名	状态	需要的输入	用途
actions	stable	none	列出 KDebug 当前支持的操作，并显示每项操作的状态和所需输入。
batch	stable	none	一次提交多项 KDebug 操作，按顺序执行并分别返回结果。
schema	stable	none	查看某项操作允许输入哪些 JSON 字段，以及会返回哪些字段。
npi.capabilities	experimental	none	检查当前 Verdi 是否提供 KDebug 需要的 Tcl NPI 命令。许可证是否可用仍要单独测试。

actions

列出 KDebug 当前支持的操作，并显示每项操作的状态和所需输入。

状态: stable

输入: none

实现: actions

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/actions.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action actions
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "actions",
  "summary": {
    "action_count": 1,
    "removed_count": 0
  }
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `action_count`, `removed_count`。需要逐项明细时再看 `data`。本页示例取自 `kdebug/examples/responses/actions.basic.json`。

batch

一次提交多项 KDebug 操作，按顺序执行并分别返回结果。

状态: stable

输入: none

实现: batch

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.requests	array<object>	必填	batch 操作中要依次执行的子请求列表。	[{"api_version": "kdebug.v1", "action": "schema"}]

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/batch.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action batch \
--arg 'requests=[{"api_version": "kdebug.v1", "action": "schema"}]'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "batch",
  "summary": {
    "count": 1,
    "all_ok": true
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 count, all_ok。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\batch.basic.json。

schema

查看某项操作允许输入哪些 JSON 字段，以及会返回哪些字段。

状态: stable

输入: none

实现: schema

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.action	string	可选	要执行的操作名，例如 module.inspect。	signal.statistics
args.kind	string (request, response)	可选	要列出的对象类别，例如 ports、parameters 或 instances。	request

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/schema.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action schema \
--arg 'action=signal.statistics' \
--arg 'kind=request'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "schema",
  "summary": {
    "action": "signal.statistics",
    "kind": "request"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 action, kind。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\schema.basic.json。

mpi.capabilities

检查当前 Verdi 是否提供 KDebug 需要的 Tcl NPI 命令。许可证是否可用仍要单独测试。

状态: experimental

输入: none

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/mpi.capabilities.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action mpi.capabilities
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "mpi.capabilities",
  "summary": {
    "domain_count": 12,
    "available_domain_count": 12
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 domain_count, available_domain_count。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\mpi_actions\evidence\responses\mpi.capabilities.json。

会话生命周期

本节有 6 项操作。先通过下表找到需要的操作，再点击名称查看参数和命令。每项操作都包含：用途、需要的输入、命令示例、输出示例和读取方法。

操作名	状态	需要的输入	用途
session.close	stable	session	正常关闭一个会话并释放后端进程和临时资源。
session.doctor	stable	session	检查会话记录、后台进程、通信方式和输入文件是否正常。
session.gc	stable	none	清理过期、失效或遗留的会话状态。
session.kill	stable	session	强制终止失去响应或无法正常关闭的会话。
session.list	stable	session	列出当前用户打开的 KDebug 会话，以及每个会话加载了哪些文件。
session.open	stable	any	打开可复用的设计、波形或联合调试会话，避免每次重新加载大文件。

session.close

正常关闭一个会话并释放后端进程和临时资源。

状态: stable

输入: session

实现: session

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.session_id	string	必填	已打开会话的 ID 或别名。	case_a

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/session.close.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action session.close \  
--session case_a \  
--arg 'session_id=case_a'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "session.close",  
  "summary": {  
    "session_id": "case_a",  
    "mode": "combined"  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 session_id, mode。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/session.close.basic.json。

session.doctor

检查会话记录、后台进程、通信方式和输入文件是否正常。

状态: stable

输入: session

实现: session

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/session.doctor.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action session.doctor \  
--session case_a
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "session.doctor",
  "summary": {
    "session_id": "case_a",
    "mode": "combined"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 session_id, mode。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\session.doctor.basic.json。

session.gc

清理过期、失效或遗留的会话状态。

状态: stable

输入: none

实现: session

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/session.gc.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action session.gc
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "session.gc",
  "summary": {
    "session_id": "case_a",
    "mode": "combined"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 session_id, mode。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\session.gc.basic.json。

session.kill

强制终止失去响应或无法正常关闭的会话。

状态: stable

输入: session

实现: session

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.session_id	string	必填	已打开会话的 ID 或别名。	case_a

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/session.kill.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action session.kill \  
--session case_a \  
--arg 'session_id=case_a'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "session.kill",  
  "summary": {  
    "session_id": "case_a",  
    "mode": "combined"  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 session_id, mode。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/session.kill.basic.json。

session.list

列出当前用户打开的 KDebug 会话，以及每个会话加载了哪些文件。

状态: stable

输入: session

实现: session

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/session.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action session.list \  
--session case_a
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "session.list",
  "summary": {
    "session_id": "case_a",
    "mode": "combined"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 session_id, mode。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\session.list.basic.json。

session.open

打开可复用的设计、波形或联合调试会话，避免每次重新加载大文件。

状态: stable

输入: any

实现: session

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	可选	会话、配置、游标、列表或对象的名称。	case_a

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/session.open.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action session.open \
--daidir simv.daidir \
--fsdb waves.fsdb \
--arg 'name=case_a'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "session.open",
  "summary": {
    "session_id": "case_a",
    "mode": "combined"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 session_id, mode。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\session.open.basic.json。

设计与波形联合分析

本节有 2 项操作。先通过下表找到需要的操作，再点击名称查看参数和命令。每项操作都包含：用途、需要的输入、命令示例、输出示例和读取方法。

操作名	状态	需要的输入	用途
trace.active_driver	stable	any	结合指定时刻的波形值，判断当时究竟是哪一条赋值在驱动目标信号。

操作名	状态	需要的输入	用途
trace.active_driver_chain	stable	any	从目标信号开始逐级向上查找当时生效的驱动，并列出具影响选择结果的控制条件。

trace.active_driver

结合指定时刻的波形值，判断当时究竟是哪一条赋值在驱动目标信号。

状态: stable

输入: any

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.requested_time	string	必填	联合分析要求的波形时间。	120ns
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False
args.include_control	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.include_parity	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.include_alias_candidates	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.include_compat_fields	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.active_driver.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.active_driver \  
--session my_debug_session \  
--arg 'signal=top.u.ready' \  
--arg 'requested_time=120ns' \  
--arg 'include_trace=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.active_driver",  
  "request_id": "req-001",  
  "summary": {  
    "signal": "top.u.ready",  
    "requested_time": "120ns",  
    "active_time": "115ns",  
    "driver_status": "resolved",  
    "statement_count": 2,  
    "trace_node_count": 2,  
    "root_driver": {  
      "kind": "assignment",  
      "file": "rtl/foo.sv",  
      "line": 123  
    }  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, requested_time, active_time, driver_status, statement_count, trace_node_count, root_driver。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.active_driver.basic.json。

trace.active_driver_chain

从目标信号开始逐级向上查找当时生效的驱动，并列出具影响选择结果的控制条件。

状态: stable

输入: any

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.data_out
args.requested_time	string	必填	联合分析要求的波形时间。	50ns
args.clk_period	string	可选	显式指定时钟周期，用于采样或时间换算。	10ns
args.include_control	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.include_parity	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	按项目填写
args.include_alias_candidates	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.active_driver_chain.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.active_driver_chain \  
--daidir simv.daidir \  
--fsdb waves.fsdb \  
--arg 'signal=top.data_out' \  
--arg 'requested_time=50ns' \  
--arg 'clk_period=10ns'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.active_driver_chain",  
  "summary": {  
    "signal": "top.data_out",  
    "requested_time": "50ns",  
    "start_time": "50ns",  
    "chain_length": 3,  
    "driver_status": "resolved",  
    "termination": "primary_input",  
    "temporal_boundaries": 0  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, requested_time, start_time, chain_length, driver_status, termination, temporal_boundaries。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.active_driver_chain.basic.json。

静态设计与 Tcl NPI 模型

本节有 38 项操作。先通过下表找到需要的操作，再点击名称查看参数和命令。每项操作都包含：用途、需要的输入、命令示例、输出示例和读取方法。

操作名	状态	需要的输入	用途
control.explain	stable	design	解释控制信号的组合条件、候选驱动和来源位置。
counter.explain	stable	design	从设计结构解释计数器的更新条件、保持条件和宽度。
expr.normalize	stable	none	把表达式统一成固定写法，便于比较两个表达式、搜索和继续分析。
fsm.explain	stable	design	列出状态机的状态和转移条件，并标出默认路径及对应源码位置。
instance.map	stable	design	查找一个实例，并返回它的完整层次路径、模块定义和相关对象。
interface.resolve	stable	design	解析 SystemVerilog interface 实例及其端口、modport 和连接信息。
port.trace	stable	design	从端口出发追踪上下游连接、方向和驱动负载关系。
procedural.assignment	stable	design	查询信号在 always、initial、task 等过程块中的赋值位置。
sequential.update	stable	design	解释寄存器或时序变量的时钟、复位、使能和更新表达式。
signal.canonicalize	stable	design	把用户输入的信号简写解析成标准完整层次名。
signal.resolve	stable	design	解析设计信号并返回候选、类型、宽度和源码信息。
source.context	stable	none	显示源码目标行及其前后几行，便于确认问题所在位置。
trace.driver	stable	design	查询信号的静态驱动、赋值表达式和可选源码证据。
trace.expand	stable	design	展开一个信号的表达式依赖，补充候选和源码细节。
trace.explain	stable	design	把查到的驱动和依赖关系整理成一段容易阅读的原因说明。
trace.graph	stable	design	生成信号之间的依赖关系图，便于脚本继续做连线分析。
trace.load	stable	design	查询信号的静态负载及其使用位置。
trace.path	stable	design	搜索两个设计信号之间的依赖路径。
trace.query	stable	design	按给定条件查询静态驱动或负载关系。
language.resolve	experimental	design	按名称查找一个 RTL 对象，并返回类型、完整路径、源码位置等属性。
language.iterate	experimental	design	在指定模块或层次下，按类型列出 NPI 设计对象。
language.relate	experimental	design	读取一个 RTL 对象与另一个对象之间的指定关联，例如端口的上层连接。
language.value	experimental	design	读取 parameter 或 constant 在编译展开后的实际值。
module.objects	experimental	design	用 kind 选择对象类别，列出模块中的实例、端口、parameter、过程块等 15 类对象。
module.find_instances	experimental	design	根据模块定义名查找全部层次实例。
module.inspect	experimental	design	一次查看模块实例的参数、端口、连接和常用内部对象。
netlist.resolve	experimental	design	按名称查找编译展开后的 net、port 或 cell。
netlist.iterate	experimental	design	列出编译展开后的网表对象，例如 net、cell 或位切片。
text.line	experimental	design	通过 Tcl NPI 读取指定的源码行。
text.words	experimental	design	把指定源码行拆成单词和符号，并返回每项的 NPI 属性。
text.replace_line	experimental	design	在输出副本中替换指定源码行，不原地修改输入设计。
dm.add_net	experimental	design	在设计副本中新增一条 net，并把修改后的 RTL 写到新目录。原文件不会被覆盖。
dm.clone_module	experimental	design	复制一个模块并改名，把结果写到新目录。原模块不会被覆盖。
vcs.summary	experimental	design	查看 VCS 版本、编译参数、错误数量和已加载模块数量。
power.resolve	experimental	design	查找一个 Power NPI 对象，并返回它的名称、类型和关键属性。
power.list	experimental	design	按类型列出电源域、供电网络等 Power NPI 对象。
crdb.resolve	experimental	none	在 CRDB 中查找一个 RTL 或门级对象，并返回名称、层级和位宽等信息。
crdb.correlates	experimental	none	在 CRDB 中查找与指定 RTL 信号或门级信号对应的其他层级对象。

control.explain

解释控制信号的组合条件、候选驱动和来源位置。

状态: stable

输入: design

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/control.explain.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action control.explain \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "control.explain",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/control.explain.basic.json。

counter.explain

从设计结构解释计数器的更新条件、保持条件和宽度。

状态: stable

输入: design

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/counter.explain.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action counter.explain \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready'
```


标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "counter.explain",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\counter.explain.basic.json。

expr.normalize

把表达式统一成固定写法，便于比较两个表达式、搜索和继续分析。

状态: stable

输入: none

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.expr	string	必填	待解析、求值或查找事件的表达式。	valid && !ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/expr.normalize.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action expr.normalize \
--arg 'expr=valid && !ready'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "expr.normalize",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\expr.normalize.basic.json。

fsm.explain

列出状态机的状态和转移条件，并标出默认路径及对应源码位置。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/fsm.explain.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action fsm.explain \
--daidir simv.daidir \
--arg 'signal=top.u.ready'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "fsm.explain",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `signal`, `truncated`。需要逐项明细时再看 `data`。本页示例取自 `kdebug/examples/responses/fsm.explain.basic.json`。

instance.map

查找一个实例，并返回它的完整层次路径、模块定义和相关对象。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.instance	string	必填	设计实例的完整层次名。	top.u

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/instance.map.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action instance.map \
--daidir simv.daidir \
--arg 'instance=top.u'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "instance.map",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\instance.map.basic.json。

interface.resolve

解析 SystemVerilog interface 实例及其端口、modport 和连接信息。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.interface	string	必填	SystemVerilog interface 实例名。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/interface.resolve.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action interface.resolve \
--daidir simv.daidir \
--arg 'interface=if0'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "interface.resolve",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\interface.resolve.basic.json。

port.trace

从端口出发追踪上下游连接、方向和驱动负载关系。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.port	string	必填	待追踪的端口完整层次名。	ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/port.trace.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action port.trace \  
--daidir simv.daidir \  
--arg 'port=ready'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "port.trace",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/port.trace.basic.json。

procedural.assignment

查询信号在 always、initial、task 等过程块中的赋值位置。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/procedural.assignment.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action procedural.assignment \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "procedural.assignment",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\procedural.assignment.basic.json。

sequential.update

解释寄存器或时序变量的时钟、复位、使能和更新表达式。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/sequential.update.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action sequential.update \
--daidir simv.daidir \
--arg 'signal=top.u.ready'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "sequential.update",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\sequential.update.basic.json。

signal.canonicalize

把用户输入的信号简写解析成标准完整层次名。

状态: stable

输入: design

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.max_items	integer	可选	最多返回的对象、事务或结果项数。	20

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.canonicalize.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.canonicalize \  
  --daidir simv.daidir \  
  --arg 'signal=top.u.ready' \  
  --arg 'max_items=20'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "signal.canonicalize",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/signal.canonicalize.basic.json。

signal.resolve

解析设计信号并返回候选、类型、宽度和源码信息。

状态: stable

输入: design

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.max_items	integer	可选	最多返回的对象、事务或结果项数。	20

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.resolve.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.resolve \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready' \  
--arg 'max_items=20'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "signal.resolve",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/signal.resolve.basic.json。

source.context

显示源码目标行及其前后几行，便于确认问题所在位置。

状态: stable

输入: none

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.file	string	必填	源文件、配置文件或输入文件路径。	rtl/foo.sv
args.line	integer	必填	从 1 开始的源文件行号。	123

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/source.context.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action source.context \  
--arg 'file=rtl/foo.sv' \  
--arg 'line=123'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "source.context",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/source.context.basic.json。

trace.driver

查询信号的静态驱动、赋值表达式和可选源码证据。

状态: stable

输入: design

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.driver.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.driver \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready' \  
--arg 'include_trace=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.driver",  
  "summary": {  
    "signal": "top.u.ready",  
    "mode": "driver",  
    "driver_count": 1,  
    "confidence": "high",  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, mode, driver_count, confidence, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.driver.basic.json。

trace.expand

展开一个信号的表达式依赖，补充候选和源码细节。

状态: stable

输入: design

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.expand.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.expand \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready' \  
--arg 'include_trace=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.expand",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.expand.basic.json。

trace.explain

把查到的驱动和依赖关系整理成一段容易阅读的原因说明。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.explain.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.explain \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready' \  
--arg 'include_trace=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.explain",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.explain.basic.json。

trace.graph

生成信号之间的依赖关系图，便于脚本继续做连线分析。

状态: stable

输入: design

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.graph.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.graph \  
--daidir simv.daidir \  
--arg 'signal=top.u.ready' \  
--arg 'include_trace=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.graph",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.graph.basic.json。

trace.load

查询信号的静态负载及其使用位置。

状态: stable

输入: design

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.load.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.load \  
  --daidir simv.daidir \  
  --arg 'signal=top.u.ready' \  
  --arg 'include_trace=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "trace.load",  
  "summary": {  
    "signal": "top.u.ready",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.load.basic.json。

trace.path

搜索两个设计信号之间的依赖路径。

状态: stable

输入: design

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.to_signal	string	必填	路径搜索终点信号。	top.u.ready
args.from_signal	string	必填	路径搜索起点信号。	top.u.valid
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.path.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.path \
--daidir simv.daidir \
--arg 'from_signal=top.u.valid' \
--arg 'to_signal=top.u.ready' \
--arg 'include_trace=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "trace.path",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.path.basic.json。

trace.query

按给定条件查询静态驱动或负载关系。

状态: stable

输入: design

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.include_trace	boolean	可选	是否同时返回信号依赖关系或路径。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/trace.query.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action trace.query \
--daidir simv.daidir \
--arg 'signal=top.u.ready' \
--arg 'include_trace=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "trace.query",
  "summary": {
    "signal": "top.u.ready",
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/trace.query.basic.json。

language.resolve

按名称查找一个 RTL 对象，并返回类型、完整路径、源码位置等属性。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	npi_fixture_top.u_alu
args.scope	string	可选	FSDB 或 coverage 层次 scope。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/language.resolve.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action language.resolve \  
--daidir /data/build/simv.daidir \  
--arg 'name=npi_fixture_top.u_alu'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "language.resolve",  
  "summary": {  
    "status": "resolved",  
    "name": "npi_fixture_top.u_alu",  
    "scope": ""  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 name, scope, status。需要逐项明细时再看 data。本页示例取自 kdebug/tests/vm/npi_actions/evidence/responses/language.resolve.json。

language.iterate

在指定模块或层次下，按类型列出 NPI 设计对象。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.object_type	string	必填	NPI 模型中的对象类型。	npiParameter
args.name	string	必填	会话、配置、游标、列表或对象的名称。	npi_fixture_top.u_alu
args.scope	string	可选	FSDB 或 coverage 层次 scope。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/language.iterate.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action language.iterate \  
--daidir /data/build/simv.daidir \  
--arg 'name=npi_fixture_top.u_alu' \  
--arg 'object_type=npiParameter' \  
--limit 'max_rows=100'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "language.iterate",  
  "summary": {  
    "count": 3,  
    "reference": "npi_fixture_top.u_alu",  
    "object_type": "npiParameter",  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 reference, object_type, count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\language.iterate.json。

language.relate

读取一个 RTL 对象与另一个对象之间的指定关联，例如端口的上层连接。

状态: experimental

输入: design

实现: engine_forward

验证状态
VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.relation_type	string	必填	要查询的对象关系，例如 npiHighConn 表示端口的上层连接。	npiHighConn
args.name	string	必填	会话、配置、游标、列表或对象的名称。	npi_fixture_top.u_alu.result
args.scope	string	可选	FSDB 或 coverage 层次 scope。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/language.relate.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action language.relate \  
--daidir /data/build/simv.daidir \  
--arg 'name=npi_fixture_top.u_alu.result' \  
--arg 'relation_type=npiHighConn'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "language.relate",  
  "summary": {  
    "status": "resolved",  
    "source": "npi_fixture_top.u_alu.result",  
    "relation_type": "npiHighConn"  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 source, relation_type, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\language.relate.json。

language.value

读取 parameter 或 constant 在编译展开后的实际值。

状态: experimental

输入: design

实现: engine_forward

验证状态
VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	npi_fixture_top.u_alu.WIDTH
args.format	string {npiBinStrVal, npiOctStrVal, npiHexStrVal, npiDecStrVal, npiStringVal, npiRealVal, npiIntVal}	可选	值显示格式，例如 hex、bin、dec 或 sv。	npiDecStrVal
args.scope	string	可选	FSDB 或 coverage 层次 scope。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/language.value.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action language.value \  
--daidir /data/build/simv.daidir \  
--arg 'name=npi_fixture_top.u_alu.WIDTH' \  
--arg 'format=npiDecStrVal'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "language.value",  
  "summary": {  
    "status": "ok",  
    "name": "npi_fixture_top.u_alu.WIDTH",  
    "format": "npiDecStrVal"  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 name, format, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\language.value.json。

module.objects

用 kind 选择对象类别，列出模块中的实例、端口、parameter、过程块等 15 类对象。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 150/150 (15 kinds) 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.module	string	必填	通常填写模块实例的完整层次名；若某项操作需要模块定义名，本页会单独说明。	npi_fixture_top.u_alu
args.kind	string {continuous_assignments, functions, generate_scopes, instances, instances_in_generate, io, language_interfaces, nets, parameters, ports, primitives, always_processes, initial_processes, tasks, variables}	必填	要列出的对象类别，例如 ports、parameters 或 instances。	parameters

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/module.objects.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action module.objects \  
--daidir /data/build/simv.daidir \  
--arg 'module=npi_fixture_top.u_alu' \  
--arg 'kind=parameters' \  
--limit 'max_rows=100'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "module.objects",  
  "summary": {  
    "module": "npi_fixture_top.u_alu",  
    "count": 3,  
    "kind": "parameters",  
    "returned_count": 3,  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 module, kind, count, returned_count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\module.objects.parameters.json。

module.find_instances

根据模块定义名查找全部层次实例。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.definition	string	必填	模块定义名，用于反查全部实例。	npi_fixture_alu

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/module.find_instances.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action module.find_instances \  
--daidir /data/build/simv.daidir \  
--arg 'definition=npi_fixture_alu' \  
--limit 'max_rows=100'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "module.find_instances",  
  "summary": {  
    "count": 1,  
    "definition": "npi_fixture_alu",  
    "returned_count": 1,  
    "truncated": false  
  }  
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `definition`, `count`, `returned_count`, `truncated`。需要逐项明细时再看 `data`。本页示例取自 `kdebug\tests\vm\npi_actions\evidence\responses\module.find_instances.json`。

module.inspect

一次查看模块实例的参数、端口、连接和常用内部对象。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.module	string	必填	通常填写模块实例的完整层次名；若某项操作需要模块定义名，本页会单独说明。	npi_fixture_top.u_alu
args.sections	array<string>	可选	<code>module.inspect</code> 要返回哪些部分，例如 <code>ports</code> 、 <code>parameters</code> 或 <code>nets</code> 。	["instances","parameters","ports","io","nets","variables"]

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/module.inspect.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action module.inspect \  
--daidir /data/build/simv.daidir \  
--arg 'module=npi_fixture_top.u_alu' \  
--arg 'sections=["instances","parameters","ports","io","nets","variables"]' \  
--limit 'max_rows=100'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "module.inspect",  
  "summary": {  
    "module": "npi_fixture_top.u_alu",  
    "section_count": 8,  
    "truncated": false,  
    "counts": {  
      "parameters": 3,  
      "ports": 3,  
      "io": 3,  
      "nets": 3,  
      "variables": 1,  
      "_omitted": 3  
    }  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 module, section_count, truncated, counts。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\module.inspect.json。

netlist.resolve

按名称查找编译展开后的 net、port 或 cell。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	top.u_dut.ready
args.object_type	string	可选	NPI 模型中的对象类型。	npiNINet

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/netlist.resolve.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action netlist.resolve \  
--daidir /data/build/simv.daidir \  
--arg 'name=top.u_dut.ready' \  
--arg 'object_type=npiNINet'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "netlist.resolve",
  "summary": {
    "status": "resolved",
    "name": "npi_fixture_top.result"
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 name, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\netlist.resolve.json。

netlist.iterate

列出编译展开后的网表对象，例如 net、cell 或位切片。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.object_type	string	必填	NPI 模型中的对象类型。	npiNlNet
args.name	string	可选	会话、配置、游标、列表或对象的名称。	top.u_dut

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/netlist.iterate.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action netlist.iterate \
--daidir /data/build/simv.daidir \
--arg 'name=top.u_dut' \
--arg 'object_type=npiNlNet' \
--limit 'max_rows=100'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "netlist.iterate",
  "summary": {
    "count": 7,
    "reference": "npi_fixture_top",
    "object_type": "npiNlNet",
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 reference, object_type, count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\netlist.iterate.json。

text.line

通过 Tcl NPI 读取指定的源码行。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.file	string	必填	源文件、配置文件或输入文件路径。	/data/project/rtl/top.sv
args.line	integer >= 1	必填	从 1 开始的源文件行号。	12

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/text.line.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action text.line \  
--daidir /data/build/simv.daidir \  
--arg 'file=/data/project/rtl/top.sv' \  
--arg 'line=12'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "text.line",  
  "summary": {  
    "file": "/home/host/kverif_npi_candidate_20260728_full/kdebug/tests/vm/npi_actions/design.sv",  
    "line": 27,  
    "word_count": 14  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 file, line, word_count。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\text.line.json。

text.words

把指定源码行拆成单词和符号，并返回每项的 NPI 属性。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.file	string	必填	源文件、配置文件或输入文件路径。	/data/project/rtl/top.sv
args.line	integer >= 1	必填	从 1 开始的源文件行号。	12

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/text.words.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action text.words \  
--daidir /data/build/simv.daidir \  
--arg 'file=/data/project/rtl/top.sv' \  
--arg 'line=12' \  
--limit 'max_rows=50'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "text.words",  
  "summary": {  
    "count": 14,  
    "file": "/home/host/kverif_npi_candidate_20260728_full/kdebug/tests/vm/npi_actions/design.sv",  
    "line": 27,  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 file, line, count, truncated。需要逐项明
细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\text.words.json。

text.replace_line

在输出副本中替换指定源码行，不原地修改输入设计。

状态: experimental

输入: design

实现: engine_forward

验证状态
VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.file	string	必填	源文件、配置文件或输入文件路径。	/data/project/rtl/top.sv
args.output	string	必填	操作生成的文件路径；某些操作也用它保存一组输出设置。	/data/out/top.patched.sv
args.line	integer >= 1	必填	从 1 开始的源文件行号。	12
args.content	string	必填	写入或替换的文本内容。	assign ready = valid && enable;
args.overwrite	boolean	可选	是否允许覆盖已有输出文件。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看
schemas/v1/actions/text.replace_line.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action text.replace_line \  
--daidir /data/build/simv.daidir \  
--arg 'file=/data/project/rtl/top.sv' \  
--arg 'line=12' \  
--arg 'content= assign ready = valid && enable;' \  
--arg 'output=/data/out/top.patched.sv' \  
--arg 'overwrite=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "text.replace_line",
  "summary": {
    "status": "written",
    "file": "/home/host/kverif_npi_candidate_20260728_full/kdebug/tests/vm/npi_actions/design.sv",
    "line": 27,
    "output": "/home/host/kverif_npi_action_test_20260728_full/design/design.patched.sv"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 file, line, output, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\text.replace_line.json。

dm.add_net

在设计副本中新增一条 net，并把修改后的 RTL 写到新目录。原文件不会被覆盖。

状态: experimental

输入: design

实现: engine_forward

验证状态
VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.output_dir	string	必填	生成或修改结果的输出目录。	/data/out/dm_add_net
args.name	string	必填	会话、配置、游标、列表或对象的名称。	debug_bus
args.module	string	必填	通常填写模块实例的完整层次名；若某项操作需要模块定义名，本页会单独说明。	top
args.net_type	string	可选	新增或筛选 net 时使用的类型。	npiDmNetWire
args.packed_left	integer	可选	新增 packed 对象的左边界。	7
args.packed_right	integer	可选	新增 packed 对象的右边界。	0
args.overwrite	boolean	可选	是否允许覆盖已有输出文件。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/dm.add_net.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action dm.add_net \
--daidir /data/build/simv.daidir \
--arg 'module=top' \
--arg 'name=debug_bus' \
--arg 'net_type=npiDmNetWire' \
--arg 'packed_left=7' \
--arg 'packed_right=0' \
--arg 'output_dir=/data/out/dm_add_net' \
--arg 'overwrite=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "dm.add_net",
  "summary": {
    "module": "npi_fixture_top",
    "status": "written",
    "name": "debug_bus",
    "output_dir": "/home/host/kverif_npi_action_test_20260728_full/design/dm-add-net"
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 module, name, output_dir, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\dm.add_net.json。

dm.clone_module

复制一个模块并改名，把结果写到新目录。原模块不会被覆盖。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.output_dir	string	必填	生成或修改结果的输出目录。	/data/out/dm_clone
args.new_name	string	必填	克隆后模块或对象的新名称。	alu_debug
args.module	string	必填	通常填写模块实例的完整层次名；若某项操作需要模块定义名，本页会单独说明。	alu
args.overwrite	boolean	可选	是否允许覆盖已有输出文件。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/dm.clone_module.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action dm.clone_module \
--daidir /data/build/simv.daidir \
--arg 'module=alu' \
--arg 'new_name=alu_debug' \
--arg 'output_dir=/data/out/dm_clone' \
--arg 'overwrite=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "dm.clone_module",
  "summary": {
    "module": "npi_fixture_alu",
    "status": "written",
    "new_name": "npi_fixture_alu_clone",
    "output_dir": "/home/host/kverif_npi_action_test_20260728_full/design/dm-clone"
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 module, new_name, output_dir, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\dm.clone_module.json。

vcs.summary

查看 VCS 版本、编译参数、错误数量和已加载模块数量。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.database	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/vcs.summary.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action vcs.summary \  
--daidir /data/build/simv.daidir
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "vcs.summary",  
  "summary": {  
    "database": "/home/host/kverif_npi_action_test_20260728_full/design/simv.daidir",  
    "warnings": 1,  
    "errors": 0,  
    "modules": 3  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 database, warnings, errors, modules。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\vcs.summary.json。

power.resolve

查找一个 Power NPI 对象，并返回它的名称、类型和关键属性。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress 0 PASS / 10 license-blocked 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	top/PD_TOP
args.object_type	string	可选	NPI 模型中的对象类型。	npiPwPowerDomain

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/power.resolve.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action power.resolve \
--daidir /data/build/power_simv.daidir \
--arg 'name=top/PD_TOP' \
--arg 'object_type=npiPwPowerDomain'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": false,
  "action": "power.resolve",
  "summary": {},
  "data": null,
  "error": {
    "code": "LICENSE_UNAVAILABLE",
    "message": "required Synopsys license feature is unavailable: PowerAwareAnalysis",
    "recoverable": true,
    "candidates": [],
    "suggested_actions": [],
    "details": {
      "feature": "PowerAwareAnalysis",
      "exit_code": 0,
      "stdout": "logDir = /home/host/kverif_npi_action_test_20260728_full/power/verdiLog\n\nVerdi (R)\n\nRelease
Verdi_0-2018.09-SP2 for (RH Linux x86_64/64bit) -- Thu Feb 21 04:40:56 PDT 2019\n\nCop...",
      "stderr": ""
    }
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 ok、action、data/error。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\power.resolve.json。

power.list

按类型列出电源域、供电网络等 Power NPI 对象。

状态: experimental

输入: design

实现: engine_forward

验证状态

VM stress 0 PASS / 10 license-blocked 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.object_type	string	必填	NPI 模型中的对象类型。	npiPwElement
args.name	string	必填	会话、配置、游标、列表或对象的名称。	top/PD_TOP

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/power.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action power.list \
--daidir /data/build/power_simv.daidir \
--arg 'name=top/PD_TOP' \
--arg 'object_type=npiPwElement' \
--limit 'max_rows=100'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": false,
  "action": "power.list",
  "summary": {},
  "data": null,
  "error": {
    "code": "LICENSE_UNAVAILABLE",
    "message": "required Synopsys license feature is unavailable: PowerAwareAnalysis",
    "recoverable": true,
    "candidates": [],
    "suggested_actions": [],
    "details": {
      "feature": "PowerAwareAnalysis",
      "exit_code": 0,
      "stdout": "logDir = /home/host/kverif_npi_action_test_20260728_full/power/verdiLog\\n\\nVerdi (R)\\n\\nRelease
Verdi_0-2018.09-SP2 for (RH Linux x86_64/64bit) -- Thu Feb 21 04:40:56 PDT 2019\\n\\nCop...",
      "stderr": ""
    }
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 ok、action、data/error。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\power.list.json。

crdb.resolve

在 CRDB 中查找一个 RTL 或门级对象，并返回名称、层级和位宽等信息。

状态: experimental

输入: none

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.crdb	string	必填	CRDB 数据库或映射文件路径。	/data/build/dut.crdb
args.name	string	必填	会话、配置、游标、列表或对象的名称。	top.u_dut.ready
args.level	string (RTL, GATE)	可选	层级、深度或输出详细级别。	RTL

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/crdb.resolve.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action crdb.resolve \
--arg 'crdb=/data/build/dut.crdb' \
--arg 'name=top.u_dut.ready' \
--arg 'level=RTL'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "crdb.resolve",
  "summary": {
    "status": "resolved",
    "name": "npi_crdb_top.state",
    "level": "RTL"
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 name, level, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\crdb.resolve.json。

crdb.correlates

在 CRDB 中查找与指定 RTL 信号或门级信号对应的其他层级对象。

状态: experimental

输入: none

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.crdb	string	必填	CRDB 数据库或映射文件路径。	/data/build/dut.crdb
args.name	string	必填	会话、配置、游标、列表或对象的名称。	top.u_dut.ready
args.level	string (RTL, GATE)	可选	层级、深度或输出详细级别。	RTL

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/crdb.correlates.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action crdb.correlates \  
--arg 'crdb=/data/build/dut.crdb' \  
--arg 'name=top.u_dut.ready' \  
--arg 'level=RTL' \  
--limit 'max_rows=100'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "crdb.correlates",  
  "summary": {  
    "count": 1,  
    "name": "npi_crdb_top.state",  
    "level": "RTL",  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 name, level, count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\crdb.correlates.json。

波形、协议与 FSDB 写出

本节有 56 项操作。先通过下表找到需要的操作，再点击名称查看参数和命令。每项操作都包含：用途、需要的输入、命令示例、输出示例和读取方法。

操作名	状态	需要的输入	用途
apb.config.list	stable	waveform	列出当前波形会话中的 APB 协议配置。
apb.config.load	stable	waveform	加载并命名一组 APB 信号映射。
apb.cursor	stable	waveform	在命名 APB 事务结果上移动或读取游标。
apb.query	stable	waveform	从 FSDB 中解码 APB 传输并按地址、方向或时间过滤。

操作名	状态	需要的输入	用途
apb.transfer_window	experimental	waveform	围绕指定 APB 传输提取前后波形窗口和关键信号。
axi.analysis	stable	waveform	统计 AXI 事务的吞吐、延迟、阻塞时间和未完成事务数量。
axi.export	stable	waveform	把 AXI 事务或分析结果导出为结构化文件。
axi.channel_stall	experimental	waveform	找出 AXI 各通道中 valid 已拉高但 ready 尚未响应的时间段。
axi.config.list	stable	waveform	列出当前波形会话中的 AXI 协议配置。
axi.config.load	stable	waveform	加载并命名 AXI channel 信号映射。
axi.cursor	stable	waveform	在命名 AXI 查询结果上移动或读取游标。
axi.latency_outlier	experimental	waveform	找出延迟明显偏高的 AXI 事务，并返回相关通道信息。
axi.outstanding_timeline	experimental	waveform	列出 AXI 未完成事务数量随时间的变化。
axi.query	stable	waveform	从 FSDB 中解码并筛选 AXI 读写事务。
axi.request_response_pair	experimental	waveform	配对 AXI 请求与响应，展示端到端事务关系。
counter.statistics	stable	waveform	按时钟和 valid 条件统计计数值的最小、最大、平均和样本数。
cursor.delete	stable	waveform	删除一个命名波形游标。
cursor.get	stable	waveform	读取当前或指定波形游标的位置。
cursor.list	stable	waveform	列出会话中的全部命名波形游标。
cursor.set	stable	waveform	在指定时间设置或更新命名波形游标。
cursor.use	stable	waveform	把一个命名游标设为后续查询的当前游标。
detect_anomaly	stable	waveform	对一组信号检测未知值、异常跳变和长期不活动等现象。
event.config.list	stable	waveform	列出当前会话中的命名事件表达式配置。
event.config.load	stable	waveform	加载并命名可复用的波形事件表达式。
event.export	stable	waveform	把匹配事件的时间点和上下文导出到文件。
event.find	stable	waveform	查找表达式第一次、最后一次或全部匹配事件。
expr.eval_at	stable	waveform	在指定波形时间计算逻辑表达式。
handshake.inspect	stable	waveform	检查 valid/ready 握手的传输、等待和 stall 行为。
inspect_signal	deprecated	waveform	保留给旧脚本使用的单信号检查入口；新脚本建议使用 signal.scan 等操作。
list.add	stable	waveform	向命名信号列表追加信号。
list.create	stable	waveform	创建可复用的命名信号列表。
list.delete	stable	waveform	删除命名信号列表。
list.diff	stable	waveform	比较命名列表在两个时间点的信号值差异。
list.show	stable	waveform	列出信号列表或展示指定列表内容。
list.validate	stable	waveform	验证命名列表中的信号是否存在并可读取。
list.value_at	stable	waveform	批量读取命名列表在指定时间的值。
sampled_pulse.inspect	experimental	waveform	按采样时钟检查脉冲宽度、丢采样和边界行为。
scope.list	stable	waveform	列出 FSDB 层次 scope 和可选信号。
signal.changes	stable	waveform	返回信号在时间窗口内的跳变序列。
signal.stability	stable	waveform	检查信号在给定窗口是否保持稳定或违反稳定约束。
signal.statistics	stable	waveform	统计信号活动度、高低周期、变化次数和采样方式。
signal.trend	stable	waveform	把信号值按时间分段，提取上升、下降或稳定趋势。
rc.generate	stable	waveform	根据 position 简写映射和配置生成可复用的 Verdi RC 文件。
value.at	stable	waveform	读取一个信号在指定 FSDB 时间的值。
value.batch_at	stable	waveform	在同一时间批量读取多个信号，减少重复 FSDB 访问。
signal.scan	stable	waveform	一次汇总信号的首末值、跳变、未知值和可选样本。

操作名	状态	需要的输入	用途
verify.conditions	stable	waveform	在一个时间点验证多个信号或表达式条件。
window.verify	stable	waveform	在时间窗口内验证条件持续成立、出现次数或时序关系。
stream.config.load	stable	waveform	加载数据流、通道和数据包字段定义。
stream.config.list	stable	waveform	列出已经加载的数据流和数据包配置。
stream.show	stable	waveform	查看数据流、通道和字段布局。
stream.validate	stable	waveform	检查数据流配置是否完整，配置中的信号能否在波形中找到。
stream.query	stable	waveform	按字段、方向、时间等条件查询数据包。
stream.export	stable	waveform	导出解码后的数据包、字段和它们之间的关系。
transaction.writer.create	experimental	none	根据事务列表生成 transaction FSDB，并写入事务之间的父子关系。
fsdb.writer.create_scope	experimental	none	通过 Tcl NPI FSDB Writer 创建层次、信号和值并写出新 FSDB。

apb.config.list

列出当前波形会话中的 APB 协议配置。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.include_accesses	boolean	可选	在输出中包含每一笔 APB/AXI 访问的明细。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/apb.config.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action apb.config.list \  
--session case_a \  
--arg 'include_accesses=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "apb.config.list",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/apb.config.list.basic.json。

apb.config.load

加载并命名一组 APB 信号映射。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_accesses	boolean	可选	在输出中包含每一笔 APB/AXI 访问的明细。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/apb.config.load.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action apb.config.load \  
--session case_a \  
--arg 'name=if0' \  
--arg 'include_accesses=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "apb.config.load",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `truncated`。需要逐项明细时再看 `data`。
本页示例取自 `kdebug/examples/responses/apb.config.load.basic.json`。

apb.cursor

在命名 APB 事务结果上移动或读取游标。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.op	string	必填	游标或配置操作类型。	begin
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_accesses	boolean	可选	在输出中包含每一笔 APB/AXI 访问的明细。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/apb.cursor.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action apb.cursor \  
  --session case_a \  
  --arg 'name=if0' \  
  --arg 'op=begin' \  
  --arg 'include_accesses=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "apb.cursor",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\apb.cursor.basic.json。

apb.query

从 FSDB 中解码 APB 传输并按地址、方向或时间过滤。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_accesses	boolean	可选	在输出中包含每一笔 APB/AXI 访问的明细。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/apb.query.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action apb.query \  
  --session case_a \  
  --arg 'name=if0' \  
  --arg 'include_accesses=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "apb.query",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\apb.query.basic.json。

apb.transfer_window

围绕指定 APB 传输提取前后波形窗口和关键信号。

状态: experimental

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_accesses	boolean	可选	在输出中包含每一笔 APB/AXI 访问的明细。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/apb.transfer_window.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action apb.transfer_window \  
--session case_a \  
--arg 'name=if0' \  
--arg 'include_accesses=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "apb.transfer_window",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\apb.transfer_window.basic.json。

axi.analysis

统计 AXI 事务的吞吐、延迟、阻塞时间和未完成事务数量。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.analysis.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.analysis \
--session case_a \
--arg 'name=if0' \
--arg 'include_transactions=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.analysis",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/axi.analysis.basic.json。

axi.export

把 AXI 事务或分析结果导出为结构化文件。

状态: stable

输入: waveform

实现: engine_forward

验证状态
targeted

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.time_range	object	可选	包含 begin/end 的时间范围对象。	{"begin":"0ns","end":"1us"}
args.format	string (tsv, csv)	可选	值显示格式，例如 hex、bin、dec 或 sv。	tsv
args.output_prefix	string	可选	批量导出文件名前缀。	/tmp/if0_axi
args.start	string	可选	查询窗口起点，部分 action 使用 start 命名。	按项目填写
args.end	string	可选	查询窗口终点，例如 1us。	按项目填写
args.begin	string	可选	查询窗口起点，例如 0ns。	按项目填写
args.from	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.to	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.export.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.export \
--session case_a \
--arg 'name=if0' \
--arg 'time_range={"begin":"0ns","end":"1us"}' \
--arg 'format=tsv' \
--arg 'output_prefix=/tmp/if0_axi'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.export",
  "summary": {
    "name": "if0",
    "write_count": 12,
    "read_count": 12,
    "total_count": 24,
    "format": "tsv"
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 name, write_count, read_count, total_count, format。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\axi.export.basic.json。

axi.channel_stall

找出 AXI 各通道中 valid 已拉高但 ready 尚未响应的时间段。

状态: experimental

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.channel_stall.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.channel_stall \
--session case_a \
--arg 'name=if0' \
--arg 'include_transactions=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.channel_stall",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\axi.channel_stall.basic.json。

axi.config.list

列出当前波形会话中的 AXI 协议配置。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.config.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.config.list \  
--session case_a \  
--arg 'include_transactions=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "axi.config.list",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/axi.config.list.basic.json。

axi.config.load

加载并命名 AXI channel 信号映射。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.config.load.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.config.load \  
--session case_a \  
--arg 'name=if0' \  
--arg 'include_transactions=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.config.load",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\axi.config.load.basic.json。

axi.cursor

在命名 AXI 查询结果上移动或读取游标。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.op	string	必填	游标或配置操作类型。	begin
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.cursor.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.cursor \
--session case_a \
--arg 'name=if0' \
--arg 'op=begin' \
--arg 'include_transactions=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.cursor",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\axi.cursor.basic.json。

axi.latency_outlier

找出延迟明显偏高的 AXI 事务，并返回相关通道信息。

状态: experimental

输入: waveform

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.latency_outlier.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.latency_outlier \  
--session case_a \  
--arg 'name=if0' \  
--arg 'include_transactions=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "axi.latency_outlier",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/axi.latency_outlier.basic.json。

axi.outstanding_timeline

列出 AXI 未完成事务数量随时间的变化。

状态: experimental

输入: waveform

实现: engine_forward

验证状态

partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.outstanding_timeline.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.outstanding_timeline \
--session case_a \
--arg 'name=if0' \
--arg 'include_transactions=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.outstanding_timeline",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\axi.outstanding_timeline.basic.json。

axi.query

从 FSDB 中解码并筛选 AXI 读写事务。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/axi.query.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.query \
--session case_a \
--arg 'name=if0' \
--arg 'include_transactions=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "axi.query",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\axi.query.basic.json。

axi.request_response_pair

配对 AXI 请求与响应，展示端到端事务关系。

状态: experimental

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.include_transactions	boolean	可选	包含逐事务明细，而不只返回摘要。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/axi.request_response_pair.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action axi.request_response_pair \  
--session case_a \  
--arg 'name=if0' \  
--arg 'include_transactions=false'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "axi.request_response_pair",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果
先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `truncated`。需要逐项明细时再看 `data`。
本页示例取自 `kdebug/examples/responses/axi.request_response_pair.basic.json`。

counter.statistics

按时钟和 `valid` 条件统计计数值的最小、最大、平均和样本数。

状态: stable

输入: waveform

实现: engine_forward

验证状态
targeted

参数

参数	类型/可选值	是否必填	说明	示例值
args.time_range	object	必填	包含 begin/end 的时间范围对象。	{"begin":"100ns","end":"200ns"}
args.vld	object	必填	计数器统计使用的有效采样条件。	{"expr":"valid && ready","signals":{"valid":"top.valid","ready":"top.ready"}}
args.cnt	string	必填	要统计的计数器信号。若计数器分成高、低两段，也可用 hi/lo 指定两个信号。	{top.cnt_hi,top.cnt_lo}
args.clock	string	必填	采样时钟的完整层次信号名。	top.clk
args.sampling	string {posedge, negedge}	可选	采样策略或采样配置对象。	posedge
args.max_samples	integer	可选	最多返回的波形采样点数。	1000

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/counter.statistics.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action counter.statistics \
--session wave0 \
--arg 'clock=top.clk' \
--arg 'sampling=posedge' \
--arg 'time_range={"begin":"100ns","end":"200ns"}' \
--arg 'vld={"expr":"valid && ready","signals":{"valid":"top.valid","ready":"top.ready"}}" \
--arg 'cnt={top.cnt_hi,top.cnt_lo}' \
--arg 'max_samples=1000'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "counter.statistics",
  "summary": {
    "sample_count": 10,
    "valid_count": 6,
    "min_value": "3",
    "max_value": "9",
    "average_value": "5.5"
  }
}
```

如何读取结果
先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `sample_count`, `valid_count`, `min_value`, `max_value`, `average_value`。需要逐项明细时再看 `data`。本页示例取自 `kdebug/examples/responses/counter.statistics.basic.json`。

cursor.delete

删除一个命名波形游标。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/cursor.delete.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action cursor.delete \  
--session case_a \  
--arg 'name=if0'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "cursor.delete",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 `kdebug/examples/responses/cursor.delete.basic.json`。

cursor.get

读取当前或指定波形游标的位置。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/cursor.get.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action cursor.get \  
--session case_a
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "cursor.get",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\cursor.get.basic.json。

cursor.list

列出会话中的全部命名波形游标。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	此项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/cursor.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action cursor.list \
--session case_a
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "cursor.list",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\cursor.list.basic.json。

cursor.set

在指定时间设置或更新命名波形游标。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.time	string	必填	波形时间，例如 100ns。	100ns

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/cursor.set.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action cursor.set \  
--session case_a \  
--arg 'time=100ns'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "cursor.set",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/cursor.set.basic.json。

cursor.use

把一个命名游标设为后续查询的当前游标。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/cursor.use.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action cursor.use \  
--session case_a \  
--arg 'name=if0'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "cursor.use",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\cursor.use.basic.json。

detect_anomaly

对一组信号检测未知值、异常跳变和长期不活动等现象。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signals	array<string>	必填	多个信号完整层次名组成的数组。	["top.u.valid","top.u.ready"]

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/detect_anomaly.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action detect_anomaly \
--session case_a \
--arg 'signals=["top.u.valid","top.u.ready"]'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "detect_anomaly",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\detect_anomaly.basic.json。

event.config.list

列出当前会话中的命名事件表达式配置。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.max_examples	integer	可选	摘要中最多保留的代表性示例数。	5

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/event.config.list.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action event.config.list \  
--session case_a \  
--arg 'max_examples=5'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "event.config.list",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `truncated`。需要逐项明细时再看 `data`。
本页示例取自 `kdebug/examples/responses/event.config.list.basic.json`。

event.config.load

加载并命名可复用的波形事件表达式。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.max_examples	integer	可选	摘要中最多保留的代表性示例数。	5

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/event.config.load.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action event.config.load \  
--session case_a \  
--arg 'name=if0' \  
--arg 'max_examples=5'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "event.config.load",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\event.config.load.basic.json。

event.export

把匹配事件的时间点和上下文导出到文件。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.expr	string	必填	待解析、求值或查找事件的表达式。	valid && !ready
args.max_examples	integer	可选	摘要中最多保留的代表性示例数。	5

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/event.export.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action event.export \
--session case_a \
--arg 'expr=valid && !ready' \
--arg 'max_examples=5'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "event.export",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\event.export.basic.json。

event.find

查找表达式第一次、最后一次或全部匹配事件。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.expr	string	必填	待解析、求值或查找事件的表达式。	valid && !ready
args.max_examples	integer	可选	摘要中最多保留的代表性示例数。	5

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/event.find.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action event.find \  
--session case_a \  
--arg 'expr=valid && !ready' \  
--arg 'max_examples=5'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "event.find",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\event.find.basic.json。

expr.eval_at

在指定波形时间计算逻辑表达式。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.expr	string	必填	待解析、求值或查找事件的表达式。	valid && !ready
args.time	string	必填	波形时间，例如 100ns。	100ns

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/expr.eval_at.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action expr.eval_at \
--session case_a \
--arg 'expr=valid && !ready' \
--arg 'time=100ns'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "expr.eval_at",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/expr.eval_at.basic.json。

handshake.inspect

检查 valid/ready 握手的传输、等待和 stall 行为。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.valid	string	必填	valid 握手信号完整层次名。	top.u.valid
args.ready	string	必填	ready 握手信号完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/handshake.inspect.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action handshake.inspect \
--session case_a \
--arg 'valid=top.u.valid' \
--arg 'ready=top.u.ready'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "handshake.inspect",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/handshake.inspect.basic.json。

inspect_signal

保留给旧脚本使用的单信号检查入口；新脚本建议使用 `signal.scan` 等操作。

状态: deprecated

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/inspect_signal.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action inspect_signal \  
--session case_a \  
--arg 'signal=top.u.ready'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "inspect_signal",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `truncated`。需要逐项明细时再看 `data`。
本页示例取自 `kdebug/examples/responses/inspect_signal.basic.json`。

list.add

向命名信号列表追加信号。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signals	array<string>	必填	多个信号完整层次名组成的数组。	["top.u.valid", "top.u.ready"]
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/list.add.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.add \  
--session case_a \  
--arg 'name=if0' \  
--arg 'signals=["top.u.valid", "top.u.ready"]'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "list.add",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\list.add.basic.json。

list.create

创建可复用的命名信号列表。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signals	array<string>	必填	多个信号完整层次名组成的数组。	["top.u.valid","top.u.ready"]
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/list.create.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.create \
--session case_a \
--arg 'name=if0' \
--arg 'signals=["top.u.valid","top.u.ready"]'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "list.create",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\list.create.basic.json。

list.delete

删除命名信号列表。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/list.delete.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.delete \  
--session case_a \  
--arg 'name=if0'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "list.delete",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果
先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `truncated`。需要逐项明细时再看 `data`。
本页示例取自 `kdebug/examples/responses/list.delete.basic.json`。

list.diff

比较命名列表在两个时间点的信号值差异。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.time_b	string	必填	比较的第二个时间点。	200ns
args.time_a	string	必填	比较的第一个时间点。	100ns
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/list.diff.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.diff \  
--session case_a \  
--arg 'name=if0' \  
--arg 'time_a=100ns' \  
--arg 'time_b=200ns'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "list.diff",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\list.diff.basic.json。

list.show

列出信号列表或展示指定列表内容。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args	object	可选	这项操作没有额外的功能参数。	{}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/list.show.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.show \  
--session case_a
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "list.show",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\list.show.basic.json。

list.validate

验证命名列表中的信号是否存在并可读取。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/list.validate.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.validate \  
--session case_a \  
--arg 'name=if0'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "list.validate",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/list.validate.basic.json。

list.value_at

批量读取命名列表在指定时间的值。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.time	string	必填	波形时间，例如 100ns。	100ns
args.name	string	必填	会话、配置、游标、列表或对象的名称。	if0
args.format	string	可选	值显示格式，例如 hex、bin、dec 或 sv。	hex

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/list.value_at.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action list.value_at \
--session case_a \
--arg 'name=if0' \
--arg 'time=100ns' \
--arg 'format=hex'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "list.value_at",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\list.value_at.basic.json。

sampled_pulse.inspect

按采样时钟检查脉冲宽度、丢采样和边界行为。

状态: experimental

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/sampled_pulse.inspect.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action sampled_pulse.inspect \
--session case_a \
--arg 'signal=top.u.ready'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "sampled_pulse.inspect",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\sampled_pulse.inspect.basic.json。

scope.list

列出 FSDB 层次 scope 和可选信号。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.scope	string	必填	FSDB 或 coverage 层次 scope。	按项目填写
args.path	string	可选	scope 路径、配置路径或目标对象路径。	

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/scope.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action scope.list \  
--session case_a \  
--arg 'path='
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "scope.list",  
  "summary": {  
    "path": "top",  
    "scope_count": 2,  
    "signal_count": 0  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 path, scope_count, signal_count。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\scope.list.json。

signal.changes

返回信号在时间窗口内的跳变序列。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.time_range	object	可选	包含 begin/end 的时间范围对象。	{"start":"100ns","end":"200ns"}
args.aggregate_only	boolean	可选	只返回聚合统计，不返回逐项明细。	True

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.changes.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.changes \  
  --session case_a \  
  --arg 'signal=top.u.ready' \  
  --arg 'time_range={"start":"100ns","end":"200ns"}' \  
  --arg 'aggregate_only=true'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "signal.changes",  
  "summary": {  
    "transition_count": 2,  
    "returned_change_rows": 3,  
    "includes_initial_value": true,  
    "actual_transition_count": 2,  
    "first_change": {  
      "time": "100ns",  
      "value": "0"  
    },  
    "last_change": {  
      "time": "180ns",  
      "value": "0"  
    },  
    "semantic_note": "signal.changes rows are value-change timeline rows; use signal.statistics for sampled high cycles."  
  },  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 transition_count, returned_change_rows, includes_initial_value, actual_transition_count, first_change, last_change, semantic_note。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\signal.changes.basic.json。

signal.stability

检查信号在给定窗口是否保持稳定或违反稳定约束。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.max_items	integer	可选	最多返回的对象、事务或结果项数。	20

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.stability.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.stability \  
  --session case_a \  
  --arg 'signal=top.u.ready' \  
  --arg 'max_items=20'
```


标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "signal.stability",
  "summary": {
    "truncated": false
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/signal.stability.basic.json。

signal.statistics

统计信号活动度、高低周期、变化次数和采样方式。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.clock	string	可选	采样时钟的完整层次信号名。	top.clk
args.time_range	object	可选	包含 begin/end 的时间范围对象。	{"start": "100ns", "end": "200ns"}

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.statistics.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.statistics \
--session case_a \
--arg 'signal=top.u.ready' \
--arg 'clock=top.clk' \
--arg 'time_range={"start": "100ns", "end": "200ns"}'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "signal.statistics",
  "summary": {
    "sampling_mode": "clock",
    "sample_count": 10,
    "transition_count": 2,
    "high_cycles": 0,
    "low_cycles": 0
  }
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 sampling_mode, sample_count, transition_count, high_cycles, low_cycles。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/signal.statistics.basic.json。

signal.trend

把信号值按时间分段，提取上升、下降或稳定趋势。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.max_items	integer	可选	最多返回的对象、事务或结果项数。	20

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.trend.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.trend \  
--session case_a \  
--arg 'signal=top.u.ready' \  
--arg 'max_items=20'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "signal.trend",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/signal.trend.basic.json。

rc.generate

根据 position 简写映射和配置生成可复用的 Verdi RC 文件。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.rc_path	string	必填	生成的 Verdi RC 文件路径。	signal.rc
args.config_path	string	必填	协议、事件、stream 或 position 映射配置文件路径。	wave_view.json
args.include_preview	boolean	可选	在输出中包含写出文件或修改结果预览。	True
args.json_path	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.output_path	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.allow_invalid	boolean	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.max_preview_lines	integer	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/rc.generate.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action rc.generate \  
--session case_a \  
--arg 'config_path=wave_view.json' \  
--arg 'rc_path=signal.rc' \  
--arg 'include_preview=true'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "rc.generate",  
  "summary": {  
    "config_path": "wave_view.json",  
    "rc_path": "signal.rc",  
    "group_count": 2,  
    "signal_count": 4,  
    "expr_signal_count": 1,  
    "marker_count": 1,  
    "written": true,  
    "_omitted": 1  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 config_path, rc_path, group_count, signal_count, expr_signal_count, marker_count, written, valid。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/rc.generate.basic.json。

value.at

读取一个信号在指定 FSDB 时间的值。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.time	string	必填	波形时间，例如 100ns。	100ns
args.format	string	可选	值显示格式，例如 hex、bin、dec 或 sv。	hex

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/value.at.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action value.at \  
--session case_a \  
--arg 'signal=top.u.ready' \  
--arg 'time=100ns' \  
--arg 'format=hex'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "value.at",  
  "summary": {  
    "signal": "top.clk",  
    "time": "10ns"  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, time。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/value.at.basic.json。

value.batch_at

在同一时间批量读取多个信号，减少重复 FSDB 访问。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.time	string	必填	波形时间，例如 100ns。	100ns
args.signals	array<string>	必填	多个信号完整层次名组成的数组。	["top.u.valid","top.u.ready"]
args.format	string	可选	值显示格式，例如 hex、bin、dec 或 sv。	hex

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/value.batch_at.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action value.batch_at \  
--session case_a \  
--arg 'signals=["top.u.valid","top.u.ready"]' \  
--arg 'time=100ns' \  
--arg 'format=hex'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "value.batch_at",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug\examples\responses\value.batch_at.basic.json。

signal.scan

一次汇总信号的首末值、跳变、未知值和可选样本。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.signal	string	必填	单个信号的完整层次名。	top.u.ready
args.begin	string	可选	查询窗口起点，例如 0ns。	0ns
args.end	string	可选	查询窗口终点，例如 1us。	100ns
args.format	string	可选	值显示格式，例如 hex、bin、dec 或 sv。	hex
args.start	string	可选	查询窗口起点，部分 action 使用 start 命名。	按项目填写
args.stop	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
args.time_range	object	可选	包含 begin/end 的时间范围对象。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/signal.scan.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action signal.scan \  
--session case_a \  
--arg 'signal=top.u.ready' \  
--arg 'begin=0ns' \  
--arg 'end=100ns' \  
--arg 'format=hex' \  
--limit 'max_rows=16'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "signal.scan",  
  "summary": {  
    "signal": "top.u.ready",  
    "change_count": 2,  
    "unknown_count": 0,  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 signal, change_count, unknown_count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\signal.scan.basic.json。

verify.conditions

在一个时间点验证多个信号或表达式条件。

状态: stable

输入: waveform

实现: engine_forward

验证状态
regression

参数

参数	类型/可选值	是否必填	说明	示例值
args.time	string	必填	波形时间，例如 100ns。	100ns
args.conditions	array<object>	必填	待验证条件数组，每项描述信号、表达式和期望值。	[{"signal":"top.u.valid","op":"==","value":"'h1'"}]

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/verify.conditions.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action verify.conditions \  
--session case_a \  
--arg 'conditions=[{"signal":"top.u.valid","op":"==","value":"'h1'"}]' \  
--arg 'time=100ns'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "verify.conditions",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/verify.conditions.basic.json。

window.verify

在时间窗口内验证条件持续成立、出现次数或时序关系。

状态: stable

输入: waveform

实现: engine_forward

验证状态
partial

参数

参数	类型/可选值	是否必填	说明	示例值
args.time_range	object	必填	包含 begin/end 的时间范围对象。	{"begin":"0ns","end":"1us"}
args.conditions	array<object>	必填	待验证条件数组，每项描述信号、表达式和期望值。	[{"signal":"top.u.valid","op":"==","value":"'h1'"}]

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/window.verify.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action window.verify \  
--session case_a \  
--arg 'conditions=[{"signal":"top.u.valid","op":"==","value":""''h1"}]' \  
--arg 'time_range={"begin":"0ns","end":"1us"}'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "window.verify",  
  "summary": {  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 truncated。需要逐项明细时再看 data。
本页示例取自 kdebug/examples/responses/window.verify.basic.json。

stream.config.load

加载数据流、通道和数据包字段定义。

状态: stable

输入: waveform

实现: engine_forward

验证状态

synthetic

参数

参数	类型/可选值	是否必填	说明	示例值
args.streams	array<object>	必填	要加载的 stream 配置数组。	[{"name":"req_stream","clock":"top.clk", clock_edge":"posedge","reset":"!top.rst_n", vld":"top.req_vld","rdy":"top.req_rdy", sop":"top.req_sop","eop":"top.req_eop", data_fields":{"...
args.mode	string (replace, append)	可选	查询方式，例如只看开头、结尾、摘要或未覆盖项。 具体可选值由操作决定。	replace
args.file	string	可选	源文件、配置文件或输入文件路径。	按项目填写
args.config_path	string	可选	协议、事件、stream 或 position 映射配置文件路径。	按项目填写
args.config	object	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/stream.config.load.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action stream.config.load \  
--session wave0 \  
--arg 'mode=replace' \  
--arg 'streams=[{"name":"req_stream","clock":"top.clk","clock_edge":"posedge",  
"reset":"!top.rst_n","vld":"top.req_vld","rdy":"top.req_rdy",  
"sop":"top.req_sop","eop":"top.req_eop",  
"data_fields":{"opcode":"top.req_data[7:0]","addr":{"top.addr_hi,  
top.addr_lo},"is_wr":"top.req_cmd == 2'""'b01"}]]'
```


标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "stream.config.load",
  "summary": {
    "loaded": 1,
    "mode": "replace"
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 loaded, mode。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\stream.config.load.basic.json。

stream.config.list

列出已经加载的数据流和数据包配置。

状态: stable

输入: waveform

实现: engine_forward

验证状态
synthetic

参数

参数	类型/可选值	是否必填	说明	示例值
args.verbose	boolean	可选	是否返回更详细的诊断信息。	False

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/stream.config.list.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action stream.config.list \
--session wave0 \
--arg 'verbose=false'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "stream.config.list",
  "summary": {
    "count": 1
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 count。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\stream.config.list.basic.json。

stream.show

查看数据流、通道和字段布局。

状态: stable

输入: waveform

实现: engine_forward

验证状态

synthetic

参数

参数	类型/可选值	是否必填	说明	示例值
args.stream	string	必填	命名 stream 或 transaction stream 定义。	req_stream
args.name	string	可选	会话、配置、游标、列表或对象的名称。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/stream.show.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action stream.show \  
--session wave0 \  
--arg 'stream=req_stream'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "stream.show",  
  "summary": {  
    "stream": "req_stream",  
    "handshake": "vld/rdy",  
    "packet_enabled": true  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 stream, handshake, packet_enabled。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/stream.show.basic.json。

stream.validate

检查数据流配置是否完整，配置中的信号能否在波形中找到。

状态: stable

输入: waveform

实现: engine_forward

验证状态

synthetic

参数

参数	类型/可选值	是否必填	说明	示例值
args.stream	string	必填	命名 stream 或 transaction stream 定义。	req_stream
args.dynamic	boolean	可选	是否读取随时间变化的关系或运行时数值。	True
args.time_range	object	可选	包含 begin/end 的时间范围对象。	{"begin": "0ns", "end": "1us"}
args.max_edges	integer	可选	依赖图最多返回的边数。	256
args.start	string	可选	查询窗口起点，部分 action 使用 start 命名。	按项目填写
args.end	string	可选	查询窗口终点，例如 1us。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/stream.validate.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action stream.validate \
--session wave0 \
--arg 'stream=req_stream' \
--arg 'dynamic=true' \
--arg 'time_range={"begin":"0ns","end":"1us"}' \
--arg 'max_edges=256'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "stream.validate",
  "summary": {
    "stream": "req_stream",
    "ok": true
  }
}
```

如何读取结果

先看 `ok`: `false` 表示失败，应读取 `error`; `true` 表示成功。成功后先看 `summary` 中的 `stream`, `ok`。需要逐项明细时再看 `data`。本页示例取自 `kdebug/examples/responses/stream.validate.basic.json`。

stream.query

按字段、方向、时间等条件查询数据包。

状态: stable

输入: waveform

实现: engine_forward

验证状态
synthetic

参数

参数	类型/可选值	是否必填	说明	示例值
<code>args.stream</code>	string	必填	命名 stream 或 transaction stream 定义。	<code>req_stream</code>
<code>args.query</code>	string (summary, first_transfer, last_transfer, transfer_window, first_stall, last_stall, stall_window, first_packet, last_packet, packet_at, packet_window, match_field)	必填	数据流操作或通用操作使用的查询条件。	<code>summary</code>
<code>args.start</code>	string	可选	查询窗口起点，部分 action 使用 start 命名。	<code>0ns</code>
<code>args.end</code>	string	可选	查询窗口终点，例如 <code>1us</code> 。	<code>1us</code>
<code>args.limit</code>	integer	可选	最多返回多少条结果。部分操作使用这个简写字段。	<code>32</code>
<code>args.time_range</code>	object	可选	包含 begin/end 的时间范围对象。	按项目填写
<code>args.channel</code>	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
<code>args.match</code>	object	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
<code>args.packet_index</code>	integer >= 0	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写
<code>args.field_scope</code>	string (beat, stable, any)	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 `schemas/v1/actions/stream.query.request.schema.json`。

命令示例

```
/home/host/kverif/tools/kdebug --json action stream.query \  
  --session wave0 \  
  --arg 'stream=req_stream' \  
  --arg 'query=summary' \  
  --arg 'start=0ns' \  
  --arg 'end=1us' \  
  --arg 'limit=32'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "stream.query",  
  "summary": {  
    "stream": "req_stream",  
    "transfer_count": 128,  
    "stall_cycles": 4,  
    "packet_count": 16,  
    "truncated": false  
  }  
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 stream, transfer_count, stall_cycles, packet_count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug\examples\responses\stream.query.basic.js on。

stream.export

导出解码后的数据包、字段和它们之间的关系。

状态: stable

输入: waveform

实现: engine_forward

验证状态
synthetic

参数

参数	类型/可选值	是否必填	说明	示例值
args.stream	string	必填	命名 stream 或 transaction stream 定义。	req_stream
args.kind	string {transfer, packet, packet_beats}	可选	要列出的对象类别，例如 ports、parameters 或 instances。	transfer
args.format	string {tsv, csv, kout}	可选	值显示格式，例如 hex、bin、dec 或 sv。	tsv
args.start	string	可选	查询窗口起点，部分 action 使用 start 命名。	0ns
args.end	string	可选	查询窗口终点，例如 1us。	1us
args.output_file	string	可选	导出结果文件路径。	/tmp/req_stream.tsv
args.time_range	object	可选	包含 begin/end 的时间范围对象。	按项目填写
args.channel	string	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/stream.export.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action stream.export \  
  --session wave0 \  
  --arg 'stream=req_stream' \  
  --arg 'kind=transfer' \  
  --arg 'format=tsv' \  
  --arg 'start=0ns' \  
  --arg 'end=1us' \  
  --arg 'output_file=/tmp/req_stream.tsv'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "stream.export",  
  "summary": {  
    "stream": "req_stream",  
    "transfer_count": 128,  
    "truncated": false  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 stream, transfer_count, truncated。需要逐项明细时再看 data。本页示例取自 kdebug/examples/responses/stream.export.basic.json。

transaction.writer.create

根据事务列表生成 transaction FSDB，并写入事务之间的父子关系。

状态: experimental

输入: none

实现: engine_forward

验证状态

VM stress PASS 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.output	string	必填	操作生成的文件路径；某些操作也用它保存一组输出设置。	/data/out/transactions.fsd
args.transactions	array<object>	必填	transaction writer 要写入的事务数组。	[{"start_delta":10,"duration":20,"type":"npiFsdBWTransTransaction","label":"req0","tags":["read"]}, {"start_delta":5,"duration":10,"type":"npiFsdBWTransTransaction","label":"rsp0"}]
args.stream	string	必填	命名 stream 或 transaction stream 定义。	bus.requests
args.overwrite	boolean	可选	是否允许覆盖已有输出文件。	False
args.unit	string	可选	无显式单位数值使用的默认时间单位。	1ns
args.begin_time	integer >= 0	可选	查询窗口起点的完整字段名。	0
args.relations	array<object>	可选	transaction writer 中的关系数组。	[{"relation":"npiFsdBWRelParentChild","master":0,"slave":1}]

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/transaction.writer.create.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action transaction.writer.create \
--arg 'output=/data/out/transactions.fsdb' \
--arg 'overwrite=false' \
--arg 'unit=1ns' \
--arg 'begin_time=0' \
--arg 'stream=bus.requests' \
--arg 'transactions=[{"start_delta":10,"duration":20,"type":"npiFsdbwTransTransaction","label":"req0","tags":["read"]},{ "start_delta":5,"duration":10,"type":"npiFsdbwTransTransaction","label":"rsp0"}]' \
--arg 'relations=[{"relation":"npiFsdbwRelParentChild","master":0,"slave":1}]'
```

标准输出示例

```
{
  "api_version": "kdebug.v1",
  "ok": true,
  "action": "transaction.writer.create",
  "summary": {
    "status": "written",
    "output": "/home/host/kverif_npi_action_test_20260728_full/transactions.fsdb",
    "stream": "bus.requests",
    "transaction_count": 2,
    "relation_count": 1
  }
}
```

如何读取结果
先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 output, stream, transaction_count, relation_count, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\transaction.writer.create.json。

fsdb.writer.create_scope

通过 Tcl NPI FSDB Writer 创建层次、信号和值并写出新 FSDB。

状态: experimental

输入: none

实现: engine_forward

验证状态
VM stress PASS 10/10 + reopen 10/10 2026-07-29

参数

参数	类型/可选值	是否必填	说明	示例值
args.output	string	必填	操作生成的文件路径；某些操作也用它保存一组输出设置。	/data/out/hierarchy.fsdb
args.overwrite	boolean	可选	是否允许覆盖已有输出文件。	False
args.unit	string	可选	无显式单位数值使用的默认时间单位。	1ns
args.begin_time	integer >= 0	可选	查询窗口起点的完整字段名。	0
args.end_time_delta	integer >= 0	可选	相对起点的结束时间偏移。	100
args.operations	array<object>	可选	批量执行的操作数组。	[{"op":"scope","type":"npiFsdbScopeSvModule","name":"top"}, {"op":"scope","type":"npiFsdbScopeSvModule","name":"u_a"}, {"op":"up"}, {"op":"scope","type":"npiFsdbScopeSvModule","name":"...}
args.scopes	array<object>	可选	该参数只用于这项操作。可选值和格式见本页字段定义。	按项目填写

输入路径、返回数量上限和输出格式见 KDebug 通用参数。若要核对程序实际接受的 JSON 字段，可查看 schemas/v1/actions/fsdb.writer.create_scope.request.schema.json。

命令示例

```
/home/host/kverif/tools/kdebug --json action fsdb.writer.create_scope \  
--arg 'output=/data/out/hierarchy.fsdb' \  
--arg 'overwrite=false' \  
--arg 'unit=1ns' \  
--arg 'begin_time=0' \  
--arg 'end_time_delta=100' \  
--arg 'operations=[{"op": "scope", "type": "npiFsdbScopeSvModule", "name": "top"}, {"op": "scope", "type": "npiFsdbScopeSvModule", "name": "u_a"}, {"op": "up"}, {"op": "scope", "type": "npiFsdbScopeSvModule", "name": "u_b"}]'
```

标准输出示例

```
{  
  "api_version": "kdebug.v1",  
  "ok": true,  
  "action": "fsdb.writer.create_scope",  
  "summary": {  
    "status": "written",  
    "output": "/home/host/kverif_npi_action_test_20260728_full/hierarchy.fsdb",  
    "scope_count": 3,  
    "up_count": 1  
  }  
}
```

如何读取结果

先看 ok: false 表示失败，应读取 error; true 表示成功。成功后先看 summary 中的 output, scope_count, up_count, status。需要逐项明细时再看 data。本页示例取自 kdebug\tests\vm\npi_actions\evidence\responses\fsdb.writer.create_scope.json。

KCov：查询和推进覆盖率

KCov 通过 Verdi Tcl NPI 读取 VDB。只查一次时，可以在命令中直接写 --vdb；需要反复查询同一个 VDB 时，先用 open 打开会话，再执行多次查询会更省时间。为避免大型 VDB 被误判超时，Tcl 调用、会话启动和查询默认不设时限。

关于无限等待

KVERIF_KCOV_TCL_TIMEOUT_SEC=0、KVERIF_KCOV_STARTUP_TIMEOUT_SEC=0 和 KVERIF_KCOV_REQUEST_TIMEOUT_SEC=0 都表示一直等待。关闭会话和清理进程仍有有限等待时间，避免留下后台进程。

公共查询参数

参数	值	含义
--json	无	输出 JSON；默认输出 KOUT。
--session / --session-id	ID	使用已经打开的覆盖率会话。
--vdb	路径	直接指定 VDB；适合只查询一次的情况。
--fake	无	使用工具自带的假数据，只用于检查命令和脚本。
--scope	层次	只查询这个设计层次。
--test	名称	选择 merged 或某个具体 test；当前不支持 each。
--metrics	逗号列表	只查询 line、toggle、branch、condition、fsm 等指定覆盖类型。
--include	glob	包含模式，可重复。
--exclude	glob	排除模式，可重复。
--match-field	字段	指定匹配字段，例如 full_name、name、file。
--case-insensitive	无	名称过滤忽略大小写。
--max-items	整数	最多返回多少条结果。
--overflow	truncate error to_file summary_only	结果超过数量上限时，是截短、报错、写文件，还是只给汇总。
--output-mode	inline file both summary_only	结果放在标准输出、文件、两处都写，或只给汇总。
--output-path	路径	导出路径；相对路径落在 .kverif/kcov_exports。
--artifact-format	json ndjson csv md	导出文件格式。
--allow-absolute-path	无	显式允许绝对输出路径。
--sort-by	字段	结果排序字段。
--sort-order	asc desc	排序方向。
--arg	key=value	写入任意 action args，可重复。
--target	key=value	写入任意 target 字段，可重复。

全部命令

命令	对应操作	用途
actions	actions	列出 KCov 支持的操作。
schema	schema	查看某项 KCov 操作可以输入和返回哪些字段。
open	session.open	打开 VDB，供后续多次查询复用。
status	session.status	查看覆盖率会话是否仍在运行。
close	session.close	关闭覆盖率会话并释放资源。
tests	tests.list	列出 VDB 中包含的测试。
metrics	metrics.list	列出 VDB 中可查询的覆盖类型。
scope-summary	scope.summary	查看某个设计层次的覆盖率汇总。
scope-search	scope.search	按名称或路径查找设计层次。
cov-summary	cov.summary	汇总代码覆盖率。
cov-holes	cov.holes	列出尚未覆盖的代码项。
object-search	cov.object.search	搜索覆盖率对象。
functional-summary	functional.summary	汇总功能覆盖率。
functional-holes	functional.holes	列出尚未命中的 coverpoint、bin 等功能覆盖项。
export-summary	export.summary	导出 coverage 摘要。
export-holes	export.holes	把未覆盖项导出到文件。
export-scope-tree	export.scope_tree	导出设计层次树及各层覆盖率。
export-functional	export.functional	导出功能覆盖率汇总或未覆盖项。
scope-children	scope.children	列出某个设计层次的直接子层，或递归列出全部下级层次。
object-get	cov.object.get	按完整名称读取一个覆盖率对象。
source-map	source.map	根据源码文件和行号查找对应的覆盖率对象。
query	cov.holes	调用任意 KCov action。

kcov actions

列出 KCov 支持的操作。

命令格式

```
/home/host/kverif/tools/kcov actions [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov actions --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "actions",
  "summary": {
    "action_count": 22,
    "categories": [
      "session",
      "scope",
      "coverage",
      "export"
    ]
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov schema

查看某项 KCov 操作可以输入和返回哪些字段。

命令格式

```
/home/host/kverif/tools/kcov schema [common-options] [command-options]
```

参数	要求/默认	说明
--action	必填	操作名
--kind	request	查看请求字段还是返回字段

完整命令示例

```
/home/host/kverif/tools/kcov schema --action cov.holes --kind request --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "schema",
  "summary": {
    "schema_action": "cov.holes",
    "kind": "request",
    "required": [
      "api_version",
      "action"
    ]
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov open

打开 VDB，供后续多次查询复用。

命令格式

```
/home/host/kverif/tools/kcov open [common-options] [command-options]
```

参数	要求/默认	说明
--vdb	必填	VDB 路径
--name	自动	会话别名
--reuse / --no-reuse	默认策略	是否复用已经打开的相同 VDB
--reopen	false	即使已有会话也重新打开
--fake	false	使用测试数据

完整命令示例

```
/home/host/kverif/tools/kcov open --vdb /proj/out/simv.vdb --name nightly --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "session.open",
  "summary": {
    "session_id": "nightly",
    "vdb": "/proj/out/simv.vdb",
    "backend": "tcl_npi"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov status

查看覆盖率会话是否仍在运行。

命令格式

```
/home/host/kverif/tools/kcov status [common-options] [command-options]
```

参数	要求/默认	说明
--session	必填	会话 ID

完整命令示例

```
/home/host/kverif/tools/kcov status --session nightly --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "session.status",
  "summary": {
    "session_id": "nightly",
    "ready": true,
    "test_count": 1
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov close

关闭覆盖率会话并释放资源。

命令格式

```
/home/host/kverif/tools/kcov close [common-options] [command-options]
```

参数	要求/默认	说明
--session	必填	会话 ID

完整命令示例

```
/home/host/kverif/tools/kcov close --session nightly --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "session.close",
  "summary": {
    "session_id": "nightly",
    "closed": true
  }
}
```

结果怎么看

这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov tests

列出 VDB 中包含的测试。

命令格式

```
/home/host/kverif/tools/kcov tests [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov tests --vdb fake --fake --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "tests.list",
  "summary": {
    "matched_count": 1,
    "returned": 1,
    "tests": [
      "merged"
    ]
  }
}
```

结果怎么看

这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov metrics

列出 VDB 中可查询的覆盖类型。

命令格式

```
/home/host/kverif/tools/kcov metrics [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov metrics --vdb fake --fake --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "metrics.list",
  "summary": {
    "matched_count": 5,
    "metrics": [
      "line",
      "toggle",
      "branch",
      "condition",
      "functional"
    ]
  }
}
```

结果怎么看

这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov scope-summary

查看某个设计层次的覆盖率汇总。

命令格式

```
/home/host/kverif/tools/kcov scope-summary [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov scope-summary --vdb fake --fake --scope top.u_dut --metrics line,toggle --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "scope.summary",
  "summary": {
    "matched_count": 1,
    "scope": "top.u_dut",
    "covered": 73,
    "coverable": 100
  }
}
```

结果怎么看

这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov scope-search

按名称或路径查找设计层次。

命令格式

```
/home/host/kverif/tools/kcov scope-search [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov scope-search --vdb fake --fake --include '*fifo*' --max-items 5 --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "scope.search",
  "summary": {
    "matched_count": 1,
    "returned": 1,
    "full_name": "top.u_dut.u_fifo"
  }
}
```

结果怎么看

这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov cov-summary

汇总代码覆盖率。

命令格式

```
/home/host/kverif/tools/kcov cov-summary [common-options] [command-options]
```

参数	要求/默认	说明
--group-by	无	聚合维度

完整命令示例

```
/home/host/kverif/tools/kcov cov-summary --vdb fake --fake --metrics line,toggle,branch --group-by metric --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "cov.summary",
  "summary": {
    "line_pct": 92.5,
    "toggle_pct": 77.3,
    "branch_pct": 84.0
  }
}
```

结果怎么看

这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov cov-holes

列出尚未覆盖的代码项。

命令格式

```
/home/host/kverif/tools/kcov cov-holes [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov cov-holes --vdb fake --fake --metrics toggle,branch --max-items 3 --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "cov.holes",
  "summary": {
    "matched_count": 3,
    "returned": 3,
    "first_hole": "top.u_dut.u_fifo.credit[0] 0 -> 1"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov object-search

搜索覆盖率对象。

命令格式

```
/home/host/kverif/tools/kcov object-search [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov object-search --vdb fake --fake --include '*credit*' --match-field full_name --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "cov.object.search",
  "summary": {
    "matched_count": 1,
    "full_name": "top.u_dut.u_fifo.credit"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov functional-summary

汇总功能覆盖率。

命令格式

```
/home/host/kverif/tools/kcov functional-summary [common-options] [command-options]
```

参数	要求/默认	说明
--group-by	无	聚合维度
--levels	全部	covergroup、coverpoint、cross、bin

完整命令示例

```
/home/host/kverif/tools/kcov functional-summary --vdb fake --fake --levels covergroup,coverpoint --group-by level --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "functional.summary",
  "summary": {
    "covergroup_pct": 88.0,
    "coverpoint_pct": 84.5
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov functional-holes

列出尚未命中的 coverpoint、bin 等功能覆盖项。

命令格式

```
/home/host/kverif/tools/kcov functional-holes [common-options] [command-options]
```

参数	要求/默认	说明
--levels	全部	要返回的功能覆盖层级

完整命令示例

```
/home/host/kverif/tools/kcov functional-holes --vdb fake --fake --levels coverpoint,bin --max-items 5 --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "functional.holes",
  "summary": {
    "matched_count": 2,
    "first_bin": "uart_cp.parity_err"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov export-summary

导出 coverage 摘要。

命令格式

```
/home/host/kverif/tools/kcov export-summary [common-options] [command-options]
```

参数	要求/默认	说明
--group-by	无	聚合维度

完整命令示例

```
/home/host/kverif/tools/kcov export-summary --vdb fake --fake --metrics line,toggle --output-mode summary_only --json
```


运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "export.summary",
  "summary": {
    "matched_count": 5,
    "output_mode": "summary_only"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov export-holes

把未覆盖项导出到文件。

命令格式

```
/home/host/kverif/tools/kcov export-holes [common-options] [command-options]
```

完整命令示例

```
/home/host/kverif/tools/kcov export-holes --vdb fake --fake --metrics branch --output-path branch_holes.json --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "export.holes",
  "summary": {
    "output_path": ".kverif/kcov_exports/branch_holes.json",
    "written": 3
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov export-scope-tree

导出设计层次树及各层覆盖率。

命令格式

```
/home/host/kverif/tools/kcov export-scope-tree [common-options] [command-options]
```

参数	要求/默认	说明
--recursive / --no-recursive	默认后端策略	是否包含全部下级层次

完整命令示例

```
/home/host/kverif/tools/kcov export-scope-tree --vdb fake --fake --scope top --recursive --output-path scope_tree.json --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "export.scope_tree",
  "summary": {
    "output_path": ".kverif/kcov_exports/scope_tree.json",
    "scope_count": 4
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov export-functional

导出功能覆盖率汇总或未覆盖项。

命令格式

```
/home/host/kverif/tools/kcov export-functional [common-options] [command-options]
```

参数	要求/默认	说明
--levels	全部	功能覆盖层级
--mode	summary	导出汇总还是未覆盖项

完整命令示例

```
/home/host/kverif/tools/kcov export-functional --vdb fake --fake --levels coverpoint,bin --mode holes --output-path func_holes.csv --artifact-format csv --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "export.functional",
  "summary": {
    "output_path": ".kverif/kcov_exports/func_holes.csv",
    "written": 2
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov scope-children

列出某个设计层次的直接子层，或递归列出全部下级层次。

命令格式

```
/home/host/kverif/tools/kcov scope-children [common-options] [command-options]
```

参数	要求/默认	说明
--recursive	false	返回全部下级层次

完整命令示例

```
/home/host/kverif/tools/kcov scope-children --vdb fake --fake --scope top.u_dut --recursive --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "scope.children",
  "summary": {
    "matched_count": 2,
    "children": [
      "top.u_dut.u_fifo",
      "top.u_dut.u_ctrl"
    ]
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov object-get

按完整名称读取一个覆盖率对象。

命令格式

```
/home/host/kverif/tools/kcov object-get [common-options] [command-options]
```

参数	要求/默认	说明
--object / --name	必填	完整对象名
--include-children	false	同时返回直接子对象
--max-children	无	最多返回多少个子对象

完整命令示例

```
/home/host/kverif/tools/kcov object-get --vdb fake --fake --object top.u_dut --include-children --max-children 5 --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "cov.object.get",
  "summary": {
    "full_name": "top.u_dut",
    "coverable": 100,
    "child_count": 2
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov source-map

根据源码文件和行号查找对应的覆盖率对象。

命令格式

```
/home/host/kverif/tools/kcov source-map [common-options] [command-options]
```

参数	要求/默认	说明
--file	必填	源码文件
--line	必填	行号
--window	0	同时搜索前后多少行

完整命令示例

```
/home/host/kverif/tools/kcov source-map --vdb fake --fake --file rtl/ctrl.sv --line 88 --window 5 --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "source.map",
  "summary": {
    "file": "rtl/ctrl.sv",
    "line": 88,
    "matched_count": 1,
    "object": "top.u_dut.u_ctrl.branch_7"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

kcov query

调用任意 KCov action。

命令格式

```
/home/host/kverif/tools/kcov query [common-options] [command-options]
```

参数	要求/默认	说明
action	必填	action 位置参数

完整命令示例

```
/home/host/kverif/tools/kcov query cov.holes --vdb fake --fake --arg metrics='["line"]' --max-items 2 --json
```

运行后会看到

```
{
  "api_version": "kcov.v1",
  "ok": true,
  "action": "cov.holes",
  "summary": {
    "matched_count": 3,
    "returned": 3,
    "first_hole": "top.u_dut.u_fifo.credit[0] 0 -> 1"
  }
}
```

结果怎么看
这里的数值只用来说明字段。实际 covered、coverable、missing 和 matched_count 取决于你查询的 VDB。

KBit：位宽和位运算

KBit 专门处理位宽、符号和位运算，不读取 RTL、FSDB，也不调用 NPI。例如，KDebug 查到一个寄存器值后，可以交给 KBit 做切片、符号扩展或条件判断。这样 Bash、Perl 等脚本不用自己解析 8'hff 这类 SystemVerilog 数值。

全部参数

参数	适用范围/默认	含义
value	命令相关	SystemVerilog 数值，例如 8'hff。
expr	eval/check 必填	受支持的 SV 风格表达式。
msb / lsb / bit	slice/index 必填	从 0 开始的 bit 索引。
count	repeat 必填	重复次数。
--to	trunc/zext/sext/align 必填	目标宽度或对齐粒度。
--width	可选或 mask 必填	结果宽度；conv/eval 可在计算后 resize。
--lsb	0	mask 最低置位 bit。
--var name=value	可重复	表达式变量。
--values FILE	可选	从 JSON 文件读取多组变量值。
--state 2 4	2	2 表示只允许 0/1；4 表示保留 X/Z。
--signed / --unsigned	保持输入属性	强制结果有符号或无符号。
--json	false	输出 kbit.result.v1 JSON。
--pretty	false	缩进 JSON。
--stdio	agent serve 必填	使用一行请求一行响应的 stdio 协议。

kbit conv

解析 SystemVerilog 数值并转换显示格式

命令格式

```
/home/host/kverif/tools/kbit conv [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit conv "8'shff" --json --pretty
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "conv",
  "input": "8'shff",
  "result": {
    "width": 8,
    "signed": true,
    "known": true,
    "unsigned": 255,
    "signed_value": -1,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit eval

计算支持的 SystemVerilog 常量表达式

命令格式

```
/home/host/kverif/tools/kbit eval [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit eval "data[15:8] == 8'hbe" --var "data=32'hdead_beef" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "eval",
  "input": "data[15:8] == 8'hbe",
  "result": {
    "width": 1,
    "signed": false,
    "known": true,
    "unsigned": 1,
    "signed_value": -1,
    "_omitted": 4
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit slice

取出一段连续位

命令格式

```
/home/host/kverif/tools/kbit slice [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit slice "32'hdead_beef" 15 8 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "slice",
  "input": "32'hdead_beef",
  "result": {
    "width": 8,
    "signed": false,
    "known": true,
    "unsigned": 190,
    "signed_value": -66,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit index

取出某一位

命令格式

```
/home/host/kverif/tools/kbit index [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit index "8'h80" 7 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "index",
  "input": "8'h80",
  "result": {
    "width": 1,
    "signed": false,
    "known": true,
    "unsigned": 1,
    "signed_value": -1,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit concat

按顺序拼接多个位向量

命令格式

```
/home/host/kverif/tools/kbit concat [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit concat "4'ha" "4'h5" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "concat",
  "input": [
    "4'ha",
    "... 1 more"
  ],
  "result": {
    "width": 8,
    "signed": false,
    "known": true,
    "unsigned": 165,
    "signed_value": -91,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit repeat

把一个位向量重复指定次数

命令格式

```
/home/host/kverif/tools/kbit repeat [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit repeat 4 "2'b10" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "repeat",
  "input": {
    "count": 4,
    "value": "2'b10"
  },
  "result": {
    "width": 8,
    "signed": false,
    "known": true,
    "unsigned": 170,
    "signed_value": -86,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit trunc

截断到指定宽度

命令格式

```
/home/host/kverif/tools/kbit trunc [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit trunc "16'h12ff" --to 8 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "trunc",
  "input": "16'h12ff",
  "result": {
    "width": 8,
    "signed": false,
    "known": true,
    "unsigned": 255,
    "signed_value": -1,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit zext

零扩展到指定宽度

命令格式

```
/home/host/kverif/tools/kbit zext [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit zext "8'h80" --to 16 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "zext",
  "input": "8'h80",
  "result": {
    "width": 16,
    "signed": false,
    "known": true,
    "unsigned": 128,
    "signed_value": 128,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit sext

符号扩展到指定宽度

命令格式

```
/home/host/kverif/tools/kbit sext [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit sext "8'h80" --to 16 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "sext",
  "input": "8'h80",
  "result": {
    "width": 16,
    "signed": true,
    "known": true,
    "unsigned": 65408,
    "signed_value": -128,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit reverse

把位顺序首尾反转

命令格式

```
/home/host/kverif/tools/kbit reverse [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit reverse "8'b1000_0001" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "reverse",
  "input": "8'b1000_0001",
  "result": {
    "width": 8,
    "signed": false,
    "known": true,
    "unsigned": 129,
    "signed_value": -127,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit mask

生成一段连续为 1 的掩码

命令格式

```
/home/host/kverif/tools/kbit mask [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit mask --width 13 --lsb 3 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "mask",
  "input": {
    "width": 13,
    "lsb": 3
  },
  "result": {
    "width": 16,
    "signed": false,
    "known": true,
    "unsigned": 65528,
    "signed_value": -8,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit align

向上对齐数值

命令格式

```
/home/host/kverif/tools/kbit align [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit align "13'd17" --to 8 --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "align",
  "input": "13'd17",
  "result": {
    "width": 13,
    "signed": false,
    "known": true,
    "unsigned": 24,
    "signed_value": 24,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit popcount

统计有多少位为 1

命令格式

```
/home/host/kverif/tools/kbit popcount [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit popcount "32'hdead_beef" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "popcount",
  "input": "32'hdead_beef",
  "result": {
    "width": 5,
    "signed": false,
    "known": true,
    "unsigned": 24,
    "signed_value": -8,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit onehot

判断是否恰好只有一位为 1

命令格式

```
/home/host/kverif/tools/kbit onehot [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit onehot "8'h20" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "onehot",
  "input": "8'h20",
  "result": {
    "width": 1,
    "signed": false,
    "known": true,
    "unsigned": 1,
    "signed_value": -1,
    "_omitted": 4
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit onehot0

判断是否没有位为 1，或只有一位为 1

命令格式

```
/home/host/kverif/tools/kbit onehot0 [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit onehot0 "8'h00" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "onehot0",
  "input": "8'h00",
  "result": {
    "width": 1,
    "signed": false,
    "known": true,
    "unsigned": 1,
    "signed_value": -1,
    "_omitted": 4
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit gray2bin

Gray code 转二进制

命令格式

```
/home/host/kverif/tools/kbit gray2bin [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit gray2bin "4'b1110" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "gray2bin",
  "input": "4'b1110",
  "result": {
    "width": 4,
    "signed": false,
    "known": true,
    "unsigned": 11,
    "signed_value": -5,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit bin2gray

二进制转 Gray code

命令格式

```
/home/host/kverif/tools/kbit bin2gray [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit bin2gray "4'b1011" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "bin2gray",
  "input": "4'b1011",
  "result": {
    "width": 4,
    "signed": false,
    "known": true,
    "unsigned": 14,
    "signed_value": -2,
    "_omitted": 3
  },
  "warnings": []
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit check

用变量或 values 文件验证条件

命令格式

```
/home/host/kverif/tools/kbit check [options]
```

完整命令示例

```
/home/host/kverif/tools/kbit check --expr 'valid && ready' --var "valid=1'b1" --var "ready=1'b1" --json
```

运行后会看到

```
{
  "ok": true,
  "schema": "kbit.result.v1",
  "op": "check",
  "input": "valid && ready",
  "result": {
    "width": 1,
    "signed": false,
    "known": true,
    "unsigned": 1,
    "signed_value": -1,
    "_omitted": 3
  },
  "matched": true,
  "evaluated": {
    "ready": "1'h1",
    "valid": "1'h1"
  },
  "_omitted": 1
}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

kbit agent serve

从标准输入逐行接收 JSON，并逐行返回结果

命令格式

```
/home/host/kverif/tools/kbit agent serve [options]
```

参数	要求/默认	说明
--stdio	必填	从标准输入逐行读取 JSON，并把结果逐行写到标准输出

完整命令示例

```
/home/host/kverif/tools/kbit agent serve --stdio
```

运行后会看到

```
{"id":1,"ok":true,"result":{"op":"eval","result":{"bool":true}}}
```

结果怎么看

退出码为 0 且 ok=true 才表示计算成功。成功后从 result 读取计算值；失败时读取 error.code。

KEntry：拼接并拆解多拍数据

KEntry 用来处理一个字段分散在多个时钟拍中的情况。YAML 文件说明每个字段占哪些位，JSONL 文件保存每一拍采到的数据片段。KEntry 会先拼成完整数据，再返回各字段的原始值，以及每个字段来自哪些片段。它不会直接读取 FSDB，也不会猜测协议含义；采集波形并生成 fragments 的工作由调用脚本负责。

全部参数

参数	要求/默认	含义
--config FILE	decode/explain/validate 必填	YAML 字段布局。
--input FILE	decode 必填, validate 可选	JSONL 格式的逐拍数据片段。
--json	false	输出 JSON。
--pretty	false	缩进 JSON。
- / request.json / '{...}'	JSON 兼容入口	从 stdin、文件或字面量读取完整 request。

kentry decode

按配置拼接多拍数据，并拆出各字段的原始值

命令格式

```
/home/host/kverif/tools/kentry decode [--config FILE] [--input FILE] [--json] [--pretty]
```

参数	要求/默认	说明
--config	必填	字段布局 YAML 文件
--input	decode 必填	逐拍数据片段 JSONL 文件
--json	建议	让脚本读取 JSON 结果

完整命令示例

```
/home/host/kverif/tools/kentry decode --config /home/host/kverif/kentry/examples/entry.yaml --input /home/host/kverif/kentry/examples/fragments.jsonl --json
```

运行后会看到

```
{
  "ok": true,
  "api_version": "kentry.v1",
  "action": "decode",
  "schema": {
    "name": "demo_entry",
    "version": 1
  },
  "total_bits": 20,
  "entry_raw": "0xab234",
  "fields": {
    "opcode": {
      "bits": "[3:0]",
      "msb": 3,
      "lsb": 0,
      "width": 4,
      "raw_hex": "0x4",
      "_omitted": 2
    },
    ... output excerpt: 19 lines omitted ...
  }
}
```

结果怎么看

decode 成功后，从 fields.*.raw_hex 或 raw_bin 读取字段值，从 source 查看字段来自哪些片段。errors 非空时不要继续给出成功结论。

kentry explain

查看配置中每个字段占哪些位

命令格式

```
/home/host/kverif/tools/kentry explain [--config FILE] [--input FILE] [--json] [--pretty]
```

参数	要求/默认	说明
--config	必填	字段布局 YAML 文件
--json	建议	让脚本读取 JSON 结果

完整命令示例

```
/home/host/kverif/tools/kentry explain --config /home/host/kverif/kentry/examples/entry.yaml --json
```

运行后会看到

```
{
  "ok": true,
  "api_version": "kentry.v1",
  "action": "explain",
  "schema": {
    "name": "demo_entry",
    "version": 1
  },
  "total_bits": 20,
  "fragment_byte_order": "msb_first",
  "bit_numbering": "byte_lsb0",
  "_omitted": 3
}
```

结果怎么看
decode 成功后，从 fields.*.raw_hex 或 raw_bin 读取字段值，从 source 查看字段来自哪些片段。errors 非空时不要继续给出成功结论。

kentry validate

检查配置和采样数据格式是否正确

命令格式

```
/home/host/kverif/tools/kentry validate [--config FILE] [--input FILE] [--json] [--pretty]
```

参数	要求/默认	说明
--config	必填	字段布局 YAML 文件
--input	decode 必填	逐拍数据片段 JSONL 文件
--json	建议	让脚本读取 JSON 结果

完整命令示例

```
/home/host/kverif/tools/kentry validate --config /home/host/kverif/kentry/examples/entry.yaml --input /home/host/kverif/kentry/examples/fragments.jsonl --json
```


运行后会看到

```
{
  "ok": true,
  "api_version": "kentry.v1",
  "action": "validate",
  "schema": {
    "name": "demo_entry",
    "version": 1
  },
  "total_bits": 20,
  "warnings": [],
  "errors": []
}
```

结果怎么看
decode 成功后，从 fields.raw_hex 或 raw_bin 读取字段值，从 source 查看字段来自哪些片段。errors 非空时不要继续给出成功结论。

KLoc: 从日志编号找到源码

KLoc 把仿真日志中的短编号 loc_id 还原成源码文件和行号。它还能显示目标行前后的源码、统计哪些位置重复报错，并在原日志旁补上源码位置。KLoc 只依赖 Python 标准库。

全部参数

参数	要求/默认	含义
loc_id	resolve/context 必填	例如 L_00000001。
log	stats/annotate 必填	仿真日志路径。
--map FILE	resolve/context 必填	loc_id 与源码位置的 JSONL 对照表；stats/annotate 可自动查找。
--before N / --after N	20	目标行前后各显示多少行源码。
--top N	20	最多列出多少个高频报错位置。
--json	false	resolve/context/stats 输出 JSON。

kloc resolve

把 loc_id 还原为文件和行号

命令格式

```
/home/host/kverif/tools/kloc resolve [options]
```

完整命令示例

```
/home/host/kverif/tools/kloc resolve L_00000001 --map /proj/out/sim.log.kloc.jsonl --json
```

运行后会看到

```
{
  "ok": true,
  "action": "resolve",
  "loc_id": "L_00000001",
  "file": "/proj/tb/scoreboard.sv",
  "line": 3,
  "msg_id": "PKT_MISMATCH"
}
```

结果怎么看
resolve 或 context 失败时不要猜源码位置。先检查 --map 文件是否正确，再确认 loc_id 是否存在。

kloc context

显示 loc_id 附近源码

命令格式

```
/home/host/kverif/tools/kloc context [options]
```

完整命令示例

```
/home/host/kverif/tools/kloc context L_00000001 --map /proj/out/sim.log.kloc.jsonl --before 3 --after 3 --json
```

运行后会看到

```
{
  "ok": true,
  "action": "context",
  "loc_id": "L_00000001",
  "file": "/proj/tb/scoreboard.sv",
  "line": 3,
  "msg_id": "PKT_MISMATCH",
  "context": [
    {
      "line": 2,
      "hit": false,
      "text": " function void check();"
    },
    "... 2 more"
  ]
}
```

结果怎么看

resolve 或 context 失败时不要猜源码位置。先检查 --map 文件是否正确，再确认 loc_id 是否存在。

kloc stats

统计日志中的高频 loc_id

命令格式

```
/home/host/kverif/tools/kloc stats [options]
```

完整命令示例

```
/home/host/kverif/tools/kloc stats /proj/out/sim.log --map /proj/out/sim.log.kloc.jsonl --top 10 --json
```

运行后会看到

```
{
  "ok": true,
  "action": "stats",
  "log": "/proj/tb/sim.log",
  "map": "/proj/tb/sim.log.kloc.jsonl",
  "unique_locations": 1,
  "total_occurrences": 2,
  "rows": [
    {
      "count": 2,
      "loc_id": "L_00000001",
      "file": "/proj/tb/scoreboard.sv",
      "line": 3,
      "msg_id": "PKT_MISMATCH"
    }
  ]
}
```

结果怎么看

resolve 或 context 失败时不要猜源码位置。先检查 --map 文件是否正确，再确认 loc_id 是否存在。

kloc annotate

把位置提示插入日志输出

命令格式

```
/home/host/kverif/tools/kloc annotate [options]
```

完整命令示例

```
/home/host/kverif/tools/kloc annotate /proj/out/sim.log --map /proj/out/sim.log.kloc.jsonl
```

运行后会看到

```
[loc] L_00000001 -> /proj/tb/scoreboard.sv:3
UVM_ERROR L_00000001 @ 100ns: packet mismatch
UVM_ERROR L_00000001 @ 120ns: packet mismatch
```

结果怎么看

resolve 或 context 失败时不要猜源码位置。先检查 --map 文件是否正确，再确认 loc_id 是否存在。

KSVA：读懂和检查断言

KSVA 用来阅读和检查 SVA 断言。它既能生成一段通俗说明，也能把断言拆成 Surface IR、Sequence IR、Timeline IR 三种结构化结果，供脚本继续分析。KSVA 不运行仿真，只处理断言源码。

全部参数

参数	要求/默认	含义
--file FILE	全部命令必填	SVA/SystemVerilog 文件。
--property NAME	lint 可选; explain/parse 必填	目标 property。
--json	explain 可选	输出解释 JSON。
--markdown	explain 可选	输出 Markdown。
--strict	false	遇到暂不支持的语法时立即报错退出。
--emit LEVEL	parse 必填	选择输出 surface-ir、sequence-ir 或 timeline-ir。

ksva list

列出文件中的 property 和 assertion

命令格式

```
/home/host/kverif/tools/ksva list [options]
```

完整命令示例

```
/home/host/kverif/tools/ksva list --file /proj/tb/checks.sva
```

运行后会看到

```
Properties:
  p_req_ack
  p_data_match
  p_rose_req
Assertions:
  a_req_ack: assert property (p_req_ack)
  a_data_match: assert property (p_data_match)
```

结果怎么看

CI 中建议使用 `--strict`，让不支持的语法直接失败。写给人看的报告用 `explain`；脚本继续分析时用 `parse` 返回的 IR JSON。

ksva scan

统计 SVA 语法构造

命令格式

```
/home/host/kverif/tools/ksva scan [options]
```

完整命令示例

```
/home/host/kverif/tools/ksva scan --file /proj/tb/checks.sva
```

运行后会看到

```
File: /home/host/kverif/ksva/tests/golden_ir/test_input.sva
Property blocks: 3
Inline assertions: 2
Operators:
  |-> 2
  |=> 1
  ##N 2
  $rose 1
```

结果怎么看

CI 中建议使用 `--strict`，让不支持的语法直接失败。写给人看的报告用 `explain`；脚本继续分析时用 `parse` 返回的 IR JSON。

ksva lint

检查断言中容易出错或暂不支持的写法

命令格式

```
/home/host/kverif/tools/ksva lint [options]
```

完整命令示例

```
/home/host/kverif/tools/ksva lint --file /proj/tb/checks.sva --property p_req_ack
```

运行后会看到

```
No issues found.
```

结果怎么看

CI 中建议使用 `--strict`，让不支持的语法直接失败。写给人看的报告用 `explain`；脚本继续分析时用 `parse` 返回的 IR JSON。

ksva explain

用较容易阅读的文字解释断言检查什么

命令格式

```
/home/host/kverif/tools/ksva explain [options]
```

完整命令示例

```
/home/host/kverif/tools/ksva explain --file /proj/tb/checks.sva --property p_req_ack --json --strict
```

运行后会看到

```
{
  "tool": "ksva",
  "command": "explain",
  "result": {
    "schema_version": "ksva.timeline_ir.v1",
    "property": "p_req_ack",
    "kind": "property",
    "clock": {
      "signal": "clk",
      "edge": "posedge"
    },
    "disable_expr": "! rst_n",
    "_omitted": 6
  },
  "diagnostics": []
}
```

结果怎么看

CI 中建议使用 `--strict`，让不支持的语法直接失败。写给人看的报告用 `explain`；脚本继续分析时用 `parse` 返回的 IR JSON。

ksva parse

把断言输出为供程序继续处理的结构化数据

命令格式

```
/home/host/kverif/tools/ksva parse [options]
```

完整命令示例

```
/home/host/kverif/tools/ksva parse --file /proj/tb/checks.sva --property p_req_ack --emit timeline-ir
```

运行后会看到

```
{
  "schema_version": "ksva.timeline_ir.v1",
  "property": "p_req_ack",
  "kind": "property",
  "clock": {
    "signal": "clk",
    "edge": "posedge"
  },
  "disable_expr": "! rst_n",
  "trigger": {
    "cycle": 0,
    "expr": "req",
    "captures": []
  },
  "obligations": [
    {
      "id": "ob_0_0",
      "kind": "eventually",
      "expr": "ack",
      ... output excerpt: 7 lines omitted ...
    }
  ]
}
```

结果怎么看
CI 中建议使用 `--strict`，让不支持的语法直接失败。写给人看的报告用 `explain`；脚本继续分析时用 `parse` 返回的 IR JSON。

KBerif：保存和查询验证环境知识

KBerif 用来保存和查询验证环境知识。每个主题有一份简短摘要，需要时还可以展开详细说明。命令默认输出便于人阅读的 KOUT；脚本需要 JSON 时，把全局参数 `--json` 放在子命令前，例如 `kberif --json status`。

公共与命令参数

参数	要求/默认	含义
<code>--json</code>	全局, false	查询命令输出 JSON；放在子命令前。
<code>--kind bt it st soc</code>	config init 必填	环境类型。
<code>--force / --merge</code>	false	覆盖或合并已有配置。
<code>--dry-run</code>	false	只展示要写入的文件。
<code>--output DIR</code>	当前目录	配置输出项目根目录。
<code>--model NAME</code>	init 必填	首次生成知识条目时调用的模型名；配置文件不会保存密钥。
<code>--all</code>	validate 可选	校验全部状态。
<code>topic / card_id</code>	命令相关	主题名或知识条目 ID。
<code>--detail</code>	false	让 <code>get</code> 直接返回详细说明。
<code>--mode MODE</code>	brief 必填	指定使用场景，例如 <code>debug</code> 。
<code>--stdin</code>	写命令必填	从标准输入读取 JSON 或 Markdown。
<code>--stdio</code>	agent serve 必填	通过标准输入/输出逐行收发 JSON。
<code>--write</code>	false	允许 agent 修改知识库。

kberif config init

创建一种验证环境的基础配置

命令格式

```
/home/host/kverif/tools/kberif [--json] config init [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif config init --kind bt --dry-run --output /proj/nic
```

运行后会看到

```
列出将创建的 kind.toml 和模板路径。
```

结果怎么看
请通过 KBerif 命令维护 `.kberif` 目录。业务脚本不要直接修改 `cards.json` 或详细说明索引。

kberif init

调用指定模型生成第一批知识条目

命令格式

```
/home/host/kverif/tools/kberif [--json] init [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif init --model opus
```

运行后会看到

显示初始化进度、生成条目数和检查结果。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif validate

检查目录、知识条目和详细说明是否一致

命令格式

```
/home/host/kverif/tools/kberif [--json] validate [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif validate --all
```

运行后会看到

ok

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif status

查看知识库状态

命令格式

```
/home/host/kverif/tools/kberif [--json] status [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif status
```

运行后会看到

返回 env_kind、card_count、detail_count 和 validation 状态。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif repair-catalog

根据已有知识条目重建目录

命令格式

```
/home/host/kverif/tools/kberif [--json] repair-catalog [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif repair-catalog
```

运行后会看到

返回目录中的条目数量 catalog_card_count。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif list-topics

列出可以查询的主题

命令格式

```
/home/host/kverif/tools/kberif [--json] list-topics [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif list-topics
```

运行后会看到

返回主题名称和摘要。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif get

读取某个主题的摘要或详细说明

命令格式

```
/home/host/kverif/tools/kberif [--json] get [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif get backpressure --detail
```

运行后会看到

加 --detail 时输出 Markdown 详细说明；不加时输出结构化摘要。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif detail

读取详细说明，或从标准输入更新它

命令格式

```
/home/host/kverif/tools/kberif [--json] detail [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif detail backpressure
```

运行后会看到

输出该主题的 Markdown 详细说明。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif brief

按当前工作场景整理一份精简说明

命令格式

```
/home/host/kverif/tools/kberif [--json] brief [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif brief --mode debug
```

运行后会看到

输出可直接交给调试脚本或 AI 客户端的短文本。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif card upsert

从标准输入新增或更新知识条目

命令格式

```
/home/host/kverif/tools/kberif [--json] card upsert [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif card upsert --stdin < /proj/card.json
```

运行后会看到

成功时写入条目；格式错误时返回具体字段错误。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif card append-key-items

向已有知识条目追加要点

命令格式

```
/home/host/kverif/tools/kberif [--json] card append-key-items [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif card append-key-items backpressure --stdin < /proj/items.json
```

运行后会看到

写入后再次检查数据格式。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif agent serve

从标准输入逐行接收 JSON 请求

命令格式

```
/home/host/kverif/tools/kberif [--json] agent serve [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif agent serve --stdio
```

运行后会看到

每行返回一条 JSON；加 --write 才允许修改知识库。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif bt / it / st / soc

在指定验证环境中读取主题

命令格式

```
/home/host/kverif/tools/kberif [--json] bt / it / st / soc [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif bt / it / st / soc bt backpressure
```

运行后会看到

返回对应环境中的主题摘要。

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif hook

内部 Claude hook 校验入口

命令格式

```
/home/host/kverif/tools/kberif [--json] hook [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif hook validate-write
```

运行后会看到

```
退出码表达 allow/deny；外部脚本通常不直接调用。
```

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

kberif bootstrap-state

测试用无模型状态初始化

命令格式

```
/home/host/kverif/tools/kberif [--json] bootstrap-state [options]
```

完整命令示例

```
cd /proj/nic && /home/host/kverif/tools/kberif bootstrap-state
```

运行后会看到

```
创建空状态；仅用于测试和维护。
```

结果怎么看

请通过 KBerif 命令维护 .kberif 目录。业务脚本不要直接修改 cards.json 或详细说明索引。

KEDA Runner：按批准清单运行 EDA 命令

KEDA Runner 只允许运行配置文件中明确批准的 EDA 命令和参数。init 会保存当前 EDA 环境，run 会在前台等待命令执行结束并返回相同退出码。长任务请放在 tmux、nohup 或调度系统中运行，避免终端断开后任务被终止。

全部参数

参数	要求/默认	含义
--config FILE	.keda-runner.yaml	配置文件，可放在主命令前。
--refresh	false	让 init 重新读取并保存当前环境。
--action NAME	describe/run 必填	配置中批准的操作名。
--target NAME	由操作定义决定	该操作允许使用的目标，例如 compile 或 run。
--option KEY=VALUE	可重复	给操作传参数；可选值和格式受配置文件限制。
--quiet	false	不打印 KEDA Runner 的开头信息。
--dry-run	false	只检查并打印最终命令，不真正执行，也不要求先运行 init。

keda-runner init

读取 EDA 环境并保存一份可复用记录

命令格式

```
/home/host/kverif/tools/keda-runner [--config FILE] init [options]
```

完整命令示例

```
/home/host/kverif/tools/keda-runner init --refresh
```

运行后会看到

```
[keda-runner] snapshot: /home/host/.keda_runner/env.snapshot  
[keda-runner] checks: vcs=/home/synopsys/vcs/bin/vcs  
[keda-runner] status=ready
```

结果怎么看

run 返回的退出码与实际 EDA 命令相同。监控脚本还可以读取 runner_pid、child_pid 和日志路径。

keda-runner env-info

查看已经保存的 EDA 环境

命令格式

```
/home/host/kverif/tools/keda-runner [--config FILE] env-info [options]
```

完整命令示例

```
/home/host/kverif/tools/keda-runner env-info
```

运行后会看到

```
shell: tcsh  
workdir: /proj/nic/work  
snapshot: ready  
checks.vcs: /home/synopsys/vcs/bin/vcs
```

结果怎么看

run 返回的退出码与实际 EDA 命令相同。监控脚本还可以读取 runner_pid、child_pid 和日志路径。

keda-runner list-actions

列出配置允许执行的操作

命令格式

```
/home/host/kverif/tools/keda-runner [--config FILE] list-actions [options]
```

完整命令示例

```
/home/host/kverif/tools/keda-runner list-actions
```

运行后会看到

```
sim  
regression  
coverage
```

结果怎么看

run 返回的退出码与实际 EDA 命令相同。监控脚本还可以读取 runner_pid、child_pid 和日志路径。

keda-runner describe-action

查看某项操作最终会运行什么

命令格式

```
/home/host/kverif/tools/keda-runner [--config FILE] describe-action [options]
```

完整命令示例

```
/home/host/kverif/tools/keda-runner describe-action --action sim
```

运行后会看到

```
action: sim
command: make
fixed_args: [-j8]
targets: [compile, run]
options: [TEST, SEED]
```

结果怎么看

run 返回的退出码与实际 EDA 命令相同。监控脚本还可以读取 runner_pid、child_pid 和日志路径。

keda-runner run

在前台运行一项已批准的命令

命令格式

```
/home/host/kverif/tools/keda-runner [--config FILE] run [options]
```

完整命令示例

```
/home/host/kverif/tools/keda-runner run --action sim --target compile --option TEST=smoke --option SEED=123
```

运行后会看到

```
[keda-runner] runner_pid=543212 child_pid=543213
[keda-runner] action=sim target=compile
[keda-runner] command: make -j8 compile TEST=smoke SEED=123
[keda-runner] exit_code=0
```

结果怎么看

run 返回的退出码与实际 EDA 命令相同。监控脚本还可以读取 runner_pid、child_pid 和日志路径。

复用长会话：Loop、MCP 与 LSF

Loop 让 Bash、Perl、Python 等脚本通过普通命令复用 KDebug 或 KCov 会话，不需要安装或调用任何 SDK。本机通信使用 Unix socket。MCP 则是给 AI 客户端准备的可选入口；两种入口使用同一套会话和启动方式，都可以在本机直接运行，也可以提交到 LSF。

启动 Loop 服务

参数	默认	含义
--socket PATH	/tmp/kverif-loop-<uid>.sock	脚本与服务通信使用的 Unix socket 路径。
--backend direct lsf	环境变量或 direct	direct 表示本机启动，lsf 表示提交到 LSF。

启动命令

```
KVERIF_LOOP_BACKEND=direct /home/host/kverif/tools/kverif-loop-server \
--socket /tmp/kverif-loop-host.sock
```

示例输出

```
kverif_loop_server:
  socket: /tmp/kverif-loop-host.sock
  backend: direct
  ready: true
```

Loop 客户端参数

参数	默认	含义
--socket PATH	按 UID 生成	与 Loop 服务相同的 socket 路径。
--timeout-sec N	cov.* 无限，其他 30s	等待返回结果的秒数；0 或负数表示一直等待。
--pretty	false	缩进 JSON-RPC 返回结果。
--json OBJECT	无	发送一条完整 JSON-RPC 请求；否则可从 stdin 读取 JSONL。
--session ID	query/close 必填	要查询或关闭的会话别名。
--action NAME	query 必填	要执行的 KDebug 或 KCov 操作。
--output-format	kout	选择 KOUT、JSON，或带 JSON-RPC 外层字段的 envelope。
--arg key=value	可重复	传给这项操作的功能参数。
--limit key=value	可重复	限制最多返回多少项或查多深。
--output key=value	可重复	设置结果格式和详细程度。
--queue / --resource	无	使用 LSF 时指定队列和资源。

kverif-loop-client ping

检查会话服务是否仍在运行

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] ping [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock ping
```

运行后会看到

```
{
  "id": "cli-ping",
  "ok": true,
  "method": "server.ping",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看
同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client debug-open

打开可重复查询的 KDebug 会话

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] debug-open [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock debug-open --name wave0 --fsdb /proj/out/waves.fsdb --daidir /proj/out/simv.daidir
```

运行后会看到

```
{
  "id": "cli-debug-open",
  "ok": true,
  "method": "debug.session.open",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client debug-list

列出已经打开的 KDebug 会话

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] debug-list [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock debug-list
```

运行后会看到

```
{
  "id": "cli-debug-list",
  "ok": true,
  "method": "debug.session.list",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client debug-close

关闭一个 KDebug 会话

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] debug-close [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock debug-close --session wave0
```

运行后会看到

```
{
  "id": "cli-debug-close",
  "ok": true,
  "method": "debug.session.close",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client debug-query

在已打开的 KDebug 会话中执行查询

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] debug-query [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock debug-query --session wave0 --action value.at --arg signal=top.clk --arg time=10ns --output-format json
```

运行后会看到

```
{
  "id": "cli-debug-query",
  "ok": true,
  "method": "debug.query",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client cov-open

打开可重复查询的 KCov 会话

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] cov-open [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock cov-open --name cov0 --vdb /proj/out/simv.vdb
```

运行后会看到

```
{
  "id": "cli-cov-open",
  "ok": true,
  "method": "cov.session.open",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client cov-list

列出已经打开的 KCov 会话

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] cov-list [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock cov-list
```

运行后会看到

```
{
  "id": "cli-cov-list",
  "ok": true,
  "method": "cov.session.list",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client cov-close

关闭一个 KCov 会话

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] cov-close [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock cov-close --session cov0
```

运行后会看到

```
{
  "id": "cli-cov-close",
  "ok": true,
  "method": "cov.session.close",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

kverif-loop-client cov-query

在已打开的 KCov 会话中执行查询

命令格式

```
/home/host/kverif/tools/kverif-loop-client [common-options] cov-query [options]
```

完整命令示例

```
/home/host/kverif/tools/kverif-loop-client --socket /tmp/kverif-loop-host.sock cov-query --session cov0 --action cov.holes --arg metrics='["branch"]' --limit max_items=5 --output-format json
```

运行后会看到

```
{
  "id": "cli-cov-query",
  "ok": true,
  "method": "cov.query",
  "result": {
    "status": "ok"
  }
}
```

结果怎么看

同一个会话中的请求会依次执行，不同会话可以并行。覆盖率查询默认一直等待，直到返回结果。

给 AI 客户端使用的 MCP 入口

启动命令

```
PYTHONPATH=/home/host/kverif/kverif_mcp/src:/home/host/kverif \
KVERIF_HOME=/home/host/kverif \
KVERIF_MCP_BACKEND=direct \
/home/host/kverif/tools/kverif-mcp
```

MCP 不是普通脚本的必需项。Shell、Perl 和 Python 脚本直接运行 tools 目录中的命令即可；只有需要通过 MCP 协议调用工具的 AI 客户端才要使用这一入口。下表列出当前注册的 MCP 工具和参数。

MCP 工具	所属组	参数	功能
kverif_ping	通用		检查 KVerif MCP 服务是否仍在运行。
kverif_batch	通用	batch_file, output_file	读取 NDJSON 批处理文件，按顺序执行其中的多项 MCP 工具请求。
kverif_debug_list_actions	设计/波形		列出 KDebug 当前支持的操作。
kverif_debug_get_schema	设计/波形	action, kind='request'	查看某项 KDebug 操作可以输入和返回哪些 JSON 字段。
kverif_debug_raw_request	设计/波形	request, output_format='kout'	直接执行一份完整的 KDebug JSON 请求，不打开可复用会话。
kverif_debug_session_open	设计/波形	name, daidir=None, fsdb=None, queue=None, resource=None	打开可重复查询的 KDebug 设计或波形会话。
kverif_debug_session_list	设计/波形	include_native=False	列出由 MCP 服务管理的 KDebug 会话。
kverif_debug_session_close	设计/波形	name=None, session_id=None	关闭 KDebug 会话并释放后台进程。
kverif_debug_query	设计/波形	session, action, args=None, limits=None, output=None, output_format='kout'	在已打开的 KDebug 会话中执行一项操作。
kverif_cov_list_actions	覆盖率		列出 KCov 当前支持的操作。
kverif_cov_get_schema	覆盖率	action, kind='request'	查看某项 KCov 操作可以输入和返回哪些 JSON 字段。
kverif_cov_raw_request	覆盖率	request, output_format='kout'	直接执行一份完整的 KCov JSON 请求，不打开可复用会话。
kverif_cov_session_open	覆盖率	name, vdb, queue=None, resource=None	打开可重复查询的覆盖率数据库会话。
kverif_cov_session_list	覆盖率		列出由 MCP 服务管理的 KCov 会话。
kverif_cov_session_close	覆盖率	name=None, session_id=None	关闭 KCov 会话并释放后台进程。
kverif_cov_query	覆盖率	session, action, args=None, limits=None, output=None, output_format='kout'	在已打开的 KCov 会话中执行一项覆盖率查询。
kverif_bit_convert	位运算	value, width=0, signed=False, unsigned=False, state='2', output_format='kout'	解析 SystemVerilog 数值，并转换进制、位宽或符号属性。
kverif_bit_eval	位运算	expr, vars=None, width=0, signed=False, unsigned=False, state='2', output_format='kout'	计算一条位运算或常量表达式。
kverif_bit_slice	位运算	value, msb, lsb, state='2', output_format='kout'	从一个数值中取出指定的连续位。
kverif_bit_check	位运算	expr, vars=None, values=None, state='2', output_format='kout'	代入变量值，判断位表达式是否成立。
kverif_entry_decode	多拍数据	config_path=None, input_path=None, config=None, fragments=None, output_format='kout'	按配置拼接多拍数据，并拆出各字段的原始值。
kverif_entry_explain	多拍数据	config_path, output_format='kout'	查看多拍数据配置中每个字段占哪些位。
kverif_entry_validate	多拍数据	config_path=None, input_path=None, config=None, fragments=None, output_format='kout'	检查多拍数据配置和输入片段格式是否正确。
kverif_loc_resolve	日志定位	loc_id, map_path, output_format='kout'	把日志中的 loc_id 还原成源码文件和行号。
kverif_loc_context	日志定位	loc_id, map_path, before=20, after=20, output_format='kout'	根据 loc_id 显示目标行前后的源码。
kverif_loc_stats	日志定位	log_path, map_path=None, top=20, output_format='kout'	统计仿真日志中重复最多的 loc_id。
kverif_loc_annotate	日志定位	log_path, map_path=None, output_format='kout'	在仿真日志旁补上对应的源码位置。
kverif_context_status	知识查询	project_root=None, output_format='kout'	查看 KBerif 知识库中有哪些环境、主题和详细说明。
kverif_context_topics	知识查询	project_root=None, output_format='kout'	列出 KBerif 中可以查询的主题。
kverif_context_brief	知识查询	mode='debug', project_root=None, output_format='kout'	按指定工作场景整理一份精简的验证环境说明。
kverif_context_topic	知识查询	topic, detail=False, project_root=None, output_format='kout'	读取一个主题的摘要，并可选择同时返回详细说明。
kverif_context_topic_detail	知识查询	topic, project_root=None, output_format='markdown'	读取一个主题的完整 Markdown 说明。
kverif_context_validate	知识查询	project_root=None, output_format='kout'	检查 KBerif 的主题摘要和详细说明是否一致。
kverif_context_init_config	知识维护	kind, project_root=None	创建 KBerif 环境类型配置；必须显式启用写权限。
kverif_context_init_project	知识维护	model, project_root=None	初始化 KBerif 项目目录；必须显式启用写权限。
kverif_context_repair_index	知识维护	project_root=None	根据已有知识条目重建 KBerif 目录；必须显式启用写权限。

MCP 工具	所属组	参数	功能
kverif_sva_list_properties	断言	file, output_format='kout'	列出 SVA 文件中的 property 和 assertion 名称。
kverif_sva_scan_constructs	断言	file, output_format='kout'	统计 SVA 文件中使用了哪些语法结构。
kverif_sva_parse_property	断言	file, property, emit='timeline-ir', output_format='kout'	把一条 SVA property 转成供程序继续处理的结构化数据。
kverif_sva_explain_property	断言	file, property, strict=False, output_format='kout'	用较容易阅读的文字解释一条 SVA property。
kverif_tools	通用	category=None, include_write=True	列出当前 MCP 服务已经注册的 KVerif 工具。
kverif_tool_help	通用	name	查看一个 KVerif MCP 工具的参数和详细说明。
kverif_wave_value_at	设计/波形	signal, time='0ns', session='', output_format='kout'	读取某个信号在指定波形时间的值。
kverif_wave_changes	设计/波形	signal, begin='0ns', end='100ns', session='', output_format='kout'	列出信号在指定时间范围内的跳变。
kverif_wave_generate_rc	设计/波形	config_path, rc_path, session='', output_format='kout'	根据配置生成 Verdi RC 文件。
kverif_design_trace_driver	设计/波形	signal, time='0ns', session='', output_format='kout'	结合指定时刻的波形，查找当时真正生效的信号驱动。

LSF 自检

命令

```
/home/host/kverif/tools/kverif-lsf-doctor
```

示例输出

```
kverif_lsf_doctor:
  python_version: 3.11.9
  mcp_sdk_import: ok
  mode: direct
  process_started: true
  actions_ok: true
  quit_ok: true
```

脚本怎样调用 KVerif

二次开发脚本只需要运行 tools 目录中的命令，然后检查退出码并读取标准输出中的 JSON。不需要导入 KVerif 的 Python 包，也不需要 source 内部 Tcl，更不要直接调用 NPI 函数。下面四个示例都会先运行工具，再读取结果并给出脚本自己的判断。

建议保存这些信息

建议保存实际命令、工具返回的原始 JSON、脚本最后的判断、输入文件哈希和工具版本。只要出现 ok=false、truncated=true、error，或关键字段缺失，脚本就不应输出 PASS。

Bash: 检查波形是否正常

脚本

```
#!/usr/bin/env bash
set -euo pipefail
KDEBUG=/home/host/kverif/tools/kdebug
OUT=/proj/out/health; mkdir -p "$OUT"
"$KDEBUG" --json action signal.scan \
  --fsdb /proj/out/waves.fsd \
  --arg signal=top.u_dut.ready --arg begin=0ns --arg end=1us \
  --max-rows 500 > "$OUT/tool.json"
python3 - "$OUT/tool.json" "$OUT/conclusion.json" <<'PY'
import json, sys
r=json.load(open(sys.argv[1])); s=r.get("summary", {})
ok=r.get("ok") is True and not s.get("truncated")
status="HEALTHY" if ok and s.get("change_count",0)>0 and s.get("unknown_count",0)==0 else "REVIEW"
json.dump({"status":status,"evidence":s},open(sys.argv[2],"w"),sort_keys=True)
raise SystemExit(0 if status=="HEALTHY" else 3)
PY
```

脚本得到的结果

```
status=HEALTHY; evidence.change_count=14; evidence.unknown_count=0
```

csh: 判断是否还有未覆盖项

脚本

```
#!/bin/csh -f
set KCOV=/home/host/kverif/tools/kcov
set OUT=/proj/out/cov_holes.json
$KCOV cov-holes --vdb /proj/out/simv.vdb --metrics branch \
  --max-items 200 --json >! $OUT
python3 -c 'import json,sys; r=json.load(open(sys.argv[1])); n=r["summary"].get("matched_count",0); print("CLOSED" if n==0 else "
OPEN:%d"%n); sys.exit(0 if n==0 else 3)' $OUT
exit $status
```

脚本得到的结果

```
OPEN:7
exit_code=3
```

Perl: 多时刻批量采样

脚本

```
#!/usr/bin/env perl
use strict; use warnings; use JSON::PP qw(decode_json encode_json);
my $k='/home/host/kverif/tools/kdebug';
my @cmd=($k,'--json','value-batch','--fsdb','/proj/out/waves.fsd',
  '--signal','top.req','--signal','top.ack','--time','100ns');
open my $fh, '-|', @cmd or die $!; local $/; my $raw=<$fh>; close $fh;
my $r=decode_json($raw); die encode_json($r->{error}) unless $r->{ok};
my $v=$r->{data}{values};
my $status=($v->{'top.req'} eq "1'b1" && $v->{'top.ack'} eq "1'b1")
  ? 'TRANSFER' : 'WAIT';
print encode_json({status=>$status,time=>'100ns',evidence=>$v}),"
";
```

脚本得到的结果

```
{"status":"TRANSFER","time":"100ns","evidence":{"top.req":"1'b1","top.ack":"1'b1"}}
```

Python: 模块连接完整性

脚本

```
#!/usr/bin/env python3
import json, subprocess
k='/home/host/kverif/tools/kdebug'
cmd=[k,'--json','action','module.inspect','--daidir','/proj/out/simv.daidir',
  '--arg','module=top.u_dut','--arg','sections=["ports","io","instances"]']
r=json.loads(subprocess.run(cmd,check=True,text=True,capture_output=True).stdout)
if not r.get('ok'): raise RuntimeError(r.get('error'))
ports=r['data']['sections']['ports']
unconnected=[x['object']['full_name'] for x in ports
  if not any(x.get('connections',{}).values())]
conclusion={'status':'PASS' if not unconnected else 'OPEN_PORTS',
  'module':'top.u_dut','unconnected':unconnected}
print(json.dumps(conclusion,sort_keys=True))
raise SystemExit(0 if not unconnected else 3)
```

脚本得到的结果

```
{"module":"top.u_dut","status":"OPEN_PORTS","unconnected":["top.u_dut.debug_o"]}
```

几个常用的组合用法

要解决的问题	操作顺序	脚本最后得到什么
定位波形问题	session.open -> signal.scan -> trace.active_driver_chain -> source.context	第一次异常的时间、当时生效的驱动链和源码位置
检查模块连线	module.inspect -> module.objects(kind=instances/ports/parameters) -> trace.graph	未连接端口、实例定义、实际参数值和依赖图
推进覆盖率	kcov open -> cov.holes -> source.map -> 回归 -> cov.summary -> close	未覆盖项、对应源码，以及回归前后的覆盖率变化
分析协议性能	axi.query -> axi.channel_stall -> axi.latency_outlier -> axi.outstanding_timeline	慢事务、发生阻塞的通道和未完成事务峰值
从日志找到源码	kloc stats -> kloc resolve/context -> kdebug source.context	重复最多的报错位置和附近源码
检查断言	ksva scan/lint -> explain -> parse timeline-ir	不支持的语法、断言要求的时序关系和结构化结果

错误处理、安全与验证

脚本怎样判断成功还是失败

检查位置	成功时	失败或不完整时	脚本应该怎么做
命令退出码	exit 0	exit 1/2/3	保存 stdout 和 stderr，停止后续成功判断。
JSON 的 ok	ok=true	ok=false + error.code	按 error.code 判断是重试、补输入，还是修正参数。
结果完整性	truncated=false	truncated=true	需要完整清单时提高数量上限，或改为导出文件。
会话状态	ready/healthy	SESSION_LOST	运行 session.doctor 检查；必要时重新打开会话。
许可证	feature available	LICENSE_UNAVAILABLE	记录为测试环境问题，不能据此写成功能通过。

凭据与日志

- API key、许可证密钥和 token 只通过运行时环境传入，不写进请求、命令记录、报告或 Git。
- 标准输出 stdout 只放程序结果；诊断信息写到 stderr，以及 ~/.kdebug、~/.kverif 下的日志文件。
- 上传测试材料前检查命令、环境信息和进程信息，删除真实凭据和内部服务器地址。
- 写操作必须指定独立的 output_dir 或 output_file；text.replace_line、dm.add_net、dm.clone_module 不得覆盖原始 RTL。
- KCov 绝对导出路径必须显式 --allow-absolute-path；含 .. 的相对路径应拒绝。

Verdi 2018 VM 验证结果

指标	结果	解释
执行用户	host	普通用户
Verdi	Verdi_O-2018.09-SP2	KDebug 通过它执行 Tcl NPI
独立测试项	36	其中 module.objects 按 15 种 kind 分别测试
操作执行次数	360	不包含重新打开生成文件的检查
PASS	340	功能检查通过
LICENSE_BLOCKED	20	power.resolve/list 缺 PowerAwareAnalysis
非预期失败	0	0
命令行调用次数	370	测试只运行 tools 目录中的命令，没有直接调用 Tcl/NPI

已执行压测命令

```
KDEBUG_STRESS_ITERATIONS=10 KDEBUG_STRESS_PARALLEL=2 KVERIF_HOME=/home/host/kverif_npi_candidate_20260728_full
KDEBUG_BIN=/home/host/kverif_npi_candidate_20260728_full/tools/kdebug bash
/home/host/kverif_npi_candidate_20260728_full/kdebug/tests/vm/npi_actions/stress.sh /home/host/kverif_npi_action_stress_20260729
```

Power 测试说明

除 Power 外，20 项新增 NPI 操作都通过了 10 轮功能测试。Power 两项能够正常启动并准确报告许可证缺失，但这不能证明 Power 查询功能已经通过。获得 PowerAwareAnalysis 许可证后仍要重新测试。

附录

环境变量速查

变量	含义
KVERIF_HOME	KVerif 仓库或部署根目录。
PATH	应包含 \$KVERIF_HOME/tools。
VERDI_HOME	Verdi 安装根目录。
NPIL1_PATH	可选；Tcl NPI L1 目录，wrapper 可从 VERDI_HOME 推导。
LD_LIBRARY_PATH	应包含 Verdi NPI LINUX64 library。
SNPSLMD_LICENSE_FILE	Synopsys license 配置；文档和日志不得记录真实凭据。
KDEBUG_TRANSPORT	新 session 默认 transport：uds、tcp 或 file。
KDEBUG_FILE_TRANSPORT_TIMEOUT_MS	file transport 请求等待时限。
KDEBUG_FILE_KEEP_HISTORY	0 时删除成功请求历史；默认保留。
KVERIF_OUTPUT	设为 json 时让 KDebug 默认输出 JSON。
KVERIF_KCOV_TCL_TIMEOUT_SEC	KCov Tcl 调用时限；默认 0 表示无限等待。
KVERIF_KCOV_STARTUP_TIMEOUT_SEC	KCov session startup 时限；默认 0。
KVERIF_KCOV_REQUEST_TIMEOUT_SEC	KCov query 时限；默认 0。
KVERIF_MCP_BACKEND	direct 或 lsf。
KVERIF_MCP_ENABLE_DEBUG	控制 MCP 是否注册 KDebug 工具。
KVERIF_MCP_ENABLE_COV	控制 MCP 是否注册 KCov 工具。
KVERIF_MCP_ENABLE_BIT	控制 MCP 是否注册 KBit 工具。
KVERIF_MCP_ENABLE_ENTRY	控制 MCP 是否注册 KEntry 工具。
KVERIF_MCP_ENABLE_LOC	控制 MCP 是否注册 KLoc 工具。
KVERIF_MCP_ENABLE_CONTEXT	控制 MCP 是否注册 KBerif 只读工具。
KVERIF_MCP_ENABLE_SVA	控制 MCP 是否注册 KSVa 工具。
KVERIF_MCP_ENABLE_CONTEXT_WRITE	控制 MCP 是否注册 KBerif 写工具。
KVERIF_LSF_SESSION_QUEUE	LSF session 使用的队列。
KVERIF_LOOP_BACKEND	Loop 的启动方式：direct 表示本机运行，lsf 表示提交到 LSF。
KVERIF_LOOP_SOCKET	Loop 客户端与服务通信使用的 Unix socket 路径。
PYTHON	覆盖 wrapper 选择的 Python。
KBERIF_PYTHON	覆盖 KBerif Python 3.11 runtime。

常见状态和术语

术语	定义
design	这项操作需要 simv.daidir、filelist，或已经加载设计的 session。
waveform	这项操作需要 FSDB，或已经加载波形的 session。
combined/any	combined 表示同时需要设计和波形；any 表示两者至少提供一种。
stable	这项操作已经稳定，自动测试会检查其主要行为不被意外改坏。
experimental	功能已经实现，但字段或行为以后仍可能调整。
deprecated	只为兼容旧脚本而保留，新脚本不要再使用。
KOUT	KVerif 默认使用的、便于人在终端阅读的结构化文本。
标准示例	仓库中与字段定义配套的请求和返回示例，只用于说明格式，不代表你的项目实测值。
只调用命令行	二次开发脚本只运行 tools 目录中的可执行文件，不直接调用内部 Tcl、NPI 或 Python 模块。

KDebug 操作字母索引

操作名	分类	状态	需要的输入	用途
actions	builtin	stable	none	列出 KDebug 当前支持的操作，并显示每项操作的状态和所需输入。
apb.config.list	waveform	stable	waveform	列出当前波形会话中的 APB 协议配置。
apb.config.load	waveform	stable	waveform	加载并命名一组 APB 信号映射。
apb.cursor	waveform	stable	waveform	在命名 APB 事务结果上移动或读取游标。
apb.query	waveform	stable	waveform	从 FSDB 中解码 APB 传输并按地址、方向或时间过滤。
apb.transfer_window	waveform	experimental	waveform	围绕指定 APB 传输提取前后波形窗口和关键信号。
axi.analysis	waveform	stable	waveform	统计 AXI 事务的吞吐、延迟、阻塞时间和未完成事务数量。
axi.channel_stall	waveform	experimental	waveform	找出 AXI 各通道中 valid 已拉高但 ready 尚未响应的时间段。
axi.config.list	waveform	stable	waveform	列出当前波形会话中的 AXI 协议配置。
axi.config.load	waveform	stable	waveform	加载并命名 AXI channel 信号映射。
axi.cursor	waveform	stable	waveform	在命名 AXI 查询结果上移动或读取游标。
axi.export	waveform	stable	waveform	把 AXI 事务或分析结果导出为结构化文件。
axi.latency_outlier	waveform	experimental	waveform	找出延迟明显偏高的 AXI 事务，并返回相关通道信息。
axi.outstanding_timeline	waveform	experimental	waveform	列出 AXI 未完成事务数量随时间的变化。
axi.query	waveform	stable	waveform	从 FSDB 中解码并筛选 AXI 读写事务。
axi.request_response_pair	waveform	experimental	waveform	配对 AXI 请求与响应，展示端到端事务关系。
batch	builtin	stable	none	一次提交多项 KDebug 操作，按顺序执行并分别返回结果。
control.explain	design	stable	design	解释控制信号的组合条件、候选驱动和来源位置。
counter.explain	design	stable	design	从设计结构解释计数器的更新条件、保持条件和宽度。
counter.statistics	waveform	stable	waveform	按时钟和 valid 条件统计计数值的最小、最大、平均和样本数。
crdb.correlates	design	experimental	none	在 CRDB 中查找与指定 RTL 信号或门级信号对应的其他层级对象。
crdb.resolve	design	experimental	none	在 CRDB 中查找一个 RTL 或门级对象，并返回名称、层级和位宽等信息。
cursor.delete	waveform	stable	waveform	删除一个命名波形游标。
cursor.get	waveform	stable	waveform	读取当前或指定波形游标的位置。
cursor.list	waveform	stable	waveform	列出会话中的全部命名波形游标。
cursor.set	waveform	stable	waveform	在指定时间设置或更新命名波形游标。
cursor.use	waveform	stable	waveform	把一个命名游标设为后续查询的当前游标。
detect_anomaly	waveform	stable	waveform	对一组信号检测未知值、异常跳变和长期不活动等现象。
dm.add_net	design	experimental	design	在设计副本中新增一条 net，并把修改后的 RTL 写到新目录。原文件不会被覆盖。
dm.clone_module	design	experimental	design	复制一个模块并改名，把结果写到新目录。原模块不会被覆盖。
event.config.list	waveform	stable	waveform	列出当前会话中的命名事件表达式配置。
event.config.load	waveform	stable	waveform	加载并命名可复用的波形事件表达式。
event.export	waveform	stable	waveform	把匹配事件的时间点和上下文导出到文件。
event.find	waveform	stable	waveform	查找表达式第一次、最后一次或全部匹配事件。
expr.eval_at	waveform	stable	waveform	在指定波形时间计算逻辑表达式。
expr.normalize	design	stable	none	把表达式统一成固定写法，便于比较两个表达式、搜索和继续分析。
fsdb.writer.create_scope	waveform	experimental	none	通过 Tcl NPI FSDB Writer 创建层次、信号和值并写出新 FSDB。
fsm.explain	design	stable	design	列出状态机的状态和转移条件，并标出默认路径及对应源码位置。
handshake.inspect	waveform	stable	waveform	检查 valid/ready 握手的传输、等待和 stall 行为。

操作名	分类	状态	需要的输入	用途
inspect_signal	waveform	deprecated	waveform	保留给旧脚本使用的单信号检查入口；新脚本建议使用 signal.scan 等操作。
instance.map	design	stable	design	查找一个实例，并返回它的完整层次路径、模块定义和相关对象。
interface.resolve	design	stable	design	解析 SystemVerilog interface 实例及其端口、modport 和连接信息。
language.iterate	design	experimental	design	在指定模块或层次下，按类型列出 NPI 设计对象。
language.relate	design	experimental	design	读取一个 RTL 对象与另一个对象之间的指定关联，例如端口的上层连接。
language.resolve	design	experimental	design	按名称查找一个 RTL 对象，并返回类型、完整路径、源码位置等属性。
language.value	design	experimental	design	读取 parameter 或 constant 在编译展开后的实际值。
list.add	waveform	stable	waveform	向命名信号列表追加信号。
list.create	waveform	stable	waveform	创建可复用的命名信号列表。
list.delete	waveform	stable	waveform	删除命名信号列表。
list.diff	waveform	stable	waveform	比较命名列表在两个时间点的信号值差异。
list.show	waveform	stable	waveform	列出信号列表或展示指定列表内容。
list.validate	waveform	stable	waveform	验证命名列表中的信号是否存在并可读取。
list.value_at	waveform	stable	waveform	批量读取命名列表在指定时间的值。
module.find_instances	design	experimental	design	根据模块定义名查找全部层次实例。
module.inspect	design	experimental	design	一次查看模块实例的参数、端口、连接和常用内部对象。
module.objects	design	experimental	design	用 kind 选择对象类别，列出模块中的实例、端口、parameter、过程块等 15 类对象。
netlist.iterate	design	experimental	design	列出编译展开后的网表对象，例如 net、cell 或位切片。
netlist.resolve	design	experimental	design	按名称查找编译展开后的 net、port 或 cell。
npi.capabilities	builtin	experimental	none	检查当前 Verdi 是否提供 KDebug 需要的 Tcl NPI 命令。许可证是否可用仍要单独测试。
port.trace	design	stable	design	从端口出发追踪上下游连接、方向和驱动负载关系。
power.list	design	experimental	design	按类型列出电源域、供电网络等 Power NPI 对象。
power.resolve	design	experimental	design	查找一个 Power NPI 对象，并返回它的名称、类型和关键属性。
procedural.assignment	design	stable	design	查询信号在 always、initial、task 等过程块中的赋值位置。
rc.generate	waveform	stable	waveform	根据 position 简写映射和配置生成可复用的 Verdi RC 文件。
sampled_pulse.inspect	waveform	experimental	waveform	按采样时钟检查脉冲宽度、丢采样和边界行为。
schema	builtin	stable	none	查看某项操作允许输入哪些 JSON 字段，以及会返回哪些字段。
scope.list	waveform	stable	waveform	列出 FSDB 层次 scope 和可选信号。
sequential.update	design	stable	design	解释寄存器或时序变量的时钟、复位、使能和更新表达式。
session.close	session	stable	session	正常关闭一个会话并释放后端进程和临时资源。
session.doctor	session	stable	session	检查会话记录、后台进程、通信方式和输入文件是否正常。
session.gc	session	stable	none	清理过期、失效或遗留的会话状态。
session.kill	session	stable	session	强制终止失去响应或无法正常关闭的会话。
session.list	session	stable	session	列出当前用户打开的 KDebug 会话，以及每个会话加载了哪些文件。
session.open	session	stable	any	打开可复用的设计、波形或联合调试会话，避免每次重新加载大文件。
signal.canonicalize	design	stable	design	把用户输入的信号简写解析成标准完整层次名。
signal.changes	waveform	stable	waveform	返回信号在时间窗口内的跳变序列。
signal.resolve	design	stable	design	解析设计信号并返回候选、类型、宽度和源码信息。
signal.scan	waveform	stable	waveform	一次汇总信号的首末值、跳变、未知值和可选样本。
signal.stability	waveform	stable	waveform	检查信号在给定窗口是否保持稳定或违反稳定约束。
signal.statistics	waveform	stable	waveform	统计信号活动度、高低周期、变化次数和采样方式。

操作名	分类	状态	需要的输入	用途
signal.trend	waveform	stable	waveform	把信号值按时间分段，提取上升、下降或稳定趋势。
source.context	design	stable	none	显示源码目标行及其前后几行，便于确认问题所在位置。
stream.config.list	waveform	stable	waveform	列出已经加载的数据流和数据包配置。
stream.config.load	waveform	stable	waveform	加载数据流、通道和数据包字段定义。
stream.export	waveform	stable	waveform	导出解码后的数据包、字段和它们之间的关系。
stream.query	waveform	stable	waveform	按字段、方向、时间等条件查询数据包。
stream.show	waveform	stable	waveform	查看数据流、通道和字段布局。
stream.validate	waveform	stable	waveform	检查数据流配置是否完整，配置中的信号能否在波形中找到。
text.line	design	experimental	design	通过 Tcl NPI 读取指定的源码行。
text.replace_line	design	experimental	design	在输出副本中替换指定源码行，不原地修改输入设计。
text.words	design	experimental	design	把指定源码行拆成单词和符号，并返回每项的 NPI 属性。
trace.active_driver	combined	stable	any	结合指定时刻的波形值，判断当时究竟是哪一条赋值在驱动目标信号。
trace.active_driver_chain	combined	stable	any	从目标信号开始逐级向上查找当时生效的驱动，并列出影响选择结果的控制条件。
trace.driver	design	stable	design	查询信号的静态驱动、赋值表达式和可选源码证据。
trace.expand	design	stable	design	展开一个信号的表达式依赖，补充候选和源码细节。
trace.explain	design	stable	design	把查到的驱动和依赖关系整理成一段容易阅读的原因说明。
trace.graph	design	stable	design	生成信号之间的依赖关系图，便于脚本继续做连线分析。
trace.load	design	stable	design	查询信号的静态负载及其使用位置。
trace.path	design	stable	design	搜索两个设计信号之间的依赖路径。
trace.query	design	stable	design	按给定条件查询静态驱动或负载关系。
transaction.writer.create	waveform	experimental	none	根据事务列表生成 transaction FSDB，并写入事务之间的父子关系。
value.at	waveform	stable	waveform	读取一个信号在指定 FSDB 时间的值。
value.batch_at	waveform	stable	waveform	在同一时间批量读取多个信号，减少重复 FSDB 访问。
vcs.summary	design	experimental	design	查看 VCS 版本、编译参数、错误数量和已加载模块数量。
verify.conditions	waveform	stable	waveform	在一个时间点验证多个信号或表达式条件。
window.verify	waveform	stable	waveform	在时间窗口内验证条件持续成立、出现次数或时序关系。

本手册包含哪些内容

本版覆盖

11 类对外工具、106 项可用的 KDebug 操作、全部 KDebug 快捷命令和 key=value 通用参数；22 条 KCov 命令，以及 KBit、KEntry、KLoc、KSVA、KBerif、KEDA Runner、Loop 客户端的命令；46 个 MCP 工具的参数；8 个按任务编排的 NPI 教程；22 项 NPI 操作和 15 类模块对象；以及 Bash、csh、Perl、Python 调用示例。

本手册根据当前仓库内容自动生成。修改操作字段或命令行解析逻辑后，应重新生成 PDF，并再次检查目录链接、内容是否齐全、文字是否越界，以及每一页能否正常显示。